

Kinetic-KAN: Solver-Aware Neural Dynamics

Project Final Report · Group A_08, Section A

Nawriz Ahmed Turjo

Student ID 2105032

Bangladesh University of
Engineering and Technology
Dhaka, Bangladesh

Abhishek Roy

Student ID 2105033

Bangladesh University of
Engineering and Technology
Dhaka, Bangladesh

Monjur Hossain Khan

(Shovon)

Student ID 2105043
Bangladesh University of
Engineering and Technology
Dhaka, Bangladesh

Shams Hossain Simanto

Student ID 2105048

Bangladesh University of
Engineering and Technology
Dhaka, Bangladesh

Md. Abrar Jahin

Student ID 2105055

Bangladesh University of
Engineering and Technology
Dhaka, Bangladesh

Abstract

Kolmogorov-Arnold Network ODEs (KAN-ODEs) replace the multilayer perceptron inside a Neural ODE with a network whose every edge is a learnable univariate function. We reproduce the Lotka-Volterra benchmark of the KAN-ODE base paper in PyTorch, implementing six explicit Runge-Kutta integrators and seven basis families from their defining formulas and verifying every integrator’s order to round-off. We then treat the paper’s three fixed design choices (integrator, basis and gradient method) as variables, extend the recipe to a damped pendulum, an SIR epidemic model and a non-mechanistic outbreak curve, and run follow-on studies on adjoint sensitivity, gradient dynamics, a learnable basis blend, solver stiffness and sparse symbolic regression. Above order two, the solver does not change accuracy: Midpoint matches Tsit5 within 0.3% at 29% of the cost. Networks learn the field their solver needs (the Euler-trained model matches the inverse modified equation), and every trained model, KAN or MLP, is an attracting limit cycle rather than the true neutral centre. At 10^4 epochs KAN-ODE extrapolates $5.4\times$ better than a parameter-matched MLP, but at 5×10^4 epochs the MLP is $6.8\times$ better. B-spline and RBF bases are tied, while global polynomials extrapolate $4\text{--}5\times$ worse. SIR’s training failure traces to a $19\times$ unit-scale mismatch at initialization; time rescaling, a conservation projection and a vanishing-field gate move extrapolation R^2 from -20.97 to $+0.998$. All results are single-seed.

Keywords

Kolmogorov-Arnold networks, neural ordinary differential equations, Runge-Kutta methods, basis functions, numerical stability, extrapolation

1 Introduction and Base Paper Review

A Neural ODE [3] learns an unknown dynamical system $\dot{\mathbf{u}}(t) = \mathbf{F}(\mathbf{u}(t))$ by fitting a vector field $\mathbf{f}_\theta(\mathbf{u}(t))$ and integrating it numerically. The usual choice for $\mathbf{f}_\theta$ is a multilayer perceptron (MLP), with fixed activations on the nodes and all learnable structure in

the linear weights, $\mathbf{y} = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b})$. A Kolmogorov-Arnold Network (KAN) [10] flips this around: every *edge* carries its own learnable univariate function $\phi(x)$, and each node just sums its incoming edges, $y_j = \sum_i \phi_{i,j}(x_i)$. The idea comes from the Kolmogorov-Arnold representation theorem, which says that any continuous multivariate function on a bounded domain can be written as a finite composition of continuous univariate functions and addition,

$$f(\mathbf{x}) = \sum_{q=1}^{2n+1} \Phi_q \left(\sum_{p=1}^n \phi_{q,p}(x_p) \right), \quad (1)$$

so no nonlinearity has to be fixed in advance; the network learns the shape of each univariate function. In practice each edge function is a fixed smooth residual plus a learnable expansion in a chosen basis. In the form we implement,

$$\phi(x) = w_b b(x) + \sum_{i=1}^G c_i B_i(\text{norm}(x)), \quad (2)$$

with $b(x) = \text{silu}(x) = x/(1 + e^{-x})$ and $\text{norm}(x) = \tanh x$. Here $B_1, \dots, B_G$ are G basis functions attached to a uniform grid of knots $z_g \in [-1, 1]$ with spacing $h = 2/(G - 1)$, the c_i are trainable coefficients, and w_b weights the residual. (The original pykan form carries $G + k$ spline functions and a separate scale w_s on the expansion; with one coefficient per basis function that scale is redundant, so we drop it.) The choice of B_i is a design decision rather than a mathematical necessity. A Gaussian radial basis function (RBF) grid, $B_i(u) = \exp(-(u - z_i)^2/2h^2)$, gives each edge smooth support that decays quickly but never reaches zero. A degree- k B-spline built by the Cox-de Boor recursion instead gives each edge compact local support, exactly zero outside its own knot span.

1.1 The base paper: KAN-ODEs

Koenig et al. [8] put this construction inside a Neural ODE’s vector field, $\dot{\mathbf{u}} = \mathbf{f}_\theta(\mathbf{u})$ with every edge of $\mathbf{f}_\theta$ built from Equation 2, and obtain the trajectory by numerically integrating that field forward in time. On the Lotka-Volterra predator-prey system they

report two central claims: a KAN-ODE with 240 parameters extrapolates the system’s oscillation accurately from a short training window, and it reaches a target training loss in roughly 10^4 epochs where a parameter-matched (252-parameter) MLP-based Neural ODE needs 10^5 , a tenfold training-speed advantage. The paper also applies KAN-ODEs to other systems, including inferring hidden physics; we focus on its Lotka-Volterra benchmark.

To get these results the paper commits to three design decisions, which we treat as free variables:

- (1) the **integrator**: an adaptive fifth-order Runge-Kutta method, Tsit5 [12];
- (2) the **basis**: Gaussian RBF;
- (3) the **differentiation method** used to get gradients through the integrator: continuous adjoint sensitivity, chosen for its constant memory cost in trajectory length.

The basis choice is not something we invented to vary. The original KAN paper [10] used cubic B-splines by default, and the base paper switched to Gaussian RBF for the ODE setting because its derivative never vanishes, which matters once the edge function sits inside a differentiated integrator rather than a static regression. We restore B-spline and add five further bases, so the trade-off behind that switch is measured rather than assumed.

1.2 Why training needs to be studied, not just the final error

Besides Lotka-Volterra, we use two textbook systems, the SIR epidemic model and a damped pendulum, chosen because their governing equations are simple and well understood. Any training failure can then be blamed on the architecture or the optimization rather than on ambiguous target dynamics. The integrator’s interaction with training is not predictable from classical numerical analysis alone, because the integrator sits inside a backpropagation graph rather than a standalone forward solve: a solver that is perfectly stable in the classical sense can still train badly, train to a wrong answer, or fail to generalize. Section 4.5 makes this concrete. A network trained through Forward Euler learns the *inverse modified equation* of backward error analysis rather than the physical field, and every trained model, KAN or MLP, replaces Lotka-Volterra’s family of neutral orbits with a single attracting limit cycle. Diagnosing behaviour like this means treating training itself as an object of study.

1.3 Summary of our work

Table 1 lists every study we ran, why we ran it, what we found, and where it is reported. In short, our objectives were to: (i) reproduce the base paper’s Lotka-Volterra benchmark with every integrator and basis implemented and verified from its defining formula; (ii) vary each of the paper’s three fixed design choices and measure what it actually costs or buys; (iii) check whether the recipe transfers to other dynamical systems, and fix it where it does not; and (iv) test whether our conclusions survive a five times larger training budget.

The rest of the paper follows the course’s required structure. Section 2 gives the numerical methods: the KAN-ODE model, the integrators, the basis families, the two gradient methods, the hybrid basis and sparse regression. Section 3 describes the datasets,

training recipe, evaluation protocol and compute budget. Section 4 reports each study in the order of Table 1, compares our findings with the base paper, and discusses the patterns that recur across studies. Section 5 concludes with limitations and future work.

2 Methodology

2.1 The KAN-ODE model

Every KAN-ODE we train has the same architecture: a two-layer network with widths $[2, 10, 2]$, each edge carrying the expansion of Equation 2 over a grid of $G = 5$ basis functions. It is wrapped inside an explicit Runge-Kutta integrator that advances the state and hands the resulting trajectory to the loss. Figure 1 shows the full training loop and marks the three components we vary.

2.1.1 One KAN layer as two matrix products. The network is two KDense layers. For a batch $\mathbf{X} \in \mathbb{R}^{B \times d_{\text{in}}}$, a layer evaluates every edge function of Equation 2 at once:

$$\Psi_{b,i,g} = B_g(\tanh X_{b,i}), \quad \mathbf{Y} = \underbrace{\Psi_{\text{flat}} \mathbf{C}^\top}_{\text{spline branch}} + \underbrace{\text{silu}(\mathbf{X}) \mathbf{W}^\top}_{\text{base branch}}, \quad (3)$$

with $\mathbf{C} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}} G}$ and $\mathbf{W} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$. Here $\Psi_{\text{flat}} \in \mathbb{R}^{B \times d_{\text{in}} G}$ is Ψ with its last two axes merged. Entry $C_{j,(i,g)}$ is the coefficient c_g of edge $i \rightarrow j$ and $W_{j,i}$ is that edge’s residual weight w_b , so $Y_{b,j} = \sum_i \phi_{i,j}(X_{b,i})$ exactly. Figure 2 traces the tensor shapes. The forward pass is short enough to show in full:

```

Forward pass of one KDense layer

x_flat = x.view(-1, self.in_features)      # [B, d_in]
x_norm = self.normalizer(x_flat)           # tanh -> [-1, 1]
basis_vals = self.basis_func(x_norm, self.grid, self.denominator)
basis_flat = basis_vals.reshape(basis_vals.size(0),
                                self.in_features * self.grid_len)
spline_out = F.linear(basis_flat, self.C)    # Psi_flat @ C^T
if self.use_base_act and self.W is not None:
    base_val = self.base_act(x_flat)         # silu(x)
    base_out = F.linear(base_val, self.W)    # silu(x) @ W^T
    out = spline_out + base_out
else:
    out = spline_out
return out.view(*orig_shape[:-1], self.out_features)

```

Parameter count. Each layer holds $d_{\text{out}} d_{\text{in}} G$ spline coefficients, $d_{\text{out}} d_{\text{in}}$ residual weights and no bias. For $[2, 10, 2]$ with $G = 5$, layer one ($2 \rightarrow 10$) has $10 \cdot 2 \cdot 5 + 10 \cdot 2 = 120$ parameters and layer two ($10 \rightarrow 2$) has $2 \cdot 10 \cdot 5 + 2 \cdot 10 = 120$, for the 240 total the base paper reports. The comparison MLPs of Section 4.1 are sized to match: the paper’s $[2, 50, 2]$ network has $2 \cdot 50 + 50 + 50 \cdot 2 + 2 = 252$ parameters, and the deeper $[2, 14, 8, 8, 2]$ network we use as the working baseline also has $(28+14)+(112+8)+(64+8)+(16+2) = 252$.

Why the efficient-kan formulation. The reference implementation, pykan [10], represents every edge with its own spline object and recomputes the grid during training. Inside a Neural ODE that cost is multiplied by the integrator: one epoch of Tsit5 on the training window calls $\mathbf{f}_\theta$ 420 times, and reverse-mode differentiation keeps every one of those calls in the autograd graph. Our layer follows the efficient-kan reformulation [1] instead: the knots are a fixed buffer, the basis is evaluated for all edges with one broadcast, and the contraction is one dense matrix product per branch

Table 1: Summary of the studies in this report.

#	Study	Why we ran it	What we observed	Sec.
1	Baseline reproduction: KAN-ODE vs. MLP-ODE	Test the base paper’s two headline claims (accuracy and training speed) at matched parameter count.	KAN-ODE extrapolates 5.4× better at 10^4 epochs, a gap explained by a 48× smaller period error. The MLP trains 4.1× <i>faster</i> . The paper’s literal tanh MLP trains only under the paper’s own learning rate and initialization.	4.1
2	Integrator ablation	The paper fixes Tsit5; does solver order matter for training?	All six solvers pass their order conditions to round-off. Above order two, accuracy is flat: Midpoint matches Tsit5 within 0.3% at 29% of the cost. The Euler model learns the inverse modified equation, and no model transfers across solver families.	4.2
3	Basis-function ablation	The paper switched from B-spline to RBF; is that justified?	B-spline and RBF are tied, with RBF 3.9× cheaper. Global polynomials extrapolate 4–5× worse. Locality, not conditioning, separates the bases.	4.3
4	Step-size and noise sensitivity	How sensitive is the fit to sampling density and observation noise?	Step size acts as a data-density choice (non-monotone). Extrapolation MSE scales as $12.9 \sigma^{2.02}$ with noise.	4.4
5	What the trained fields learned	Does low trajectory error mean the right vector field?	No. Every model is an attracting limit cycle around a spurious unstable equilibrium, accurate only in a thin tube around the training orbit.	4.5
6	Cross-domain failures and fixes	Does the Lotka-Volterra recipe transfer to the pendulum and SIR?	A longer training window fixes the pendulum. SIR’s 19× scale mismatch at initialization is fixed by time rescaling and a conservation projection; adding a vanishing-field gate gives $R^2: -20.97 \rightarrow +0.998$. The KAN advantage widens to 40.6× at twice the horizon.	4.6
7	Adjoint vs. direct autograd	The paper uses the adjoint for its memory saving; does that matter at our scale?	The adjoint uses less memory at every length tested but is 1.9–2.1× slower, so we use direct autograd.	4.7
8	Gradient-norm dynamics	Is Euler’s convergence floor caused by noisier gradients?	No ($\rho = -0.088$, $p = 0.87$); gradient spike bursts are shared across all solvers.	4.8
9	Learnable hybrid basis	Can the network learn its own B-spline/RBF blend?	The gate works as designed but never beats the better pure basis, and it overfits on the pendulum past 3000 epochs.	4.9
10	Stiffness–solver map	Do explicit solvers fail as the pendulum’s damping grows?	Zero divergences, as linear stability predicts. The failures trace to the random seed and a spurious learned equilibrium.	4.10
11	Sparse regression and epidemic transfer	How does KAN-ODE compare with SINDy under noise, and do the SIR fixes transfer to a non-mechanistic outbreak curve?	KAN-ODE is more accurate at every noise level, but degrades 1540× against SINDy’s 9×. Pruning finds no sparse structure. Two of three SIR fixes transfer.	4.11
12	Epoch-budget validation	Do the 10^4 -epoch conclusions survive five times the budget?	Solver and basis rankings hold. KAN vs. MLP <i>reverses</i> : the MLP is 6.8× better at 5×10^4 epochs.	4.12
13	Comparison with the base paper	Audit our recipe against the paper field by field.	Architecture and basis match. The KAN’s training speed is reproduced within 1.4×. The tenfold speed advantage depends on the MLP baseline’s optimizer settings.	4.13

(Equation 3). Each stage therefore adds two matrix multiplications and a handful of elementwise operations to the graph, however many edges there are. The same structure makes every basis we substitute into Equation 2 (RBF, B-spline, Chebyshev, Lagrange, Newton, IQF, RSWAF) a swappable tensor rather than a different object type.

What we implemented ourselves. We wrote the KAN layer, the seven basis functions, the six fixed-step Runge-Kutta integrators, the invariant-enforcing field wrappers used for SIR and the training loop directly in PyTorch [11], rather than importing them, so that every basis and solver could be instrumented in the same way. The integrators apply the tableaux of Section 2.2 verbatim. Tsit5 and DOPRI5 run at a fixed step with their fifth-order weights only, whereas the base paper uses Tsit5’s embedded error estimate for adaptive step control. The base paper compiles its integration loop in Julia’s SciML ecosystem; we run the same loop uncompiled in PyTorch because it was simpler for us to work with. Our wall-clock figures are therefore comparable across our own runs but not against the paper’s reported times (Section 3.4 quantifies what this costs).

2.2 Runge-Kutta integrators

Every integrator we use is an explicit Runge-Kutta method: one step of size h evaluates s stage slopes and combines them,

$$\mathbf{k}_i = \mathbf{f}_\theta\left(\mathbf{u}_n + h \sum_{j=1}^{i-1} a_{ij} \mathbf{k}_j\right), \quad \mathbf{u}_{n+1} = \mathbf{u}_n + h \sum_{i=1}^s b_i \mathbf{k}_i, \quad (4)$$

for $i = 1, \dots, s$, so a method is fully specified by its Butcher tableau $(A, \mathbf{b}, \mathbf{c})$ with $c_i = \sum_j a_{ij}$. Writing $\mathbf{u}_n$ for the state at step n , the four low-order methods are

$$\text{Euler } (p = 1): \mathbf{u}_{n+1} = \mathbf{u}_n + h \mathbf{f}_\theta(\mathbf{u}_n) \quad (5)$$

$$\text{Midpoint } (p = 2): \mathbf{u}_{n+1} = \mathbf{u}_n + h \mathbf{f}_\theta\left(\mathbf{u}_n + \frac{h}{2} \mathbf{k}_1\right) \quad (6)$$

$$\text{Heun } (p = 2): \mathbf{u}_{n+1} = \mathbf{u}_n + \frac{h}{2} (\mathbf{k}_1 + \mathbf{k}_2), \quad \mathbf{k}_2 = \mathbf{f}_\theta(\mathbf{u}_n + h \mathbf{k}_1) \quad (7)$$

$$\text{RK4 } (p = 4): \mathbf{u}_{n+1} = \mathbf{u}_n + \frac{h}{6} (\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4) \quad (8)$$

with $\mathbf{k}_1 = \mathbf{f}_\theta(\mathbf{u}_n)$ throughout and the classical RK4 stages $\mathbf{k}_2 = \mathbf{f}_\theta(\mathbf{u}_n + \frac{h}{2} \mathbf{k}_1)$, $\mathbf{k}_3 = \mathbf{f}_\theta(\mathbf{u}_n + \frac{h}{2} \mathbf{k}_2)$, $\mathbf{k}_4 = \mathbf{f}_\theta(\mathbf{u}_n + h \mathbf{k}_3)$. DOPRI5 and Tsit5 are both seven-stage 5(4) embedded pairs whose seventh stage exists only to supply the fourth-order error estimate (the “first same as last” stage); its fifth-order weight is $b_7 = 0$. At a fixed step, as here, the seventh evaluation contributes nothing to the update, so our implementation skips it and both methods cost

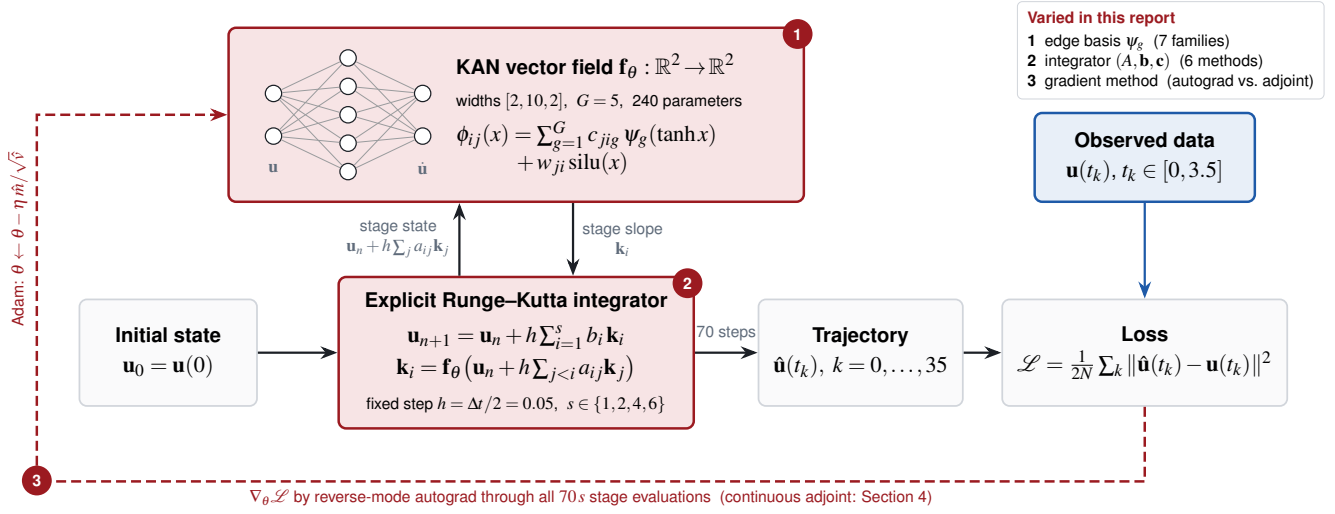


Figure 1: The KAN-ODE training loop as implemented. The integrator calls the network once per Runge-Kutta stage, so one training epoch on $[0, 3.5]$ evaluates $\mathbf{f}_\theta$ 70s times (35 sampling intervals, two sub-steps each, s stages per step). The loss gradient comes from differentiating through every one of those evaluations. Numbered badges mark the three components we vary: the edge basis (Sections 4.3 and 4.9), the integrator (Sections 4.2, 4.8 and 4.10) and the gradient method (Section 4.7).

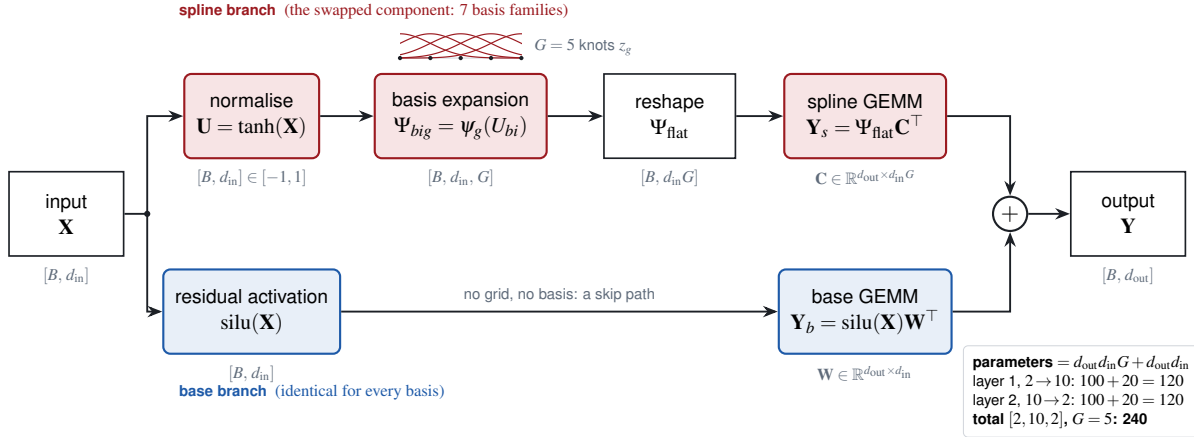


Figure 2: Tensor flow through one KDense layer (Equation 3). Only the basis-expansion step differs between the seven basis families of Section 2.3; every basis function has the same signature, $(\mathbf{U}, \mathbf{z}, h) \mapsto \Psi \in \mathbb{R}^{B \times d_{in} \times G}$, so swapping the basis changes one tensor and nothing else.

six evaluations per step. Their coefficients satisfy all 17 order conditions up to order five, and Tsitouras [12] additionally minimised the size of the sixth-order error terms. Table 2 lists the Tsit5 coefficients exactly as implemented, and Figure 3 shows where each method samples the field within one step. Every extra stage costs one more evaluation of $\mathbf{f}_\theta$, which is why we measure cost in function evaluations per epoch rather than by solver order alone.

2.3 Basis functions

Writing z_g for the g -th of G evenly spaced grid centres with spacing $h = 2/(G - 1)$, and $u = \tanh(x) \in [-1, 1]$ for the normalized edge input, the seven bases we compare are, exactly as implemented,

$$B_g^{\text{RBF}}(u) = \exp\left(-\frac{1}{2} \left(\frac{u - z_g}{h}\right)^2\right), \quad B_g^{\text{IOF}}(u) = \frac{1}{1 + \left(\frac{u - z_g}{h}\right)^2},$$

$$B_g^{\text{RSWAF}}(u) = 1 - \tanh^2\left(\frac{u - z_g}{h}\right), \quad (9)$$

Table 2: Fixed-step Tsit5 tableau as implemented (first six stages, rounded to six significant figures; the code carries 16). Row i lists c_i and $a_{i1}, \dots, a_{i,i-1}$; the last two rows list the solution weights b_i .

0					
0.161	0.161				
0.327	-0.00848066	0.335481			
0.9	2.89715	-6.35945	4.36230		
0.980026	5.32586	-11.7489	7.49554	-0.0924951	
1	5.86146	-12.9210	8.15937	-0.0715850	-0.0282691
$\mathbf{b}^\top$	0.0964608	0.01	0.479890	1.37901	-3.29007
	2.32471				

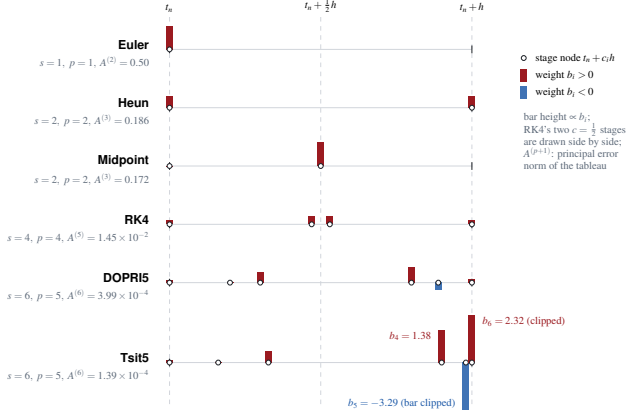


Figure 3: Stage nodes $t_n + c_i h$ and solution weights b_i of the six implemented methods, drawn from the coefficients in the code. Heun and Midpoint use the same number of evaluations but sample at opposite places: Heun averages the two ends of the step, Midpoint uses only a slope at its centre. Tsit5 reaches order five with large weights of both signs ($b_5 = -3.29$), a cancellation that is harmless in exact arithmetic and buys its small error constant.

$$B_g^{\text{Cheb}}(u) = T_{g-1}(u) = \cos((g-1) \arccos u),$$

$$B_g^{\text{Lag}}(u) = \prod_{j \neq g} \frac{u - z_j}{z_g - z_j}, \quad B_g^{\text{Newt}}(u) = \prod_{j < g} (u - z_j), \quad (10)$$

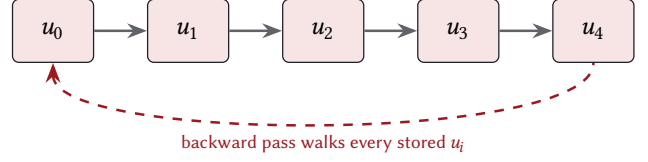
plus the cubic B-spline B_g^{spline} , built by the Cox-de Boor recursion from degree-0 indicator functions by repeated linear blending,

$$B_{g,0}(u) = \mathbf{1}[t_g \leq u < t_{g+1}],$$

$$B_{g,p}(u) = \frac{u - t_g}{t_{g+p} - t_g} B_{g,p-1}(u) + \frac{t_{g+p+1} - u}{t_{g+p+1} - t_{g+1}} B_{g+1,p-1}(u), \quad (11)$$

on the clamped knot vector $\mathbf{t} = (-1, -1, -1, -1, 0, 1, 1, 1, 1)$ for $G = 5$, $k = 3$. The clamping makes the five splines a partition of unity with $B_1(-1) = B_5(1) = 1$, and it is what gives the spline compact support: $B_{g,k}(u) = 0$ outside the $k + 2$ knots the recursion references. Gaussian RBF, IQF and RSWAF are local but never exactly zero. Chebyshev, Lagrange and Newton have no locality at all: every coefficient c_g influences $\phi(x)$ at every x . The residual term $w_b \text{silu}(x)$ in Equation 2 is the same for every basis, since it

Direct autograd: every u_i stored, $O(N_t)$ memory



Adjoint sensitivity: $O(1)$ memory, independent of N_t

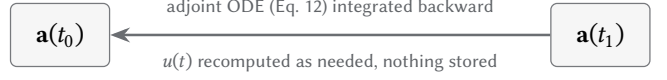


Figure 4: Direct autograd (top) keeps every intermediate state u_i in memory so the backward pass can reuse it, giving $O(N_t)$ memory. Adjoint sensitivity (bottom) instead solves the backward-time equation of Equation 12, recomputing $u(t)$ as needed, giving $O(1)$ memory regardless of N_t .

is a fixed part of the edge parameterization rather than one of the substituted terms.

2.4 Computing gradients: direct autograd and the adjoint method

There are two ways to compute gradients through a Neural ODE. **Direct autograd** treats the numerical integrator as an ordinary computational graph: each of the solver's N_t forward steps is recorded, and the reverse pass walks back through every recorded step, so peak memory grows linearly with the number of steps. **Adjoint sensitivity** [3] instead treats the forward solve as a black box and derives a second, augmented ODE that produces the same gradient without storing the intermediate states. Writing $\mathbf{a}(t) = \partial L / \partial \mathbf{u}(t)$ for the adjoint state, it satisfies its own backward-time differential equation,

$$\frac{d\mathbf{a}}{dt} = -\mathbf{a}^\top \frac{\partial \mathbf{f}_\theta}{\partial \mathbf{u}}, \quad \frac{dL}{d\theta} = \int_{t_1}^{t_0} \mathbf{a}^\top \frac{\partial \mathbf{f}_\theta}{\partial \theta} dt, \quad (12)$$

solved backward from t_1 to t_0 alongside the original state $\mathbf{u}(t)$, which is recomputed rather than stored. Memory then depends only on the size of the state and parameter vectors, not on how many steps the forward solve took: $O(1)$ in trajectory length, against direct autograd's $O(N_t)$ (Figure 4). All our training runs use direct autograd; Section 4.7 measures whether that choice is justified.

2.5 A learnable hybrid basis

Rather than choosing one basis function $B_i(x)$ in Equation 2 in advance, we can make the choice itself a trained quantity: a learnable convex blend of two candidate bases inside every edge,

$$\phi(x) = \alpha \cdot \text{Spline}(x) + \beta \cdot \text{RBF}(x), \quad (\alpha, \beta) = \text{softmax}(\text{logits}), \quad (13)$$

trained end-to-end from a neutral start ($\alpha = \beta = 0.5$) with no hand-set schedule (Figure 5). We produce the blend weights with a

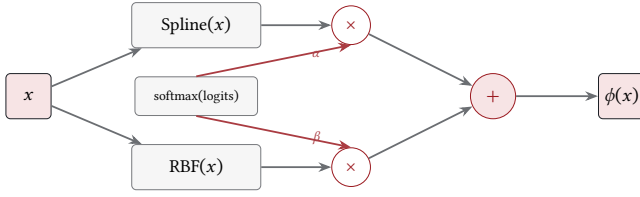


Figure 5: The hybrid-basis edge of Equation 13: every edge evaluates both bases in parallel, and a shared softmax gate produces the scalar weights α, β that multiply each basis’s output before they are summed into $\phi(x)$.

softmax over two learnable logits rather than, say, a single sigmoid-gated weight, because a softmax guarantees $\alpha + \beta = 1$ by construction. The two bases then always form a genuine convex combination, so the gate can only trade one basis off against the other instead of scaling the whole edge output up or down as a side effect. One pair of logits is shared across every edge, so the model learns a single global blend. This keeps the number of new parameters at two; it is a simplification, not a claim that per-edge blending would fail.

2.6 Sparse regression and edge pruning

As a baseline from a completely different modelling approach, we use Sparse Identification of Nonlinear Dynamics (SINDy) [2], through the PySINDy library [6]. SINDy fits a governing equation as a sparse linear combination of candidate terms from a fixed library; ours is every polynomial in x and y up to degree two, $\{1, x, y, x^2, xy, y^2\}$. The coefficients are estimated by sequentially thresholded least squares: fit by ordinary least squares against numerically estimated derivatives, zero out every coefficient below a fixed threshold, and refit on the surviving terms until the active set stops changing. This is what makes the recovered model sparse and, in principle, readable: unlike the KAN’s 240 free parameters, SINDy returns a short list of surviving terms with numeric coefficients that can be compared directly with the true equation’s. We hold the threshold fixed across the whole noise sweep so that the comparison measures noise sensitivity rather than per-level tuning.

The closest KAN-ODE analogue is to encourage sparsity during training and prune weak edges afterwards. The base paper does this by adding an L_1 penalty on the spline coefficients to the loss,

$$\mathcal{L} = \text{MSE}(\mathbf{u}_{\text{true}}, \mathbf{u}_{\text{pred}}) + \lambda \sum_i |c_i|, \quad (14)$$

where c_i ranges over every spline coefficient in Equation 2 and λ trades trajectory accuracy against how many edges the penalty drives toward zero. Every checkpoint we trained used $\lambda = 0$, the plain mean-squared-error term alone. To prune, we rank each edge by its mean spline-coefficient magnitude and mask the weakest (Section 4.11).

3 Implementation and Experiments

3.1 Datasets

All training data except the epidemic curve are simulated from known equations, with ground truth generated by SciPy’s [13] DOP853 integrator at relative and absolute tolerance 10^{-12} , far below any error we report. Table 3 summarises the four systems and their train/test splits. Every accuracy number is computed only on the test window, which the training loss never sees.

Lotka-Volterra (main benchmark). The base paper’s predator-prey system [8],

$$\begin{aligned} \dot{u}_1 &= \alpha u_1 - \beta u_1 u_2, & \dot{u}_2 &= \delta u_1 u_2 - \gamma u_2, \\ (\alpha, \beta, \gamma, \delta) &= (1.5, 1.0, 3.0, 1.0), \end{aligned} \quad (15)$$

from $\mathbf{u}(0) = (1, 1)$, sampled every $\Delta t = 0.1$. We train on $t \in [0, 3.5]$ (36 points) and score strictly on $t \in (3.5, 14.0]$ (105 unseen points). Extrapolation is scored this way because it is the only meaningful test of whether the network learned the dynamics rather than memorising the training window; a model that reproduces the 36 training points can still be badly wrong about the system. Two properties of Equation 15 serve as exact, model-independent checks later on. It has a single interior equilibrium $\mathbf{u}^* = (\gamma/\delta, \alpha/\beta) = (3, 1.5)$, where the Jacobian has purely imaginary eigenvalues $\pm i\sqrt{\alpha\gamma} = \pm 2.121i$. And it conserves the first integral

$$H(u_1, u_2) = \delta u_1 - \gamma \ln u_1 + \beta u_2 - \alpha \ln u_2, \quad \frac{dH}{dt} = 0, \quad (16)$$

so every orbit is a closed level curve of H and the phase plane is filled with a continuous family of neutral cycles. The orbit through $(1, 1)$ has $H_0 = 2$ and period $T = 3.3175$, so the training window covers 1.06 periods, the test window 3.2 further periods, and $[0, 14]$ about 4.2.

SIR epidemic model. The compartmental model splits a fixed population into susceptible (S), infected (I) and recovered (R) fractions,

$$\dot{S} = -\beta SI, \quad \dot{I} = \beta SI - \gamma I, \quad \dot{R} = \gamma I, \quad (17)$$

with $(\beta, \gamma) = (0.35, 0.1)$ and $(S, I, R)(0) = (0.99, 0.01, 0)$. It has a built-in algebraic invariant, $S + I + R = 1$, because the three terms of the field cancel when summed. The invariant holds for any parameter values, so a model that violates it has learned a field with the wrong structure, not merely an inaccurate one. We simulate $t \in [0, 80]$ days at $\Delta t = 0.5$, train on $[0, 50]$ (101 points) and test on $(50, 80]$.

Damped pendulum. The angle θ and angular velocity ω evolve under gravity and viscous friction,

$$\dot{\theta} = \omega, \quad \dot{\omega} = -\frac{g}{L} \sin \theta - \mu \omega, \quad (g/L, \mu) = (9.81, 0.5), \quad (18)$$

from $(\theta, \omega)(0) = (2, 0)$. Since $\mu > 0$, the mechanical energy $E = \frac{1}{2}mL^2\omega^2 + mgL(1 - \cos \theta)$ strictly dissipates, $dE/dt = -mL^2\mu\omega^2 \leq 0$. A model can fit the trajectory well and still violate this monotonic decay, which makes it a genuinely independent physical check. We simulate $t \in [0, 10]$ at $\Delta t = 0.05$. The original recipe trained on $[0, 3]$; the fixed recipe (Section 4.6) trains on $[0, 5]$ (101 points) and tests on $(5, 10]$. The stiffness study (Section 4.10) sweeps μ .

Table 3: Datasets, networks and train/test splits. Times are in the system’s own units (days for the two epidemic systems).

System	Network (params)	Train (points)	Test (points)
Lotka-Volterra	[2, 10, 2], $G=5$ (240)	[0, 3.5] (36)	(3.5, 14] (105)
SIR	[3, 16, 3], $G=8$ (864)	[0, 50] (101)	(50, 80] (60)
Pendulum	[2, 10, 2], $G=8$ (360)	[0, 5] (101)	(5, 10] (100)
Epidemic curve	[2, 10, 2], $G=5$ (240)	days 0–44 (45)	days 45–120 (76)

Epidemic outbreak curve. To test whether our SIR fixes rely on the SIR equations themselves, we also fit a single outbreak wave that no compartmental ODE generated. The curve is synthetic but built to look like observed case data: a Gaussian wave of width 14 centred at day 38, skewed by a tanh factor so that its infected peak falls on day 45, with 1% Gaussian observation noise, smoothed by a 7-day moving average. The state is the infected curve together with a cumulative-recovered curve, both min-max normalized to [0, 1]. It spans 120 days; the default split trains on the first 45 days and tests on the rest.

3.2 Training recipe

Table 4 lists every setting shared by the Lotka-Volterra runs; each experiment in Section 4 changes exactly one of them.

3.3 Evaluation protocol

Model selection uses the minimum *training*-window loss only, and every accuracy number is computed strictly outside the training horizon (Figure 6), so a number cannot improve just by widening what counts as fit.

Writing $\hat{\mathbf{u}}_k$ for the model’s prediction and $\mathbf{u}_k$ for the truth at the $N_e = 105$ test samples, the reported metrics are

$$\text{MSE} = \frac{1}{2N_e} \sum_k \|\hat{\mathbf{u}}_k - \mathbf{u}_k\|_2^2, \quad R^2 = 1 - \frac{\sum_k \|\hat{\mathbf{u}}_k - \mathbf{u}_k\|_2^2}{\sum_k \|\mathbf{u}_k - \bar{\mathbf{u}}\|_2^2},$$

$$\text{rel. } L_2 = \frac{(\sum_k \|\hat{\mathbf{u}}_k - \mathbf{u}_k\|_2^2)^{1/2}}{(\sum_k \|\mathbf{u}_k - \bar{\mathbf{u}}\|_2^2)^{1/2}}, \quad (19)$$

where $\bar{\mathbf{u}}$ is the componentwise mean of the true test trajectory. $R^2 = 1$ is a perfect match, $R^2 = 0$ is no better than predicting that mean, and a negative value is worse than that trivial baseline. Two further quantities describe the trained field rather than its trajectory. The **Lipschitz estimate**

$$\hat{L} = \max_{k \leq 36} \frac{\|\mathbf{f}_\theta(\mathbf{u}_k + \varepsilon \mathbf{v}_k) - \mathbf{f}_\theta(\mathbf{u}_k)\|_2}{\varepsilon}, \quad (20)$$

with $\varepsilon = 10^{-3}$ and random unit vectors $\mathbf{v}_k$, is a finite-difference sample of the local slope of $\mathbf{f}_\theta$ at the training states. It is a *lower* bound on the field’s true Lipschitz constant and measures how sharply the learned field bends. The **cost** of a solver is its number of function evaluations (NFE): an s -stage method on the training window costs $\text{NFE} = 35 \times 2 \times s = 70s$ calls of $\mathbf{f}_\theta$ per epoch. Because reverse-mode differentiation unrolls every call, training wall-clock scales with it too.

3.4 Computational budget: why 10,000 epochs

The base paper trains for 10^5 epochs. Table 5 shows what that would cost us. The paper’s integration loop is compiled by Julia’s SciML stack; ours is interpreted PyTorch on CPU, where every Runge-Kutta stage is a separate Python-level call with its own autograd bookkeeping. On our development machine one Tsit5/RBF epoch takes 0.17 s and one Tsit5/B-spline epoch 0.67 s; the cloud CPUs we used for the long runs of Section 4.12 were 2.2–2.5× slower for every KAN configuration.

A 10^5 -epoch version of the 24-configuration Lotka-Volterra sweep would need roughly 142 CPU-hours locally, or about two weeks of cloud CPU time, before the pendulum, SIR and follow-on studies. We therefore ran the sweep at 10^4 epochs, a budget at which every noise-free solver and basis configuration except Euler and four bases (Lagrange, IQF, RSWAF, Newton) reaches 10^{-4} training MSE (Table 13). That choice is only safe if the conclusions drawn at 10^4 epochs survive a longer budget, which is exactly what Section 4.12 tests.

4 Results and Discussion

We report the studies in the order of Table 1. Most of them open with a short box stating the question, what we expected, and what we observed, and close with a verdict. Sections 4.1 to 4.6 establish the baseline and the ablations, Sections 4.7 to 4.11 are follow-on studies that each isolate one question, and Section 4.12 re-tests the budget-sensitive conclusions at five times the training budget. Section 4.13 then compares our findings with the base paper, and Section 4.14 draws out the patterns that recur across studies.

4.1 Baseline reproduction: KAN-ODE vs. MLP-ODE

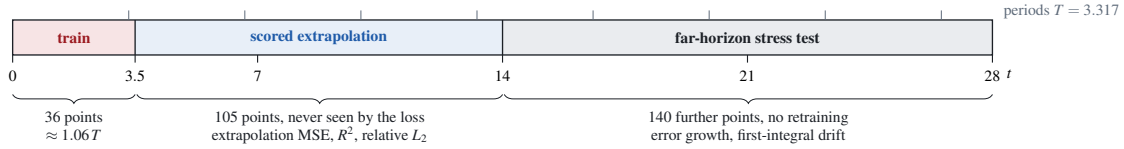
Two MLP vector fields with 252 parameters were trained under the identical recipe: the paper’s literal baseline, one hidden layer of 50 tanh units, and a deeper [2, 14, 8, 8, 2] SiLU network that trains reliably under our protocol. Table 6 and Figures 7 and 8 compare them with the KAN-ODE. A fourth column, added after auditing the paper’s own code (Section 4.13), retrains the identical literal baseline under the paper’s own learning rate and initialization scale instead of our default. Figure 7 plots that run in place of the default-setting tanh MLP; Figure 8 predates it and still shows the original three.

The fourth column shows that the paper’s literal architecture was never the problem. Given the paper’s own learning rate and initialization scale, the identical tanh network reaches $R^2 = 0.997$. We return to this in Section 4.12, which also reports its 5×10^4 -epoch continuation. The next two paragraphs compare KAN-ODE against the working SiLU baseline, columns one and two.

We find one result that confirms the paper and one that contradicts it. **KAN-ODE extrapolates 5.4× better** from a training loss only 7% lower, reproducing the paper’s accuracy claim. The checkpoints show where that gap comes from. Both fields are equally accurate on the orbit (2.84% against 2.82% mean field error), and both hold the invariant to about 3%, but the MLP’s orbit period is off by 8.0×10^{-4} against the KAN’s 1.7×10^{-5} , a 48× larger period error. On a periodic orbit a period error ε_T becomes a phase lag that grows linearly, $\approx 2\pi\varepsilon_T t/T$, so the trajectory error of the MLP keeps accumulating while the KAN’s stays near its

Table 4: Shared training recipe for all Lotka-Volterra runs.

Setting	Value	Note
Network	KAN [2, 10, 2], $G = 5$, 240 parameters	knots uniform on $[-1, 1]$, $h = 0.5$
Edge function	Equation 2, tanh normaliser, SiLU residual	Glorot-uniform initialisation of $\mathbf{C}$ and $\mathbf{W}$
Integrator	Tsit5, fixed step	2 sub-steps per sample: $h = \Delta t/2 = 0.05$
Sampling interval	$\Delta t = 0.1$	36 training points, 141 on $[0, 14]$
Loss	mean squared trajectory error on $[0, 3.5]$	no regularisation ($\lambda = 0$ in Equation 14)
Optimiser	Adam [7], learning rate 2×10^{-3}	constant learning rate, no schedule
Gradient clipping	global norm ≤ 1.0	non-finite gradients skip the step (Section 4.6)
Budget	10,000 epochs, full batch	50,000 in the budget validation (Section 4.12)
Model selection	checkpoint with minimum training MSE	the test window is never used for selection
Precision, seed	float32, seed 42	one seed per configuration ($N = 1$)

**Figure 6: Time axis of the Lotka-Volterra protocol. Tick marks above the bar are multiples of the true period. The far-horizon window (14, 28] is used only for the stress tests of Sections 4.6 and 4.12; no model is retrained or reselected for it.****Table 5: Measured per-epoch cost on our local development CPU and on cloud (Kaggle) CPU, each averaged over the full run, and the projected cost of the paper’s 10^5 -epoch budget.**

Configuration	Local s/epoch	Cloud s/epoch	Ratio	10^5 epochs, local	10^5 epochs, cloud
Euler / RBF	0.0254	0.0627	2.47×	0.7 h	1.7 h
Tsit5 / RBF (reference)	0.1705	0.4033	2.37×	4.7 h	11.2 h
Tsit5 / B-spline	0.6680	1.5438	2.31×	18.6 h	42.9 h
MLP-ODE, SiLU [2, 14, 8, 8, 2]	0.2856	0.2722	0.95×	7.9 h	7.6 h
MLP-ODE, tanh [2, 50, 2]	0.0736	0.1595	2.17×	2.0 h	4.4 h
All 24 Lotka-Volterra configurations	14.2 CPU-h at 10^4 epochs			142 h	≈ 336 h

training-window level. The extrapolation gap is a phase-accuracy gap rather than a field-accuracy gap. **The MLP converges faster, not slower:** it reaches 10^{-3} training MSE at epoch 1297 against the KAN’s 5282, 4.1× sooner. This contradicts the paper’s headline efficiency claim on this same system (a 10^4 -epoch KAN-ODE against a 10^5 -epoch MLP-ODE). Under our protocol, at matched parameter count and optimizer settings, the MLP was the faster model to train; KAN-ODE won on extrapolation, the paper’s other headline claim.

Why the paper’s literal MLP fails to train under our default optimizer settings. The [2, 50, 2] tanh network never gets below a training MSE of 1.08 at our default learning rate (2×10^{-3}) and initialization (Glorot, gain 1.0). A controlled 2000-epoch factorial run, holding those optimizer settings fixed, separates the two things that differ between it and the working baseline (Table 7).

Runs A and D share the architecture and differ only in activation. The training MSE falls from 1.21 to 3.62×10^{-3} , a 335× improvement, so at these optimizer settings, activation decides whether this architecture fits the data at all. Runs C and D share the SiLU activation and differ only in depth. The shallow network fits the window but its extrapolation error explodes to 2.68×10^2 , against 6.97×10^{-4} for the deep one, so depth decides whether

a model that trains also extrapolates. Lowering the learning rate alone (run B) makes the tanh network worse, not better. Five times the epoch budget under these same settings does not rescue it either: at 5×10^4 epochs, training MSE moves only from 1.08 to 1.01 and extrapolation gets *worse* (Table 25).

The actual cause: learning rate and initialization scale, not architecture. None of runs A–D varies the initialization scale, and the base paper’s own published code turned out to use a combination this factorial never tested: a learning rate of 10^{-2} (five times our default) together with weights initialized near zero (Glorot divided by 10^5), for the identical [2, 50, 2] tanh architecture. Retrained with only those two settings changed, the literal architecture trains (Table 6, fourth column). At 10^4 epochs it reaches a training MSE of 1.11×10^{-3} (against 1.08 before) and an extrapolation MSE of 9.65×10^{-3} , $R^2 = 0.997$ (against $R^2 = 0.42$ before), with a Lipschitz estimate of 9.23 against 38.2. At 5×10^4 epochs it reaches 5.43×10^{-5} training MSE and $R^2 = 0.9998$ on extrapolation, with zero non-finite gradient steps and no clipping in either run. Section 4.12 reports the full comparison against KAN-ODE and the SiLU MLP at both budgets, and updates the implementation audit accordingly. The paper’s literal recipe was never architecturally incompatible with this system. Our default optimizer settings, tuned

Table 6: KAN-ODE against parameter-matched MLP-ODEs at 10^4 epochs. Epoch counts are the first epoch at which the training loss falls below the threshold. The last three rows are computed from the saved checkpoints: field error is $\|f_\theta - f\|_2$ averaged over the true orbit on $(3.5, 14]$ and divided by the mean $\|f\|_2 = 4.82$ there; the period is measured from zero-crossings of the predicted prey signal over $[0, 28]$; H drift is $\max_t |H(\hat{u}(t)) - H_0|/H_0$ on $[0, 28]$. The fourth column is the identical literal baseline of the third, retrained under the paper’s own learning rate and initialization scale (Section 4.13); its 5×10^4 -epoch continuation is in Table 25.

Metric	KAN-ODE (RBF)	MLP-ODE (SiLU)	MLP-ODE (tanh, default lr/init)	MLP-ODE (tanh, paper’s lr/init)
Parameters	240	252	252	252
Training MSE	8.83×10^{-5}	9.50×10^{-5}	1.08×10^0	1.11×10^{-3}
Extrapolation MSE	8.92×10^{-5}	4.77×10^{-4}	1.69×10^0	9.65×10^{-3}
Extrapolation R^2	0.99997	0.99984	0.4215	0.99669
Relative L_2 error	0.33%	0.76%	45.2%	3.42%
Lipschitz estimate $\hat{L}$	6.91	5.07	38.22	9.23
Epochs to $10^{-2} / 10^{-3} / 10^{-4}$	3112 / 5282 / 9617	741 / 1297 / 9145	never	3461 / never / never
Wall-clock, 10^4 epochs	1705 s	2856 s	736 s	1052 s
Field error on the orbit, $(3.5, 14]$	2.84%	2.82%	–	7.03%
Relative period error	1.7×10^{-5}	8.0×10^{-4}	–	3.1×10^{-3}
Max. first-integral drift	3.1%	2.8%	–	22.7%

Table 7: Controlled breakdown of why the paper’s literal MLP baseline architecture, $[2, 50, 2]$ with hyperbolic tangent (tanh) activation, fails to train at our default learning rate and initialization scale. All four runs use 2000 epochs and Glorot initialization at gain 1.0; none varies the initialization scale (see the paragraph following this table).

Run	Architecture	Activation	Learning rate	Training MSE	Extrapolation MSE	R^2
A	$[2, 50, 2]$	tanh	2×10^{-3}	1.21×10^0	1.98×10^0	0.319
B	$[2, 50, 2]$	tanh	5×10^{-4}	1.85×10^0	9.23×10^0	−2.169
C	$[2, 14, 8, 8, 2]$	SiLU	2×10^{-3}	4.28×10^{-4}	6.97×10^{-4}	0.99976
D	$[2, 50, 2]$	SiLU	2×10^{-3}	3.62×10^{-3}	2.68×10^2	−91.06

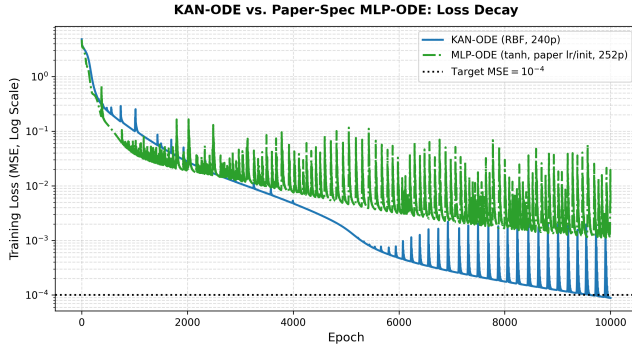


Figure 7: Training MSE per epoch for KAN-ODE and the paper’s parameter-matched tanh MLP-ODE, trained with the paper’s own learning rate and initialization (fourth column of Table 6). KAN-ODE descends steadily and reaches 10^{-4} at epoch 9617; the tanh MLP reaches 10^{-2} at epoch 3461 but never 10^{-3} within 10^4 epochs.

around the KAN-ODE side of the comparison, were the wrong settings for a wide, near-linear-at-initialization tanh network, and Table 7’s factorial measured sensitivity to activation *at those settings* rather than an architecture-level limit.

4.2 Integrator ablation

We first verify that each integrator of Section 2.2 is the method it claims to be, then sweep all six at a fixed RBF basis.

4.2.1 Order verification and linear stability.

Verifying the implementations. Two independent checks confirm that each integrator is the method it claims to be. The first is the order conditions: a Runge-Kutta method has order p exactly when $\mathbf{b}^\top \Phi(\tau) = 1/\gamma(\tau)$ for every rooted tree τ with at most p vertices, where $\Phi(\tau)$ is the tree’s elementary weight built from A and $\gamma(\tau)$ its density [5]. We enumerate all 37 rooted trees up to six vertices and evaluate each condition directly on the implemented coefficients. This gives the verified order and the principal error norm $A^{(p+1)} = (\sum_{|\tau|=p+1} [(\mathbf{b}^\top \Phi(\tau) - 1/\gamma(\tau))/\sigma(\tau)]^2)^{1/2}$, the size of the leading local-error term. The second is a work-precision study on the *true* Lotka-Volterra field: integrating from $(1, 1)$ to $t = 3.5$ at eight step sizes and comparing against the DOP853 reference gives the observed order $\hat{p}$ as the slope of $\log \|\text{error}\|$ against $\log h$ (Figure 9). Table 8 collects both.

Every method satisfies its order conditions to round-off and shows its design order in practice. The fifth-order pair shows slightly super-nominal slopes over this range because its error is still pre-asymptotic before it hits round-off. The principal error norms confirm Tsitouras’s design goal from the code’s own coefficients: Tsit5’s leading error term is $2.9\times$ smaller than

Table 8: Order verification of the implemented integrators. “Max residual” is the largest $\|\mathbf{b}^\top \Phi(\tau) - 1/\gamma(\tau)\|$ over all trees up to the design order ($1 + 1 + 2 + 4 + 9 = 17$ conditions at order five); $A^{(p+1)}$ is the principal error norm; $\hat{p}$ is the slope fitted to the work-precision data for $h \leq 0.1$ above the round-off floor.

Method	Stages s	Verified order p	Conditions checked	Max residual through order p	$A^{(p+1)}$	Observed $\hat{p}$
Forward Euler	1	1	1	0	5.00×10^{-1}	1.00
Heun	2	2	2	0	1.86×10^{-1}	2.10
Midpoint	2	2	2	0	1.72×10^{-1}	2.07
Classical RK4	4	4	8	1.1×10^{-16}	1.45×10^{-2}	3.93
DOPRI5 (6 of 7)	6	5	17	2.8×10^{-16}	3.99×10^{-4}	5.66
Tsit5 (6 of 7)	6	5	17	1.4×10^{-14}	1.39×10^{-4}	5.95

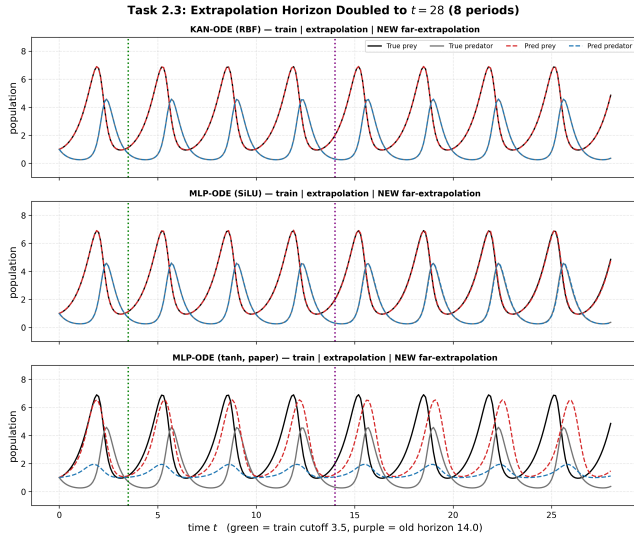


Figure 8: Predicted against true populations out to $t = 28$ for the three models, with no retraining. KAN-ODE and SiLU-MLP are visually indistinguishable from the truth. The tanh MLP (bottom) is the pre-fix run with default learning rate and initialization, not the paper-settings run of the fourth table column: it runs slow, falls behind by more than a quarter period, and never reproduces the predator peak.

DOPRI5’s. The table also sets up a point used below. Heun and Midpoint differ only in their third-order error terms, and on those Midpoint’s is marginally *smaller* (0.172 against 0.186).

Linear stability. Applied to $\dot{u} = \lambda u$, every explicit Runge-Kutta method multiplies the state by the stability polynomial $R(z) = 1 + z \mathbf{b}^\top (I - zA)^{-1} \mathbf{1}$, $z = h\lambda$, per step. Heun and Midpoint share $R(z) = 1 + z + z^2/2$, so they are indistinguishable on any linear problem. Figure 10a draws the boundaries $|R(z)| = 1$ and overlays $h\lambda$ for the eigenvalues of the true Jacobian and of the trained Tsit5 KAN’s Jacobian $J_\theta = \partial \mathbf{f}_\theta / \partial \mathbf{u}$, evaluated at all 141 states of the orbit. The learned eigenvalues track the true ones closely ($\max |\lambda| = 3.61$ against 3.83), and at $h = 0.05$ every $h\lambda$ lies within 0.19 of the origin, far inside every region. Stability in the classical sense is therefore never at issue at the training step. What differs between methods is how accurately R reproduces e^z near the origin. For a purely

oscillatory mode $z = i\omega h$, Figure 10b plots the per-step amplitude error $|R(i\omega h)| - 1$. At Lotka-Volterra’s linear frequency $\omega_0 = \sqrt{\alpha\gamma}$, a linear oscillation integrated over one period gains 39% amplitude under Euler and 0.094% under either second-order method, and loses 5.8×10^{-7} , 2.3×10^{-8} and 3.5×10^{-9} under RK4, DOPRI5 and Tsit5.

The linear picture is borne out on the true nonlinear field. Integrating the exact Lotka-Volterra field at the training discretisation and scoring it with the same extrapolation metric as the trained models isolates each solver’s own truncation error (last column of Table 9). It is 17.6 for Euler, whose trajectory leaves the positive quadrant at $t = 19.4$, $1.3\text{--}3.0 \times 10^{-4}$ for the second-order pair, and below 5×10^{-9} for the rest. The first integral of Equation 16 tells the same story (Figure 20a): by $t = 28$ it has drifted by 7.5×10^{-4} and 4.5×10^{-4} (relative) under Heun and Midpoint, 3.0×10^{-5} under RK4, and about 10^{-7} under the fifth-order pair.

4.2.2 Solver ablation. Table 9 sweeps the six integrators at fixed RBF basis; Table 10 in Section 4.3 sweeps seven basis representations at fixed Tsit5 solver. Both use the protocol of Section 3.3. We report the Lipschitz estimate of the trained field (Equation 20) alongside accuracy because it measures how smooth or jagged the basis expansion in Equation 2 ended up, independent of the trajectory error.

Truncation error dominates only at order $p = 1$. Euler’s extrapolation MSE is $28\times$ Tsit5’s, and it is the only solver that fails to reach 10^{-4} training loss within budget. Its trained field is also the least smooth of the six ($\hat{L} = 8.55$, the highest in the table), consistent with a systematic bias forcing the network to fit a sharper correction than the smoother integrators require. Above order 2 the ranking collapses into a 3.8% spread with no monotonic order dependence: at $h = 0.05$, integrator truncation error has fallen below the optimization noise floor. Midpoint matches Tsit5’s accuracy within 0.3% at one third of the function evaluations and 29% of the wall-clock, making it the cost-normalized optimum on this problem.

The last column is the striking one. Integrated with the *true* field, Euler’s own error on the extrapolation window is 17.6. The Euler-trained network, integrated with Euler, scores 2.5×10^{-3} , nearly four orders of magnitude better than the exact field under the same solver. The same holds, more mildly, for Midpoint, whose trained model (8.95×10^{-5}) beats the exact field under Midpoint (3.0×10^{-4}). The network is not learning $\mathbf{f}$; it is learning whatever field makes

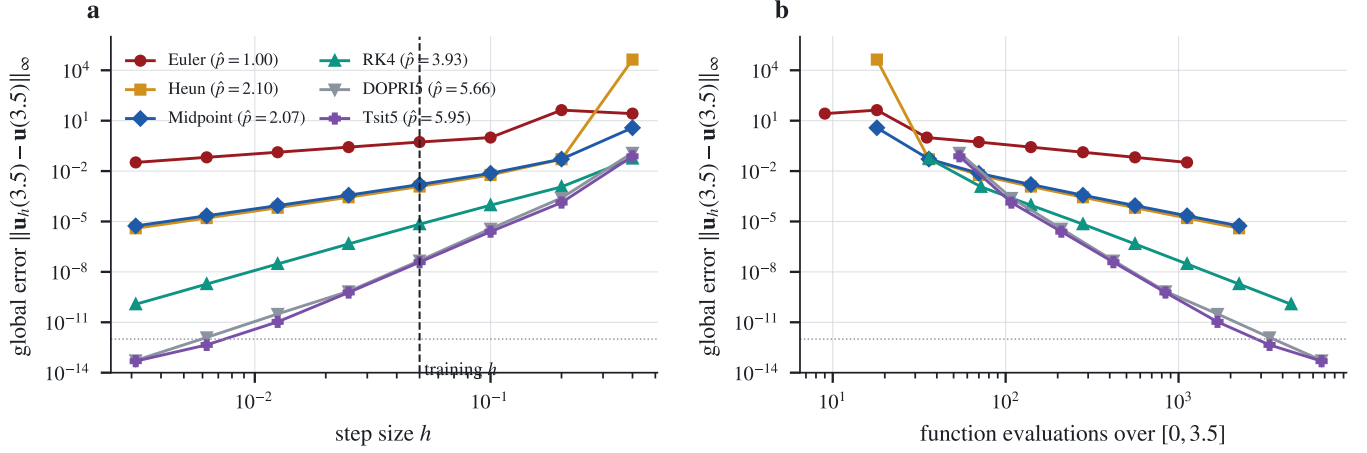


Figure 9: Global error at $t = 3.5$ of each implemented integrator applied to the true Lotka-Volterra field. (a) Against step size, with fitted slopes; the dashed line is the training step $h = 0.05$. (b) Against the number of field evaluations, the quantity that sets training cost. At $h = 0.4$ both second-order methods blow up, and the fifth-order methods reach the round-off floor near 10^{-13} .

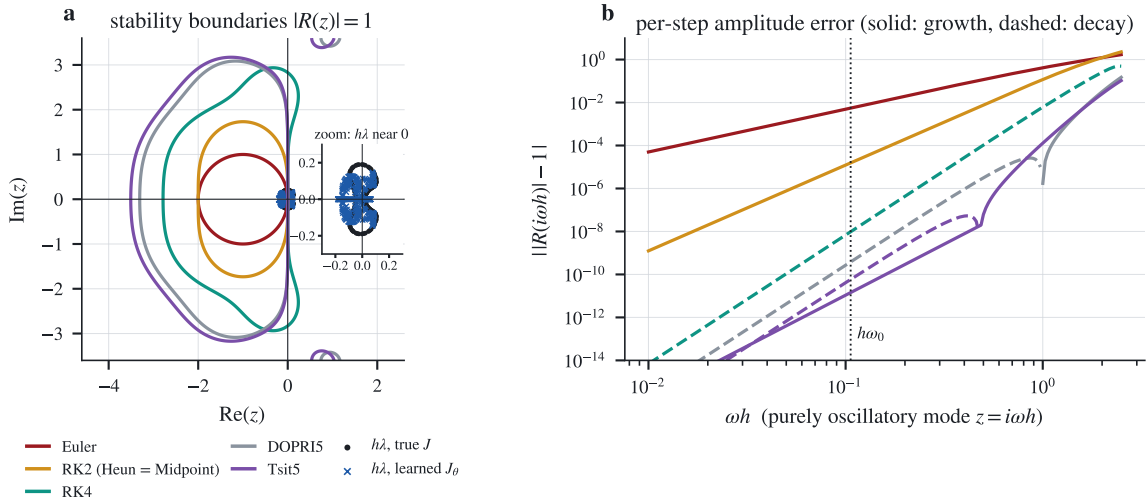


Figure 10: (a) Stability boundaries $|R(z)| = 1$ computed from each implemented tableau, with $h\lambda$ for the true Jacobian (dots) and the learned Jacobian of the Tsit5-trained KAN (crosses) at every state on the orbit, $h = 0.05$; the inset enlarges the neighbourhood of the origin. (b) Per-step amplitude error for an undamped oscillation; the dotted line marks $h\omega_0$ at the training step.

this solver's output match the data. Two experiments make that precise.

The Euler model learns the inverse modified equation. Backward error analysis states that Forward Euler applied to a field $\mathbf{g}$ follows, to second order, the modified field $\mathbf{g} - \frac{h}{2} J_{\mathbf{g}} \mathbf{g} + O(h^2)$. For Euler's output to reproduce the exact flow, the network must therefore learn

$$\mathbf{g}_h = \mathbf{f} + \frac{h}{2} J_{\mathbf{f}} \mathbf{f} + O(h^2). \quad (21)$$

On the 36 training states, the Euler-trained field differs from the true field by 9.6% (relative L_2), almost exactly the size of the predicted correction $\frac{h}{2} J_{\mathbf{f}} \mathbf{f}$ (9.2%). The learned correction $\mathbf{f}_\theta - \mathbf{f}$ and the

predicted one point the same way (cosine similarity 0.92) with a magnitude ratio of 1.04. Measured against $\mathbf{g}_h$ instead of $\mathbf{f}$, the Euler model's error falls to 3.8%, comparable to the 3.0% field error of the Tsit5-trained model. The Euler model is an accurate model of Equation 21.

Cross-solver transfer. If each model has absorbed its solver's error, it should degrade when integrated with a different solver. Figure 11 evaluates every trained model under every integrator. The Euler model is 70× worse under any other solver (0.17–0.18), and

Table 9: Solver ablation (fixed RBF basis, $h = 0.05$, 240 parameters, 10^4 epochs). The last column integrates the *true* field with the same solver and step and scores it identically: the solver’s own truncation error, with no learning involved.

Solver	Order	NFE / epoch	Training MSE	Extrap. MSE	Extrap. R^2	$\hat{L}$	Wall-clock	True-field truncation MSE
Forward Euler	1	70	1.38×10^{-4}	2.51×10^{-3}	0.99914	8.55	254 s	1.8×10^1
Heun RK2	2	140	8.32×10^{-5}	3.10×10^{-4}	0.99989	7.07	499 s	1.3×10^{-4}
Midpoint	2	140	8.86×10^{-5}	8.95×10^{-5}	0.99997	6.96	498 s	3.0×10^{-4}
Classical RK4	4	280	9.41×10^{-5}	9.29×10^{-5}	0.99997	6.88	1019 s	4.7×10^{-9}
DOPRI5	5	420	9.28×10^{-5}	9.21×10^{-5}	0.99997	6.89	1699 s	6.7×10^{-14}
Tsit5	5	420	8.83×10^{-5}	8.92×10^{-5}	0.99997	6.91	1712 s	4.7×10^{-14}

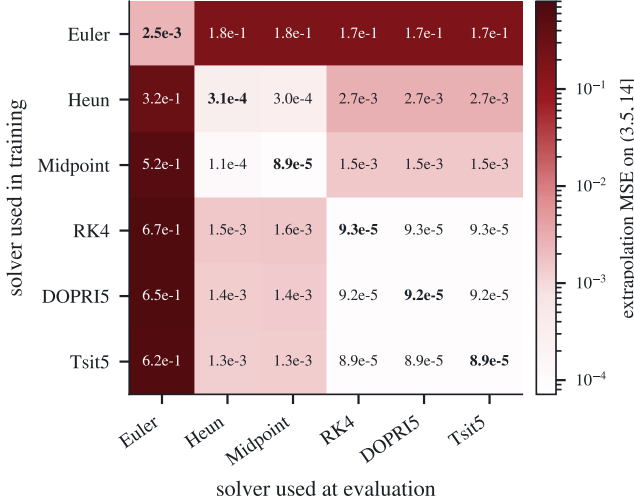


Figure 11: Extrapolation MSE of each trained model (rows) integrated with each solver (columns), $h = 0.05$. The diagonal (bold) is the configuration it was trained in.

every other model is 1000–7200× worse under Euler. The second-order models lose 9–17× when integrated with a fourth- or fifth-order method, and the fourth- and fifth-order models lose 14–17× under a second-order one. Within each family the models are interchangeable: RK4, DOPRI5 and Tsit5 models agree to three significant figures under all three high-order solvers, so these three have learned fields that coincide up to their $O(h^4)$ differences. The practical rule is that a KAN-ODE trained with a low-order solver must be deployed with that same solver. A trained field is a statement about the physics only once the training solver’s truncation error is below the model error.

Midpoint against Heun. Midpoint extrapolates 3.5× better than Heun at identical cost and order. A tempting explanation is that Heun’s end-of-step sampling adds numerical damping. The numbers above rule that out. The two methods have the same stability polynomial, so they damp identically on linear modes; Heun is actually the *more* accurate integrator of the true field (1.3×10^{-4} against 3.0×10^{-4} truncation MSE); and the third-order error norms differ by only 8%. The gap belongs to the trained models, not to the integrators. It shows up as phase accuracy: the Heun model’s orbit period is off by 4.7×10^{-4} (relative), the Midpoint model’s by

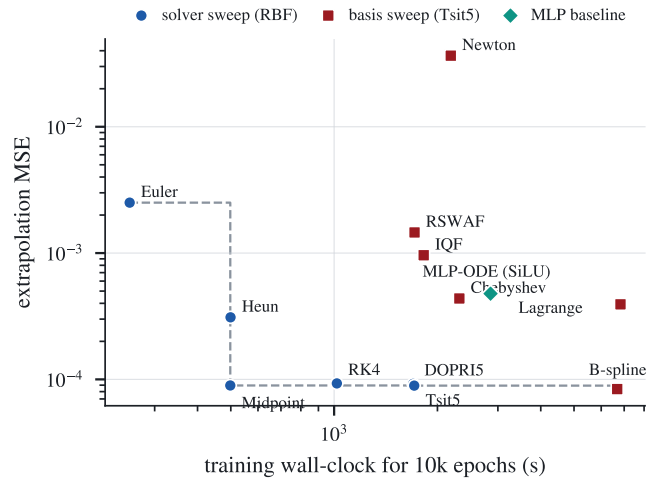


Figure 12: Training wall-clock against extrapolation MSE for the solver and basis sweeps. The dashed step is the Pareto front: Euler, Midpoint, Tsit5 and B-spline. Midpoint dominates RK4 and DOPRI5 outright.

3.5×10^{-6} (Table 12), and its error envelope grows ten times faster over $(3.5, 28]$ (5.9×10^{-3} against 5.7×10^{-4} per unit time). With a single seed per configuration this ordering could still be an initialisation effect; the solver-level explanation is refuted either way.

4.3 Basis-function ablation

We now fix the Tsit5 solver and compare the seven basis families defined in Section 2.3, starting with their conditioning and locality.

Figure 14 draws the seven families from the code itself and reports the 2-norm condition number $\kappa_2(\Psi)$ of the $400 \times G$ design matrix $\Psi_{ng} = B_g(u_n)$ on a uniform sample of $[-1, 1]$. This is the quantity that governs how well-posed fitting the coefficients c_g is. At $G = 5$ Chebyshev (2.5), Lagrange (3.7) and B-spline (5.5) are well conditioned, the three kernels sit at 8–34, and Newton is worst at 54. The growth with G separates them further: at $G = 12$ Chebyshev and B-spline are still below 6, RBF and Lagrange reach 69 and 63, and Newton reaches 2.4×10^4 , growing exponentially, because the nested products $\prod_{j < g} (u - z_j)$ become nearly collinear columns.

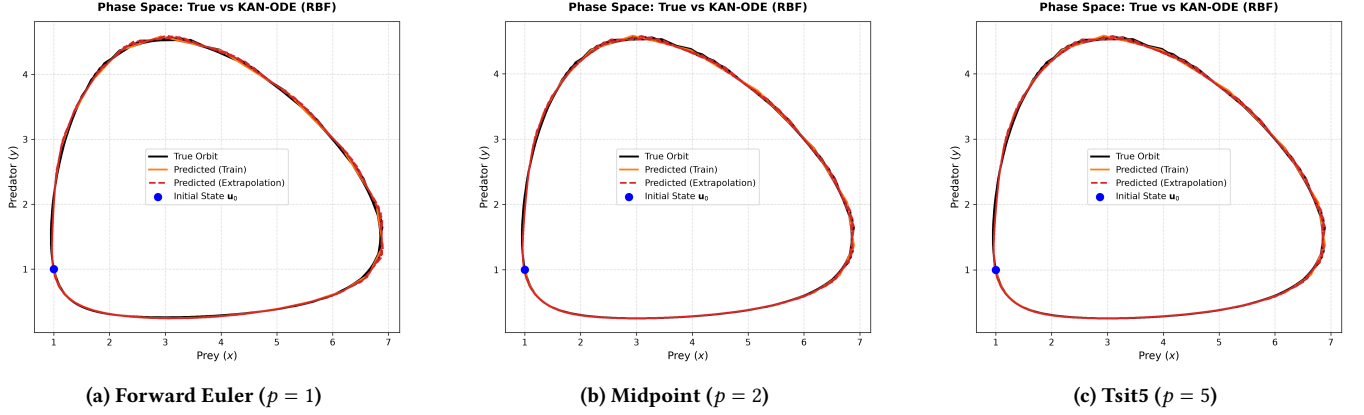


Figure 13: Phase portraits of the Euler-, Midpoint- and Tsit5-trained RBF models on $[0, 14]$ (all three plots are titled “KAN-ODE (RBF)” because the basis is shared). Each model is integrated with its own training solver. All three stay on the true orbit to plotting accuracy, including Euler, whose $28\times$ larger extrapolation MSE is phase error along the orbit rather than a visible drift off it.

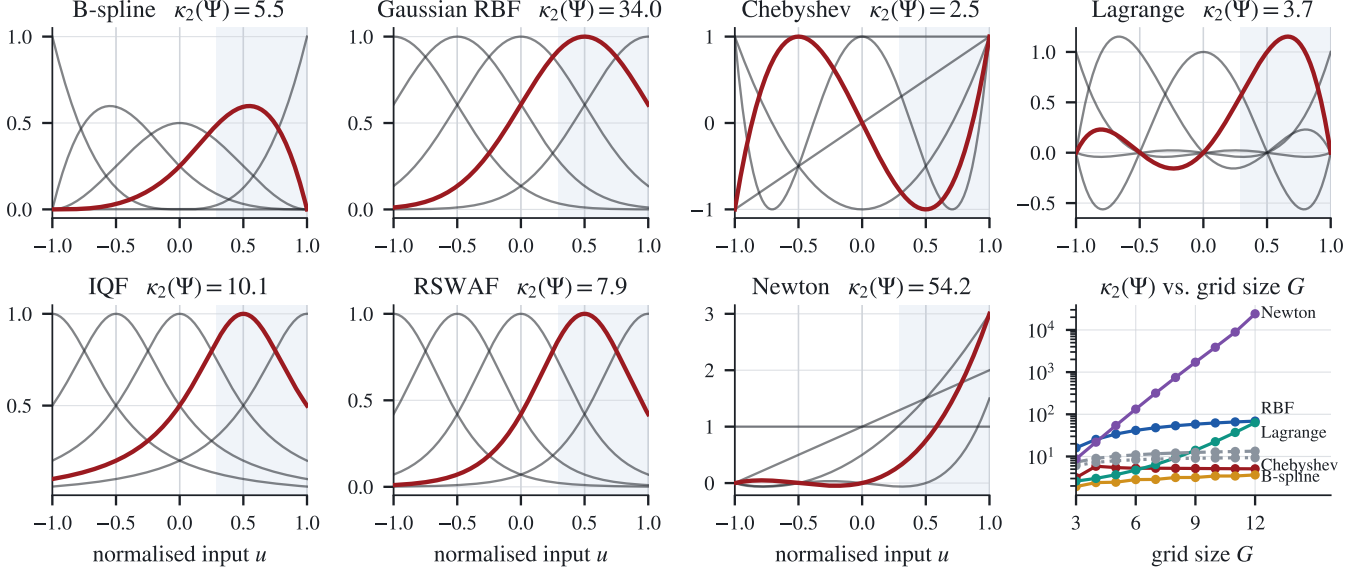


Figure 14: The seven implemented basis families on the $G = 5$ grid (vertical lines are the knots z_g), with the $g = 4$ function highlighted and the 2-norm condition number of the design matrix in each title. The blue band is the interval $u \in [0.25, 1]$ that first-layer inputs actually reach on the Lotka-Volterra orbit. The last panel shows κ_2 against grid size; dashed and dotted grey lines are IQF and RSWAF.

How much of the grid the data uses. Lotka-Volterra’s states are positive, between 0.26 and 6.9 on the orbit, so after the tanh normaliser the first layer sees only $u \in [0.25, 1.0]$, the right 37.5% of its grid. Measured on the trained RBF model, the two leftmost basis functions ($z_1 = -1$, $z_2 = -0.5$) carry 0.4% and 3.6% of the first layer’s total basis activation; for the B-spline the leftmost carries exactly zero and the next 1.7%. The second layer, whose inputs are the signed hidden activations, uses its grid evenly (every basis function 18–26%). Roughly a third of the first layer’s 100 spline coefficients are therefore nearly inert on this system. Figure 15 shows

what locality means against that data interval: raising the coefficient of z_2 changes the spline edge by at most 0.021δ on the data, the RBF edge by 0.32δ , and the Chebyshev edge by the full δ .

B-spline edges out RBF on accuracy after a boundary-knot fix, at $3.9\times$ RBF’s wall-clock time and a visibly higher Lipschitz estimate (8.44 vs. 6.91). Its compact local support lets it fit sharper local features than RBF’s smooth, slowly decaying support without being penalized elsewhere in the domain. The two bases are statistically tied on error (6% apart, inside single-seed noise), which makes RBF the better accuracy-per-second choice. Polynomial global-support

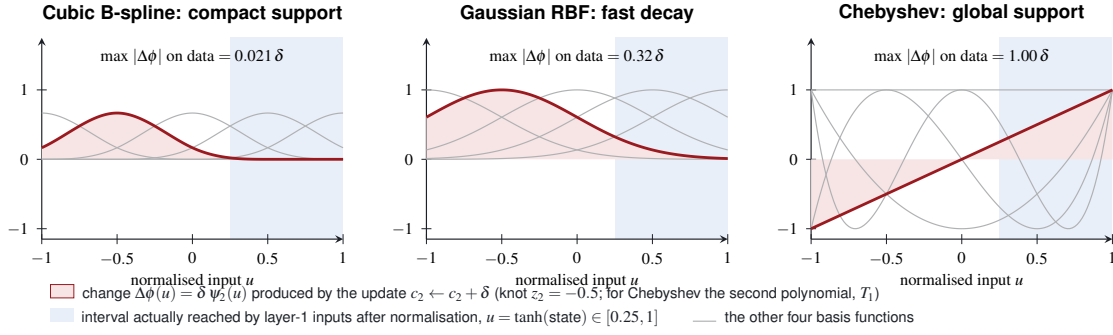


Figure 15: Locality of one coefficient update, $c_2 \leftarrow c_2 + \delta$, for three basis families (uniform-knot cubic spline shown for clarity). Since ϕ is linear in c , the change is exactly $\delta \psi_2$ (shaded). Compact support confines it to its knot span; the RBF’s Gaussian tail still reaches the data interval (blue); a global polynomial changes the edge everywhere.

Table 10: Basis ablation (fixed Tsit5 solver, $G = 5$ grid points, 240 parameters, 10^4 epochs). κ_2 is the design-matrix condition number of Figure 14.

Basis function	Training MSE	Extrap. MSE	R^2	Rel. L_2	$\hat{L}$	κ_2	Wall-clock
Cubic B-spline	7.20×10^{-5}	8.38×10^{-5}	0.99997	0.32%	8.44	5.5	6680 s
Gaussian RBF	8.83×10^{-5}	8.92×10^{-5}	0.99997	0.33%	6.91	34.0	1712 s
Lagrange	1.07×10^{-4}	3.93×10^{-4}	0.99987	0.69%	6.70	3.7	6829 s
Chebyshev	9.80×10^{-5}	4.37×10^{-4}	0.99985	0.73%	6.48	2.5	2317 s
IQF	2.08×10^{-4}	9.61×10^{-4}	0.99967	1.08%	7.16	10.1	1824 s
RSWAF	1.00×10^{-4}	1.46×10^{-3}	0.99950	1.33%	8.72	7.9	1715 s
Newton	1.47×10^{-3}	3.65×10^{-2}	0.98746	6.66%	5.65	54.2	2187 s

bases (Chebyshev, Lagrange) fit the training window well and extrapolate 4–5× worse. This is the expected signature of a basis whose every coefficient influences the whole domain rather than a local region of it. Conditioning alone does not predict the ranking: Chebyshev is the best-conditioned basis and extrapolates 5× worse than RBF, the worst-conditioned of the working kernels. Locality matters more than conditioning once κ_2 is moderate. The two heavy-tailed kernels, IQF (algebraic $O(r^{-2})$ tails) and RSWAF, train as well as the polynomials but extrapolate worst among the working bases (11–16× RBF). Newton is the outlier in both directions: it is the worst-conditioned basis and the only one that never reaches 10^{-3} training loss, and its lowest-in-table Lipschitz estimate (5.65) reflects underfitting, not desirable flatness. Figure 16 shows the same ranking in the phase plane.

4.4 Step-size and noise sensitivity

Two more one-factor sweeps vary the data rather than the model (Figure 17, Table 11). The **step-size sweep** changes the sampling interval Δt , and with it the number of training points and the solver step $h = \Delta t/2$. The trend is not monotone. $\Delta t = 0.1$ extrapolates best; $\Delta t = 0.05$ is 1.5× worse with twice the data and twice the cost; and $\Delta t = 0.2$ reaches the *lowest* training loss and the worst extrapolation (5.1×). The work-precision data rule out truncation as the cause: Tsit5’s own error on the true field at these three steps is 2.6×10^{-18} , 4.7×10^{-14} and 2.2×10^{-10} in MSE, at least six orders of magnitude below any learned error. The coarse grid hurts extrapolation because 19 samples constrain the field less (it fits them

more closely and generalises worse). The fine grid hurts it because the same 10,000 epochs buy a similar training loss on a harder, longer unrolled graph. Here, step size is a data-density choice, not a numerical-accuracy one.

The **noise sweep** adds Gaussian noise of standard deviation σ to the 36 training observations. Across $\sigma \in \{0.01, 0.05, 0.1\}$ the extrapolation MSE follows a clean power law, $\text{MSE}_{\text{extrap}} \approx 12.9 \sigma^{2.02}$. In other words, extrapolation error is proportional to the noise variance, with a gain of about 13. The training-window error against the clean truth grows more slowly ($\propto \sigma^{1.24}$) and sits below σ^2 at every level ($0.23\sigma^2$ at $\sigma = 0.1$), so the ODE constraint does denoise the fit inside the window. What it cannot do is stop the remaining field error from compounding into phase error beyond the window. The field also roughens as it absorbs the noise ($\hat{L}$ rises from 6.91 to 9.55). Section 4.11 compares this sensitivity against sparse symbolic regression.

4.5 What the trained fields learned

Trajectory error measures one orbit. Because we saved the checkpoints, we can also examine the learned field $\mathbf{f}_\theta$ itself as a dynamical system, using the invariants of Section 3.1 as ground truth.

Equilibria by Newton’s method. We solve $\mathbf{f}_\theta(\mathbf{u}^*) = \mathbf{0}$ by Newton-Raphson, $\mathbf{u} \leftarrow \mathbf{u} - J_\theta(\mathbf{u})^{-1} \mathbf{f}_\theta(\mathbf{u})$, with the Jacobian from automatic differentiation in float64, starting at the true equilibrium (3, 1.5). The iteration converges quadratically for every working model. For the Tsit5 model the residual norms are 2.1, 1.6×10^{-1} , $1.1 \times$

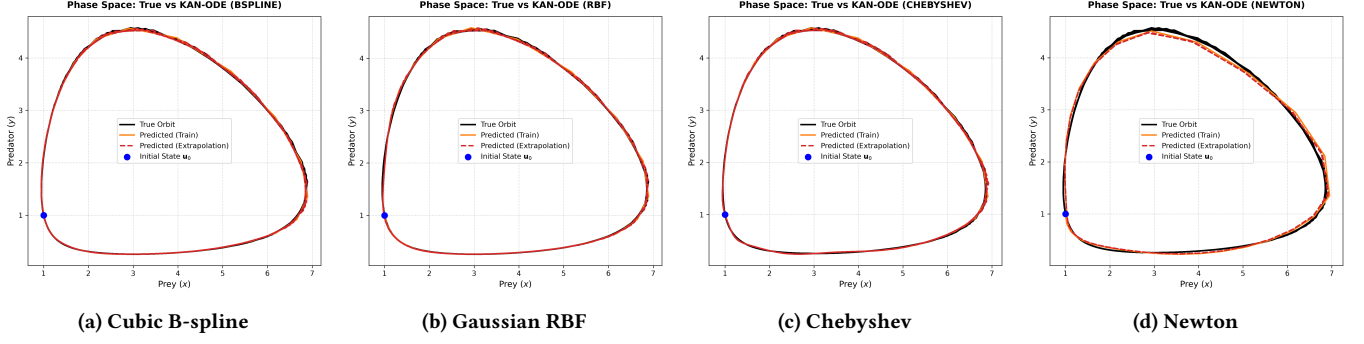


Figure 16: Phase portraits on $[0, 14]$ for four of the seven bases (Tsit5). B-spline, RBF and Chebyshev all trace the true orbit closely at this scale; Chebyshev’s $5\times$ larger extrapolation error is phase error rather than a visible departure. The Newton-basis model visibly misses the orbit near its upper and right-hand extremes.

Table 11: Step-size and observation-noise sweeps (Tsit5, RBF, 10^4 epochs). Noise is added only to the training window; all errors are measured against the clean truth.

Configuration	Training points	Training MSE	Extrap. MSE	Extrap. R^2	$\hat{L}$	Wall-clock
$\Delta t = 0.05$ ($h = 0.025$)	71	9.37×10^{-5}	1.34×10^{-4}	0.99995	7.46	3425 s
$\Delta t = 0.1$ (reference)	36	8.83×10^{-5}	8.92×10^{-5}	0.99997	6.91	1713 s
$\Delta t = 0.2$ ($h = 0.1$)	19	7.83×10^{-5}	4.51×10^{-4}	0.99985	6.64	882 s
$\sigma = 0.01$	36	1.23×10^{-4}	1.22×10^{-3}	0.99958	7.18	1686 s
$\sigma = 0.05$	36	7.41×10^{-4}	2.54×10^{-2}	0.99129	8.69	1672 s
$\sigma = 0.1$	36	2.28×10^{-3}	1.37×10^{-1}	0.95282	9.55	1534 s

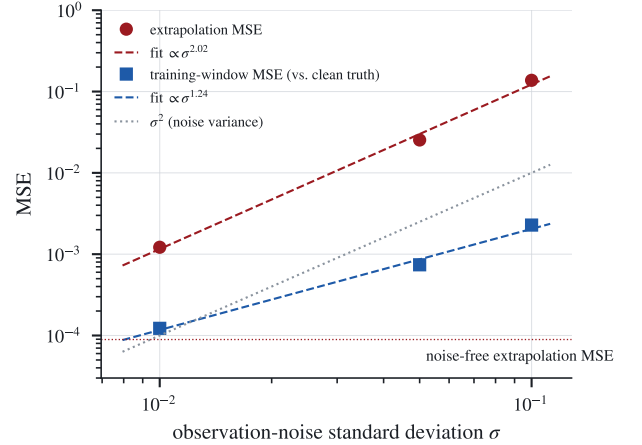
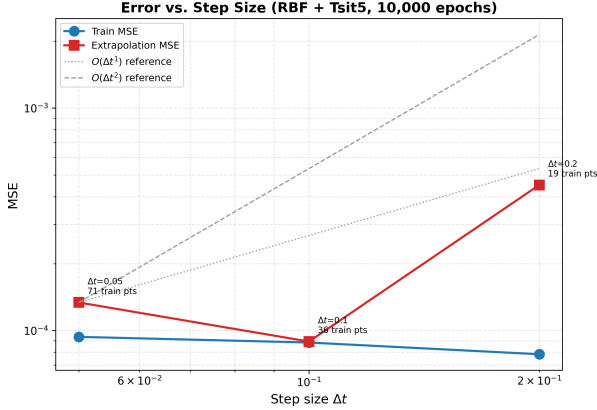


Figure 17: Left: error against sampling interval, with $O(\Delta t)$ and $O(\Delta t^2)$ reference slopes; changing Δt also changes the number of training samples. Right: error against observation-noise level, with least-squares power-law fits; the dotted line is the noise variance σ^2 .

10^{-2} , 9.5×10^{-6} , 3.4×10^{-12} , then exactly zero. The residual is roughly squared at every step, which is the signature of quadratic convergence ($\|F_{k+1}\|/\|F_k\|^2$ stays between 0.04 and 0.43). The roots, however, are far from the truth (Table 12). Every RBF solver model places its equilibrium near (1.9, 1.04), about 1.15 from (3, 1.5), and it is an unstable focus with $\text{Re } \lambda \approx +1.2$, where the true system has $\text{Re } \lambda = 0$. The imaginary parts are close to the true 2.12. The Chebyshev and Lagrange models have *stable* foci instead. Newton

iteration on the Chebyshev model does not converge within 30 steps, and on the Newton-basis model it diverges.

The learned system is a limit-cycle oscillator. An unstable focus surrounded by an orbit that trajectories stay on is the signature of an attracting limit cycle, not of a conservative centre. We confirm this by integrating each learned field from four initial conditions whose true orbits have $H \in \{0.85, 0.86, 2.0, 3.5\}$ (Figure 18). In

Table 12: Numerical analysis of the trained fields. $\mathbf{u}^*$: learned equilibrium (Newton-Raphson from $(3, 1.5)$); λ : eigenvalues of $J_\theta(\mathbf{u}^*)$; period error from zero-crossings of the predicted prey signal over $[0, 28]$; H drift is $\max_t |H(\hat{\mathbf{u}}) - H_0|/H_0$ on $[0, 28]$. True values: $\mathbf{u}^* = (3, 1.5)$, $\lambda = \pm 2.121i$, $T = 3.3175$, drift 0.

Model	$\mathbf{u}^*$	$\ \mathbf{u}^* - \mathbf{u}_{\text{true}}^*\ $	$\text{Re } \lambda$	$ \text{Im } \lambda $	Period error	H drift
Euler	(1.80, 1.02)	1.30	+0.51	2.05	-1.7×10^{-3}	4.2%
Heun	(1.91, 1.03)	1.19	+1.11	2.03	-4.7×10^{-4}	3.1%
Midpoint	(1.93, 1.04)	1.16	+1.17	2.01	-3.5×10^{-6}	3.1%
RK4	(1.94, 1.05)	1.15	+1.21	1.99	$+7.8 \times 10^{-5}$	3.2%
DOPRI5	(1.94, 1.04)	1.16	+1.20	2.00	$+3.5 \times 10^{-5}$	3.2%
Tsit5 / RBF	(1.94, 1.04)	1.15	+1.20	2.01	$+1.7 \times 10^{-5}$	3.1%
B-spline	(1.66, 0.94)	1.46	+1.18	2.99	$+1.3 \times 10^{-4}$	1.3%
Lagrange	(2.67, 2.15)	0.73	-0.44	5.21	-9.9×10^{-4}	3.9%
Chebyshev	Newton iteration does not converge				$+6.0 \times 10^{-4}$	5.1%
IQF	(2.05, 1.19)	1.00	+0.26	1.70	-2.2×10^{-3}	76%
RSWAF	(2.15, 0.91)	1.03	+1.00	1.69	$+6.3 \times 10^{-4}$	5.4%
Newton	Newton iteration diverges				-6.4×10^{-3}	12%
MLP-ODE (SiLU), 10^4	(2.35, 1.12)	0.76	+0.30	2.75	$+8.0 \times 10^{-4}$	2.8%
KAN-ODE, 5×10^4	(1.51, 0.80)	1.65	+0.67	2.85	-1.6×10^{-4}	0.9%
MLP-ODE (SiLU), 5×10^4	(3.65, 1.30)	0.68	+0.95	1.83	$+1.0 \times 10^{-4}$	0.4%

the true system the four orbits are distinct nested curves. Under every learned field (KAN and MLP, at 10^4 and at 5×10^4 epochs) all four trajectories converge onto the single training orbit, with late-time H between 2.000 and 2.008 for every starting point. Both architectures have replaced Lotka-Volterra’s one-parameter family of neutral orbits with one isolated, attracting cycle through the training data. This explains two things at once. First, the flat extrapolation error of the KAN-ODE (Table 17) is partly a property of the attractor: amplitude errors are damped back onto the cycle, so only phase error can accumulate. That is why the period error in Table 12 predicts the extrapolation ranking better than any field metric. Second, the models say nothing reliable about any other initial condition. They have learned an orbit, not the vector field.

Where the field is right. Figure 19 maps the field error $\|\mathbf{f}_\theta - \mathbf{f}\|$ over the phase plane. It is small only in a thin tube around the training orbit. On the orbit the error is 2.7–2.8% of the mean field magnitude for both 10k models and 0.6–1.1% at 50k. Across the plotted box the median error is 20–34%, and for every model less than 3% of the box is below 1% error. Longer training sharpens the tube without widening it: the 50k KAN’s median off-orbit error (34%) is higher than the 10k model’s (24%).

Invariant and error growth. Figure 20b tracks the first integral along each solver model’s own prediction. All of them hold H to within 3–4%, and the drift is oscillatory rather than secular, as it must be on an attracting cycle. The learned models’ invariant error is dominated by model error, orders of magnitude above the solver drift in panel (a), except for Euler. Figure 21 shows the pointwise error out to $t = 28$. The error envelopes of the Midpoint and Tsit5 models grow by $5.5\text{--}5.7 \times 10^{-4}$ per unit time, those of Heun and Chebyshev by $6\text{--}8 \times 10^{-3}$, and Euler’s by 2.1×10^{-2} . Panel (b) previews Section 4.12. At 5×10^4 epochs the MLP error is lower than the KAN’s throughout, while the 50k KAN has a *larger* far-horizon error than the 10k KAN (maximum 6.5×10^{-2} against 3.4×10^{-2} on $(14, 28]$), despite its better scored-window MSE.

What the edges look like. Figure 22 plots the twenty first-layer edge functions $\phi_{i,j}$ of the RBF and B-spline models over the range each input covers on the orbit. Over that range most edges are close to affine, with mild curvature concentrated at small populations, where tanh is least saturated. None of them resembles a term of Equation 15. The bilinear products $u_1 u_2$ are assembled by composition through the second layer rather than carried by any single edge, which is consistent with the pruning result of Section 4.11.

4.5.1 Training dynamics across the sweeps. Figures 23 and 24 show the test MSE of every configuration during training, and Table 13 records when each one’s training MSE crosses fixed thresholds. Five of the six solver curves lie on top of each other throughout. Euler separates from about epoch 2000 and stays above the others until the final approach, so solver choice changes where training ends rather than how it starts. From about epoch 6000 every solver shows periodic loss spikes of one to two orders of magnitude that recover within a few hundred epochs. The basis curves separate much earlier. B-spline is lowest almost throughout and Newton highest, and B-spline, Chebyshev and RSWAF reach 10^{-2} training MSE first (1900–2050 epochs, against RBF’s 3112). Only B-spline turns that head start into better extrapolation.

What the ablation and the numerical analysis establish.

- All six integrators satisfy their order conditions to round-off and show their design order; Tsit5’s leading error constant is $2.9\times$ smaller than DOPRI5’s.
- Above order two, solver choice does not change accuracy at $h = 0.05$; Midpoint is the cost optimum (Tsit5’s accuracy at 29% of the time).
- The network learns the field that its training solver needs, not the physical field: the Euler model is an accurate model of the inverse modified equation, and models do not transfer across solver families.
- Locality, not conditioning, separates the bases; on this system a third of the first layer’s grid is idle because states are positive.

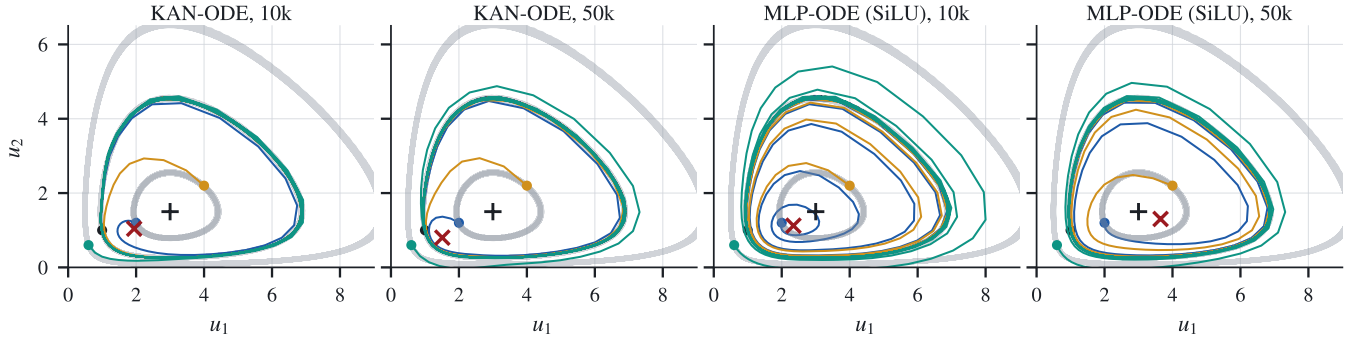


Figure 18: Trajectories of each learned field from four initial conditions (coloured dots) over $t \in [0, 40]$, against the four corresponding true orbits (grey). Red crosses mark the learned equilibria, black plus signs the true one. Every learned field attracts all four onto the training orbit.

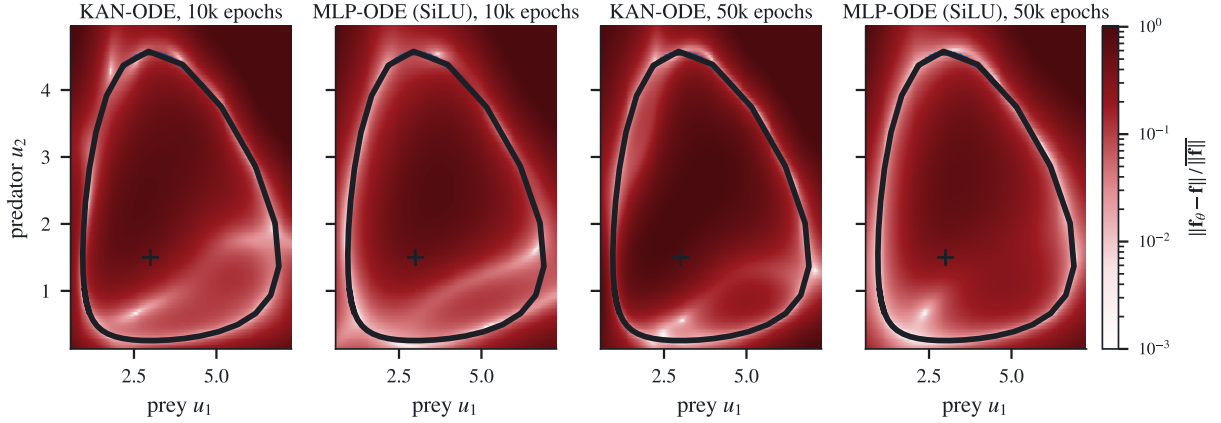


Figure 19: Field error $\|f_0(u) - f(u)\|$, normalised by the mean true field magnitude on the orbit (4.82), clipped to $[10^{-3}, 1]$. Thick black: training segment; blue: rest of the orbit; plus sign: true equilibrium.

Table 13: First epoch at which the training MSE falls below each threshold (“–”: not reached within 10^4 epochs).

Solver	10^{-2}	10^{-3}	10^{-4}	Basis	10^{-2}	10^{-3}	10^{-4}
Euler	3533	6729	–	B-spline	1894	4826	8458
Heun	3144	5564	9512	Gaussian RBF	3112	5282	9617
Midpoint	3102	5285	9599	Chebyshev	1931	3892	9828
RK4	3153	5318	9798	Lagrange	2924	5407	–
DOPRI5	3132	5305	9796	IQF	2561	5726	–
Tsit5	3112	5282	9617	RSWAF	2047	4177	–
MLP-ODE (SiLU)	741	1297	9145	Newton	4577	–	–

- Every trained model, KAN or MLP, is an attracting limit-cycle oscillator around a spurious unstable equilibrium. It extrapolates the training orbit well and says nothing reliable about any other.

about the model’s behavior *before* training, rather than search the hyperparameter space after a failed run.

4.6 Cross-domain failures and fixes

Applying the Lotka-Volterra recipe unchanged to the damped pendulum and to SIR failed on both systems, each for its own reason. Both diagnoses follow the same discipline: measure a quantity

Pendulum: extending the training window. The pendulum recipe originally trained on $t \in [0, 3]$ and diverged in extrapolation ($R^2 = -1.318$). Extending the training window to $t_{\text{train}} = 5.0$, a single flag, resolves it:

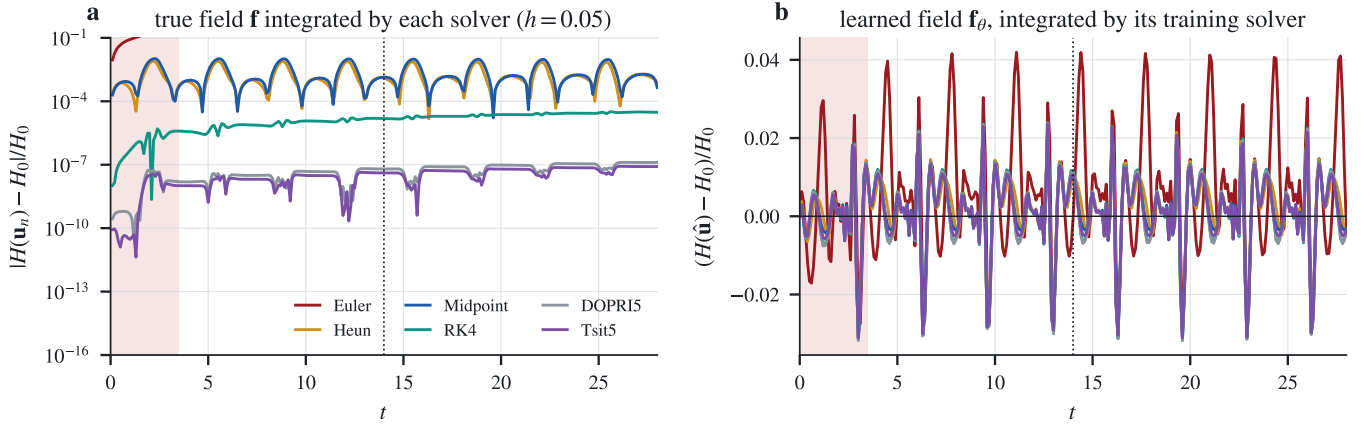


Figure 20: Relative drift of the Lotka-Volterra first integral H (Equation 16). (a) The exact field integrated by each solver at $h = 0.05$: pure truncation drift (Euler’s trajectory leaves the domain of H at $t = 19.4$). (b) Each trained model integrated by its training solver. Shaded: training window; dotted: end of the scored horizon.

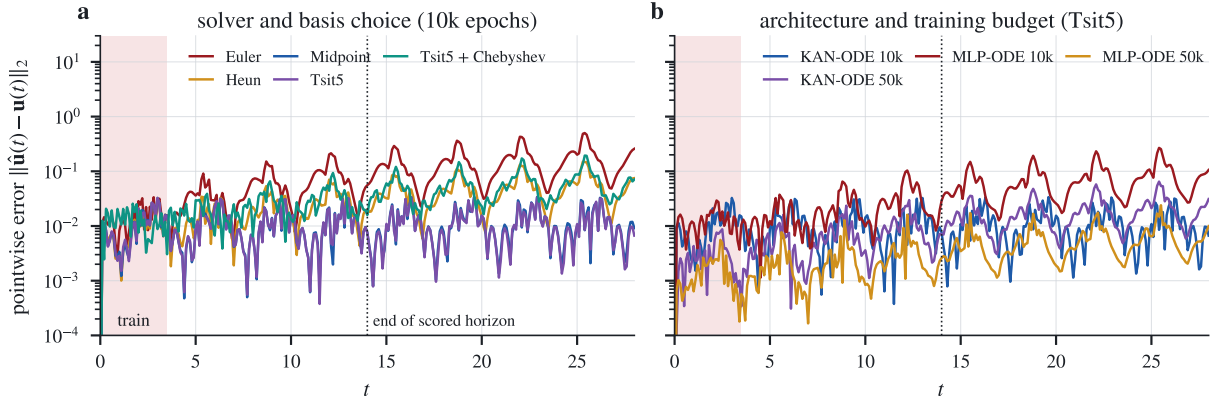


Figure 21: Pointwise trajectory error on $[0, 28]$. (a) Solver and basis choice at 10^4 epochs (Midpoint and Tsit5 overlap). (b) KAN-ODE and MLP-ODE at 10^4 and 5×10^4 epochs.

Table 14: Pendulum fix: best configuration versus the original failure.

Configuration	Full-horizon MSE	Extrapolation R^2	Verdict
Original ($t_{\text{train}} = 3.0$)	8.99×10^{-1}	-1.318	diverges
Extended window ($t_{\text{train}} = 5.0$, SiLU)	4.48×10^{-2}	+0.647	adopted

Question

Why does the Lotka-Volterra recipe, applied unchanged, diverge on the pendulum?

Expected (hypothesis)

SiLU’s residual branch cannot represent the sign-changing derivative $\dot{\theta} = \omega$ – an architectural limit.

Observed

Refuted. A longer training window fixes it under either activation; SiLU fits the sign change fine once the window is long enough.

An earlier hypothesis held that SiLU’s residual branch $W \cdot \text{silu}(x)$ cannot represent a sign-changing derivative such as $\dot{\theta} = \omega$. A factorial control that crosses activation with training-window length refutes it directly. With the longer window, SiLU reaches an angular-velocity root-mean-square error of 0.0114 radians and reproduces the true turning point ($\theta_{\min} = -1.380$ against a true value of -1.385). The training window is what fixes the pendulum. The activation choice affects only how quickly the fit is reached: the identity activation converges $\sim 1000\times$ faster at the short window, but does not match the longer window’s accuracy.

SIR: a unit-scale mismatch. SIR diverged to an invalid (not-a-number) value at epoch 8686.

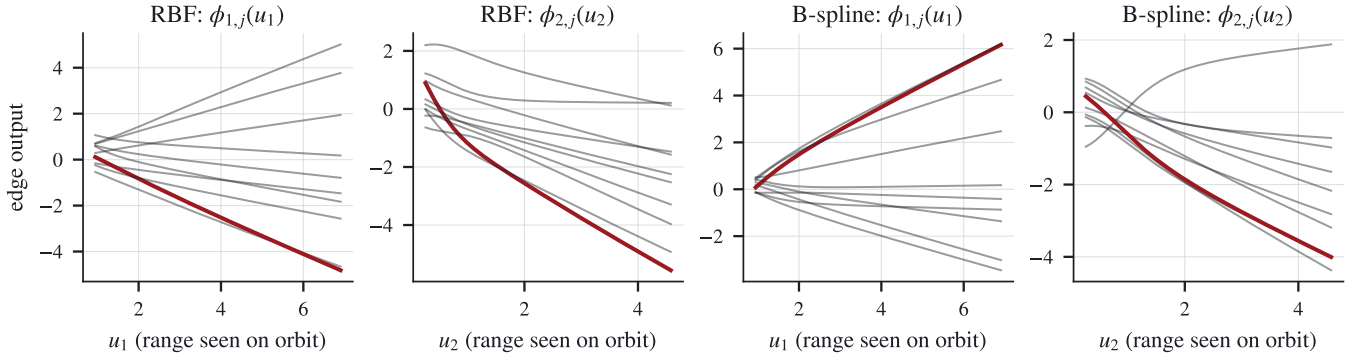


Figure 22: First-layer edge functions (spline plus residual branch) of the trained RBF and B-spline models, each input varied over its range on the orbit with the other held at its mean. The edge with the largest range is highlighted.

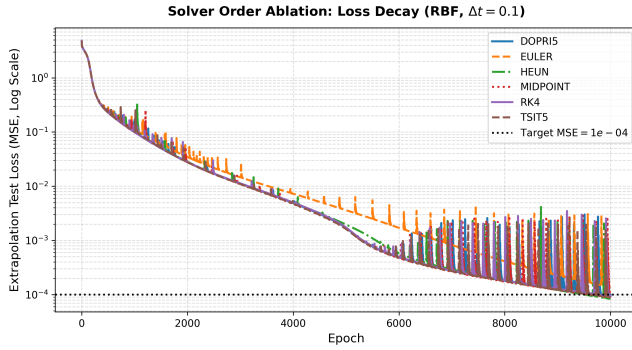


Figure 23: Test MSE during training for the six solvers (RBF basis), logged every ten epochs; the dotted line is 10^{-4} .

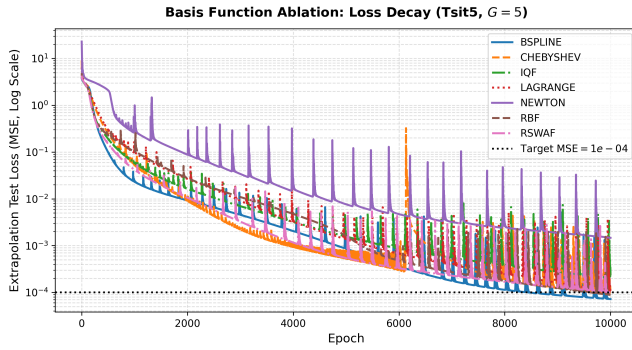


Figure 24: Test MSE during training for the seven bases (Tsit5 solver), logged every ten epochs; the dotted line is 10^{-4} .

Question

What causes SIR's divergence, and is it a scale problem, an optimizer defect, or a modeling gap?

Expected

Measuring the untrained network's own field magnitude against the true dynamics, before any training, should reveal whether the two are mismatched in scale.

Observed

A 19 $\times$ speed mismatch at initialization. Rescaling time removes the divergence entirely; no optimizer or architecture change was needed.

Measuring the learned field magnitude *before any training step* explained the divergence directly. A Glorot-initialised KAN starts with $|f_\theta| \approx 0.203$, against SIR's true field magnitude of 0.011: a 19 $\times$ speed mismatch. To see why this is catastrophic rather than merely inaccurate, note that backpropagation through an explicit solver differentiates roughly $T/\Delta t$ nested nonlinear steps, and the resulting gradient sensitivity grows exponentially as e^{LT} , where L is the Lipschitz constant of $\mathbf{f}_\theta$ and T is the integration horizon. Lotka-Volterra trains at $T = 3.5$ with $|f| \sim 10^0$ and converges. The pendulum trains at $T = 5.0$ with $|f| \sim 10^0$ and converges after the window fix. SIR sits at $T = 50$ with $|f| \sim 10^{-2}$. There the product LT is large enough that training actually fails at the gradient overflow recorded at epoch 11, not at the later epoch 8686.

The optimizer was behaving rationally. With an epoch-0 loss of 3.16×10^2 , collapsing the field to $\mathbf{f}_\theta \rightarrow \mathbf{0}$ reduces the loss to $\approx 10^{-1}$, a 3,000 $\times$ improvement reachable immediately. SIR's natural timescale is the recovery period $1/\gamma = 10$ days. Substituting $\tau = t/10$ gives $dy/d\tau = 10 \mathbf{f}(\mathbf{y})$ over $\tau \in [0, 5]$, which puts SIR in the same regime the other two systems already train in. A separate mass-drift symptom is fixed by projecting the learned field onto the zero-sum subspace, $f_{\text{proj}}(\mathbf{y}) = f(\mathbf{y}) - \frac{1}{n} \sum_i f_i(\mathbf{y})$. This enforces the SIR invariant (Equation 17) as a property of the flow rather than as a penalty on the fit. A third fix, a *vanishing-field gate*, multiplies the learned field by the infected fraction I , so that $I = 0$ is a manifold of equilibria exactly as in Equation 17, where every term carries a factor of I . Unlike the first two fixes, this one assumes something about the physics. Table 15 therefore reports the first two fixes on their own and then with the gate added. Rescaling and projection alone remove the divergence and cut extrapolation RMSE by 4.4 $\times$. The gate gives a further 25 $\times$, and only the full recipe reaches a high extrapolation R^2 . Over this window the true signal varies very little (standard deviation 0.002–0.017), so R^2 is harsh and RMSE is the more informative number.

Other fixes considered. We tested several more interventions. Table 16 records them for completeness, including those we did not adopt.

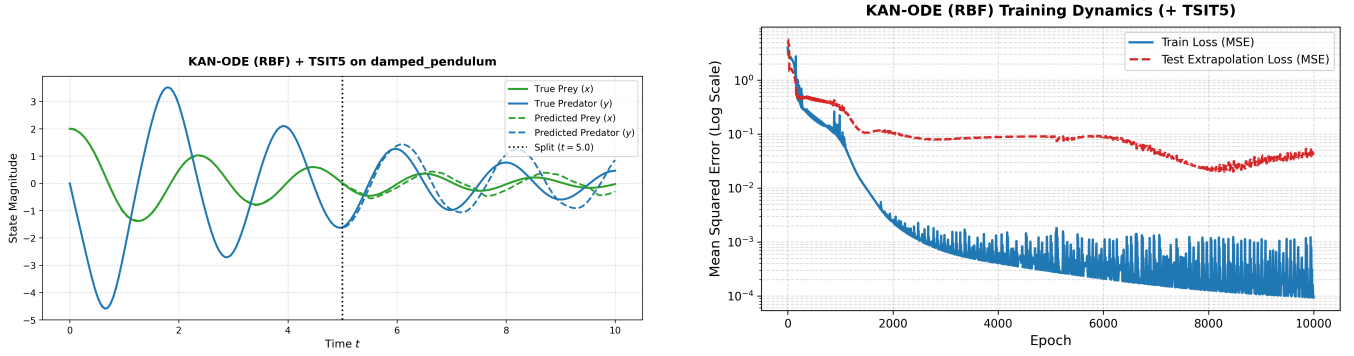


Figure 25: Fixed-pendulum trajectory reconstruction (left) and training/extrapolation loss curves (right), using the extended-window recipe.

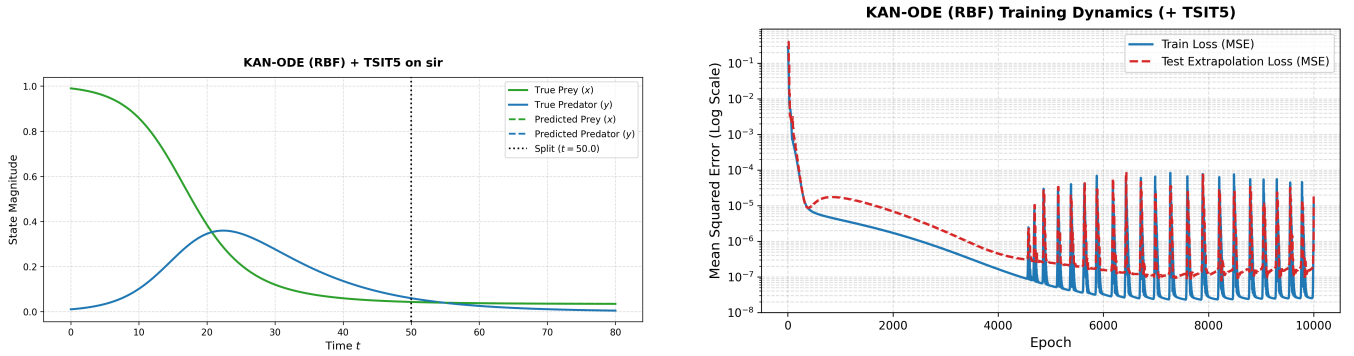


Figure 26: SIR with all three fixes: reconstruction of S and I over the full horizon (left; the plot’s generic legend labels them prey and predator) tracks the true dynamics closely; training and extrapolation loss curves (right) decrease together without the gradient overflow that terminated the original run.

Table 15: SIR before and after the fixes, at the full 10,000-epoch budget. “Rescale + project” applies time rescaling and the conservation projection, which assume nothing beyond Equation 17; “+ gate” adds the vanishing-field gate. Mass error is $\max_t |S + I + R - 1|$.

Metric	Before fix	Rescale + project	+ gate
Training MSE	4.39×10^{-4}	1.08×10^{-5}	2.28×10^{-8}
Extrapolation RMSE	0.0614	0.0140	5.7×10^{-4}
Mass error	0.0045	0.0000	0.0000
Extrapolation R^2	-20.97	-0.145	+0.9981
Max. gradient norm	5.82×10^{18}	92.3	1.19
Final model	NaN (epoch 8686)	finite	finite

Methodological lesson carried into every later study. Both diagnoses came from measuring the learned field’s own magnitude at initialization, before training ever ran. The same init-time measurement reappears verbatim in the epidemic-curve test of Section 4.11.

4.6.1 Extended extrapolation: the KAN advantage widens with horizon. With the fixes in place, we extended the Lotka-Volterra extrapolation horizon from four periods ($t \in (3.5, 14]$) to eight periods ($t \in (3.5, 28]$), using the same trained model with no retraining.

The KAN-ODE’s error is essentially flat across eight periods ($8.83 \rightarrow 8.92 \rightarrow 9.72 \times 10^{-5}$, a 10% rise from the training window to double the original horizon), while the parameter-matched MLP degrades 8.3× over the same extension. The KAN-ODE advantage therefore widens with horizon, from 5.4× at four periods to **40.6×** at eight. This is a materially stronger claim than the near-horizon comparison presented earlier: KAN-ODE error does not accumulate, so the gap grows with every additional period.

4.6.2 Pendulum energy dissipation: an independent physical check. The fixed pendulum recipe captures 94.2% of the true energy dissipation and tracks $E(t)$ to 0.6% inside the training window. However, the strict monotonic-decay invariant ($dE/dt \leq 0$, Equation 18) is violated on 40 of 200 predicted steps. The model is under-damped: the true angular displacement decays to 0.004 by $t = 9.1$, while the predicted trajectory keeps oscillating at 0.237. This energy diagnostic independently confirms the under-damping, expressed as a violated conservation law rather than a trajectory RMSE (Figure 27).

Table 16: Every stability intervention tested, including the rejected ones.

Intervention	Verdict	Evidence
Longer training window (pendulum)	Adopted	R^2 improves from -1.318 to $+0.647$; works for both activations tested
Non-finite gradient guard with automatic abort	Adopted	Caught the divergence of an intermediate SIR recipe (conservation penalty plus loss weighting) at epoch 3055 and preserved a valid checkpoint instead of overwriting it with an invalid one
Time rescaling (SIR)	Adopted	Removes both the divergence and the frozen-field behavior
Conservation via subspace projection (SIR)	Adopted	Mass error reduced from 4.5×10^{-3} to 0.0000 (exact by construction)
Vanishing-field gate (SIR)	Adopted	Extrapolation RMSE reduced a further $25\times$ ($0.0140 \rightarrow 5.7 \times 10^{-4}$); the only fix that assumes a physical property
Identity activation (pendulum)	Speed only	Roughly $1000\times$ faster to fit at the short window, but worse extrapolation and stability at the long window
Standard-deviation loss weighting	System-dependent	Worst-performing pendulum option; superseded for SIR by time rescaling
Conservation via penalty term	Rejected	Wrong mechanism: the training window’s mass was already close to conserved, so the penalty had nothing to correct (mass error still 0.0086)
Widened basis-function grid limits	Rejected	Performed worse than the untouched baseline configuration

Table 17: Extrapolation MSE as the horizon doubles, same trained models.

Model	Training $[0, 3.5]$	Near extrapolation $(3.5, 14]$	Far extrapolation $(14, 28]$	Degradation
KAN-ODE (RBF, 240 parameters)	8.83×10^{-5}	8.92×10^{-5}	9.72×10^{-5}	$1.1\times$
MLP-ODE (SiLU, 252 parameters)	9.50×10^{-5}	4.78×10^{-4}	3.95×10^{-3}	$8.3\times$
MLP-ODE (tanh, default lr/init, pre-fix)	1.08×10^0	1.69×10^0	4.03×10^0	$2.4\times$

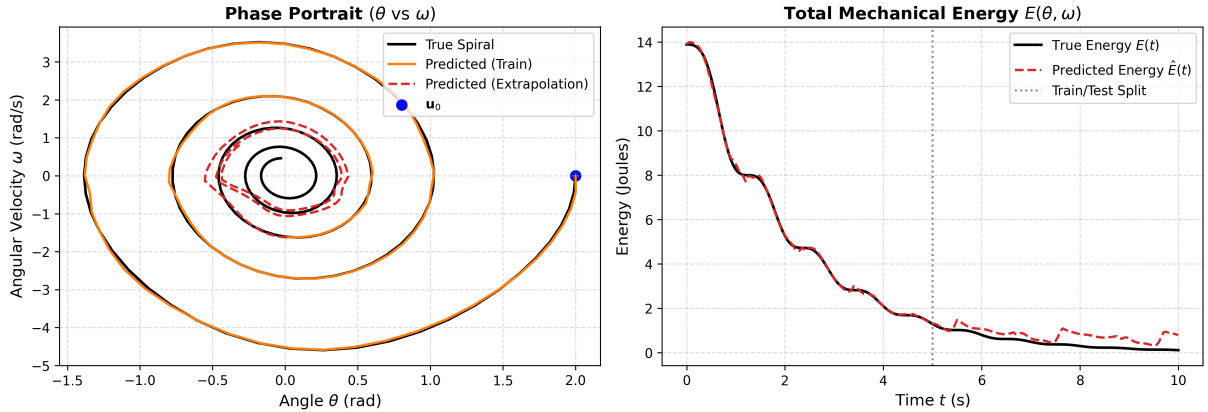
Damped Pendulum ($\mu = 0.5$): Phase Portrait & Energy Dissipation

Figure 27: Fixed pendulum ($\mu = 0.5$, $t_{\text{train}} = 5$): phase portrait (left) and total mechanical energy $E(\theta, \omega)$ (right). Inside the training window the predicted energy follows the true curve closely; past the split at $t = 5$ it stays above the truth and rises on 40 of 200 predicted steps, the under-damping described in the text.

4.7 Adjoint sensitivity vs. direct autograd

Our own implementation documentation adopted direct autograd on the grounds that the adjoint method’s $O(1)$ memory advantage (Section 2.4) does not matter for our short trajectories ($N_t \in [36, 101]$). We test that claim directly on an RTX 3060. The integrator is held fixed (classical RK4 at matched step size on both paths), and a forward-trajectory check confirms the two implementations agree to $\sim 10^{-7}$ relative error, so only the differentiation method varies.

Question

Does adjoint sensitivity’s constant memory cost make it the better choice at our trajectory lengths, as our own documentation assumes?

Expected

Direct autograd should be cheaper in memory at these short lengths, with adjoint’s advantage only appearing once trajectories grow much longer.

Observed

Adjoint already uses less memory at every length tested. The deciding cost is wall-clock, not memory: adjoint is $1.9\text{--}2.1\times$ slower.

Finding. The measured memory data contradicts the premise behind the original choice, but still supports its conclusion. The $O(N_t)$ vs. $O(1)$ scaling in Equation 12 predicts a crossover length below

Table 18: Peak VRAM and wall-clock, direct autograd vs. adjoint sensitivity.

System	Trajectory length N_t	Method	Peak VRAM (MB)	Seconds per 1000 epochs
Lotka-Volterra	36	Direct autograd	18.61	317.5
Lotka-Volterra	36	Adjoint	17.08	672.9
SIR	101	Direct autograd	21.52	984.8
SIR	101	Adjoint	17.12	1871.6

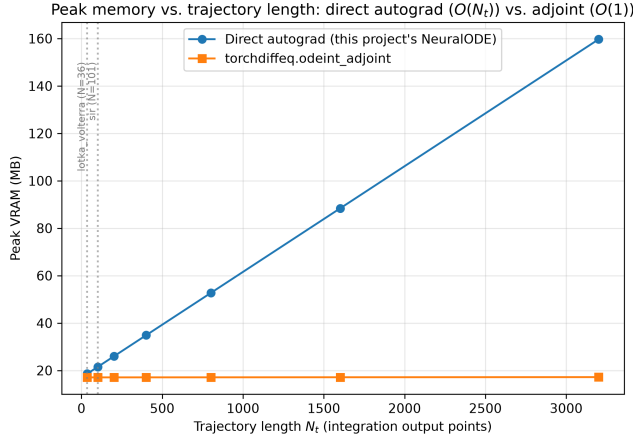


Figure 28: Peak memory vs. trajectory length $N_t \in \{36, \dots, 3201\}$: direct autograd grows linearly (≈ 0.045 MB per output point); adjoint stays flat (0.7% increase over an $89\times$ length increase).

which direct autograd should still win on raw memory, because it avoids the adjoint method’s own bookkeeping overhead. The data shows no such crossover inside the tested range. Adjoint uses *less* memory than direct autograd at every tested length, including the shortest ($N_t = 36$: 17.08 vs. 18.61 MB). As Figure 28 shows, direct autograd’s per-step storage cost is already large enough at our scale to outweigh whatever constant overhead the adjoint bookkeeping carries. Extrapolating the measured linear slope of direct autograd’s memory growth, N_t would need to reach $\approx 225,000$ before it threatens a 12 GB card. For us the deciding factor is wall-clock: solving the continuous adjoint ODE backward in time costs a consistent 1.9–2.1 $\times$ overhead at both tested lengths. That turns a 53-minute Lotka-Volterra run into ~ 112 minutes, for a memory saving that never matters in practice (peak usage is 0.17% of card capacity). Gradient relative error is 2.85×10^{-7} and 4.88×10^{-7} respectively, well inside a 5×10^{-3} tolerance. Both methods agree numerically, so the choice reduces to a resource trade-off, and direct autograd is the faster option for every dataset in our scope ($N_t \leq 201$).

4.8 Gradient-norm dynamics

The benchmark ablation found Forward Euler uniquely unable to reach 10^{-4} training loss within budget, with $28\times$ worse extrapolation MSE than Tsit5 (Table 9). The candidate mechanism we test here: a first-order solver commits $O(\Delta t)$ local error every step,

which could inject measurably noisier gradients and destabilize Adam’s moment estimates.

Question

Does a lower-order solver’s larger truncation error show up as measurably noisier gradients during training?

Expected

Gradient-norm noise should fall as solver order rises, correlating with the accuracy ranking already seen in the benchmark ablation.

Observed

No reliable correlation ($\rho = -0.088$, $p = 0.87$); the ranking is non-monotone and its sign flips depending on an arbitrary warm-up cut. Refuted.

The noise measure is the standard deviation of epoch-to-epoch *relative* gradient change, which is scale-invariant by construction:

$$\eta = \text{std}(\Delta \log_{10} g), \quad (22)$$

$$\Delta \log_{10} g_i = \log_{10} g_{i+1} - \log_{10} g_i.$$

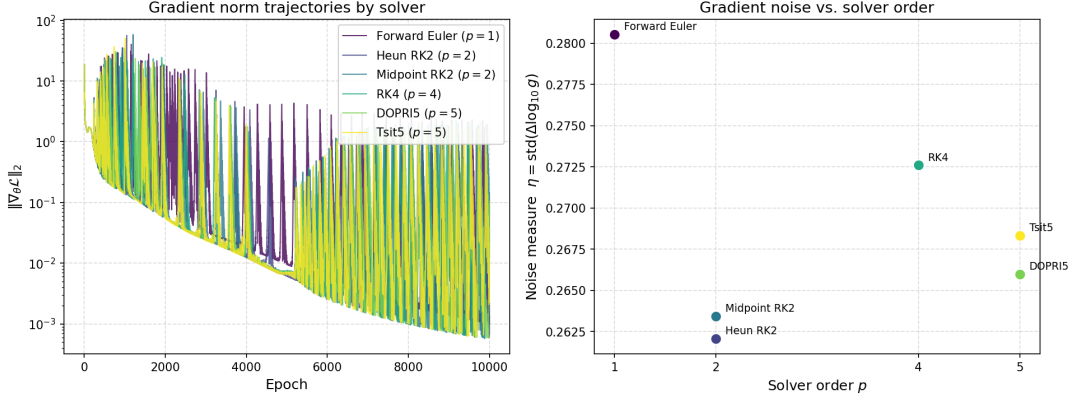
We compare η against solver order p using Spearman rank correlation rather than the more familiar Pearson correlation. Pearson measures how linearly two quantities move together. That presumes the numeric spacing of solver order is meaningful, i.e. that the jump from order 1 to order 2 is comparable in size to the jump from order 4 to order 5. There is no reason to expect η to depend on solver order that linearly. The hypothesis under test is only that noise falls *monotonically* as order rises, which is exactly what Spearman correlation measures, since it compares ranks rather than raw values. It also handles the deliberate tie at $p = 2$ (Heun and Midpoint are both second-order methods) natively, where Pearson would need an arbitrary tie-breaking rule.

Spearman $\rho = -0.088$, $p = 0.87$ ($n = 6$). Three findings jointly refute the hypothesis. First, the effect size is a 7% spread across all six solvers. Second, the ranking is non-monotone (RK4, order 4, is the second-noisiest solver). Third, the sign of ρ flips across warm-up cuts ($-0.088 \rightarrow -0.441 \rightarrow +0.706$ as the first 0, 2000, 5000 epochs are dropped), so no cut reaches significance and the direction of any relationship depends on an arbitrary analyst choice.

Finding. Gradient-norm roughness does not explain Euler’s convergence floor. All six traces are instead dominated by shared, order-independent spike bursts (1176 spike-epochs for Euler, 866 for Tsit5 in the last 4000 epochs, near-identical burst structure). Four checks rule out a numerical defect: zero non-finite gradients or losses across 60,000 logged epochs; loss decreases monotonically in its running minimum for every solver; loss is lower five epochs after a spike in 64.5% of cases; and median loss is elevated on spike epochs (3.90×10^{-4} vs. 1.80×10^{-4} on calm epochs), ruling out a spurious-gradient/still-loss signature. The most plausible reading is edge-of-stability optimizer behaviour. Gradient descent with a fixed step size and no learning-rate schedule is

Table 19: Gradient-noise measure η vs. solver order, six solvers, $n = 6$.

Solver	Order p	Noise η	Best training MSE	Noise rank
Forward Euler	1	0.2805	1.377×10^{-4}	1 (noisiest)
Classical RK4	4	0.2726	9.413×10^{-5}	2
Tsit5	5	0.2683	8.833×10^{-5}	3
DOPRI5	5	0.2660	9.283×10^{-5}	4
Midpoint RK2	2	0.2634	8.857×10^{-5}	5
Heun RK2	2	0.2621	8.315×10^{-5}	6

**Figure 29: Gradient-norm traces, all six solvers: large sustained spike bursts (one to two orders of magnitude above the local median) dominate every trace, including Tsit5, independent of solver order.**

known to settle into a regime where it repeatedly overshoots the walls of a narrowing loss valley as training approaches a sharp minimum. The result is exactly this pattern: a large transient spike followed by fast recovery, rather than either smooth convergence or a genuine divergence. Because this behavior appears with near-identical structure in Euler’s and Tsit5’s traces alike, it is a property of the optimizer and the loss surface’s local geometry, not of the integrator’s truncation error. Euler’s convergence floor is better explained by the earlier systematic-bias hypothesis (Section 4.2): a first-order integrator’s $O(\Delta t)$ local error biases *where* the optimizer settles, without making the path it takes to get there measurably rougher.

4.9 Learnable hybrid basis

The basis ablation (Section 4.3) found B-spline (compact local support, $3.9\times$ RBF’s cost) and RBF (smooth global support, cheap) statistically tied on accuracy. We therefore train the hybrid edge of Section 2.5 (Equation 13) on Lotka-Volterra and on the fixed pendulum (Section 4.6). If one global blend systematically beat both pure bases, the basis choice in Equation 2 would be worth learning rather than fixing by hand.

Question

Can a network do better by learning its own blend of B-spline and RBF, rather than committing to one basis in advance?

Expected

A learned blend should match or beat the better pure basis, by adaptively favoring local support where useful and global support elsewhere.

Observed

Hybrid never beats the better pure basis on either system. On the pendulum, it actively overfits past 3000 epochs in a way pure RBF does not.

Lotka-Volterra: no benefit, real cost. The gate settles into a genuine partial blend, retaining $\approx 12\%$ B-spline, which confirms the mechanism works as designed. Hybrid ties both pure bases on final accuracy while paying roughly the *sum* of their per-epoch compute.

Pendulum: overfitting emerges only past a specific budget. A short probe run at 2000 epochs first surfaced a practical wrinkle. The gate moved in the correct direction (toward RBF) but too slowly, so the network still carried meaningful B-spline contamination at an epoch count where the pure-RBF pendulum recipe was already nearly converged. Giving the gate its own, faster learning rate ($15\times$ the base rate, direction untouched) closed nearly all of that gap at the same probe budget (Figure 31, top), and we carried this into the full run.

At the full 10,000-epoch budget, hybrid matches pure RBF’s training fit almost exactly, but its extrapolation collapses ($R^2 = -2.17$). The monitor loss bottoms out near epoch 2950 (≈ 0.054), then rises to a ≈ 0.4 plateau for the remaining 7000

Table 20: Hybrid basis vs. pure bases, full 10,000-epoch budget.

System / basis	Training MSE	Extrapolation MSE	R^2	Final gate
Lotka-Volterra – RBF (pure)	8.83×10^{-5}	8.92×10^{-5}	1.0000	–
Lotka-Volterra – B-spline (pure)	7.20×10^{-5}	8.38×10^{-5}	1.0000	–
Lotka-Volterra – Hybrid	1.47×10^{-4}	1.78×10^{-4}	0.9999	$\alpha=0.12, \beta=0.88$
Pendulum – RBF (pure)	9.28×10^{-5}	0.090	+0.647	–
Pendulum – Hybrid (10,000 epochs)	9.44×10^{-5}	0.809	–2.17	$\alpha=0.02, \beta=0.98$

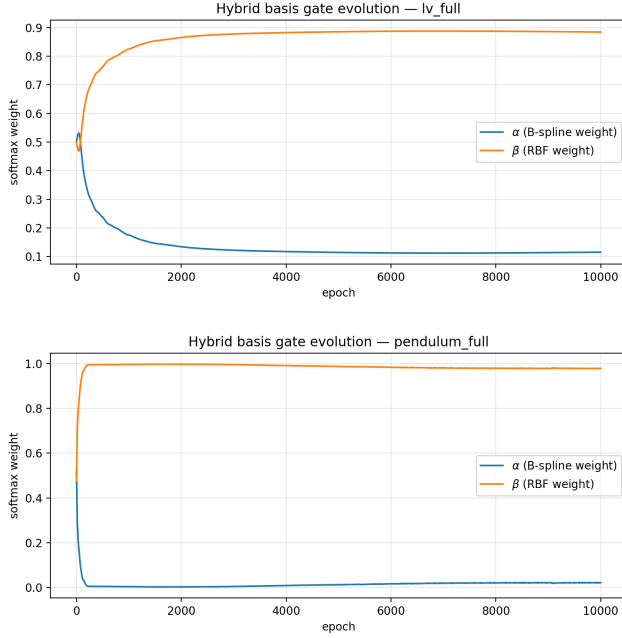


Figure 30: Learned α/β gate evolution: Lotka-Volterra sets into a genuine partial blend (top); the pendulum gate reaches $\approx 99.5\%$ RBF by epoch 250 and ends at 98% (bottom); it settles well before the overfitting onset discussed below.

epochs while training loss keeps improving. This is textbook overfitting, visible directly in the saved training history. The gate settles well before this onset: β reaches ≈ 0.995 by epoch 250, some 2750 epochs before overfitting begins, and pure RBF trained on the identical recipe shows no such pattern at all. Capping training at the pre-overfit optimum (epoch 3000, chosen from the 10,000-epoch run’s own point of minimum monitor loss) shows hybrid *ahead* of pure RBF at the same epoch count (0.055 vs. 0.081 full-horizon MSE) with extrapolation $R^2 = +0.580$, approaching pure RBF’s fully-converged number (+0.647). Figure 32 shows both runs side by side: the loss curves of the full run make the overfitting onset visible, and its phase portrait shows the extrapolation collapsing into a small wrong loop, while the 3000-epoch run still follows the decaying spiral.

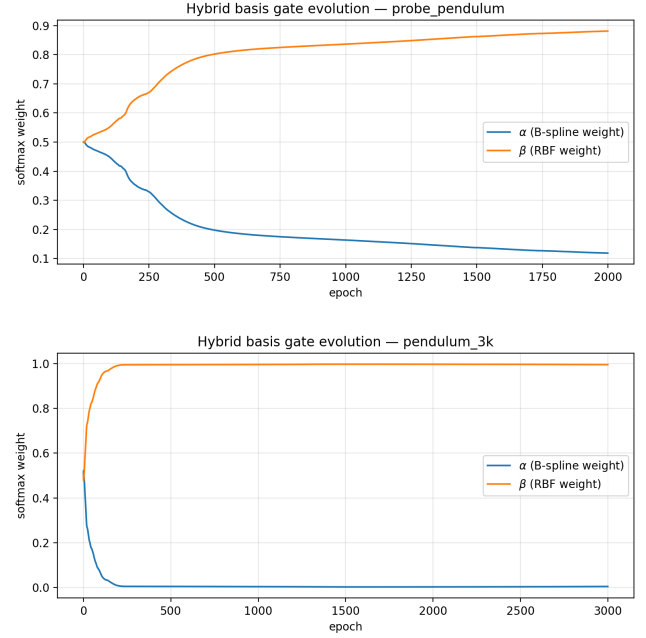


Figure 31: Top: the 2000-epoch probe run’s gate trace, showing the slow drift toward RBF that motivated the gate-learning-rate fix. Bottom: the gate trace for the 3000-epoch pre-overfitting-optimum run discussed below, settling to $\beta \approx 0.996$.

Verdict. The learnable hybrid basis never outperforms the better pure basis on either system at any budget tested. Its gate mechanism works exactly as designed (gradient-driven, converging differently and sensibly per system), but that alone does not buy an accuracy edge. On the pendulum specifically, a network that *starts* as a blend generalizes measurably worse over long training than one that starts pure RBF, despite an almost-identical final gate and matching training accuracy.

4.10 Stiffness–solver map and loss landscape

Numerical stiffness (a system whose fastest and slowest timescales differ by orders of magnitude) is where explicit integrators are classically expected to struggle. We sweep damping ratio $\mu \in \{0.1, 0.5, 1.0, 2.0, 5.0, 8.0\}$ against four explicit solvers {Euler, Midpoint, RK4, Tsit5} [12] on the damped pendulum, at a fixed

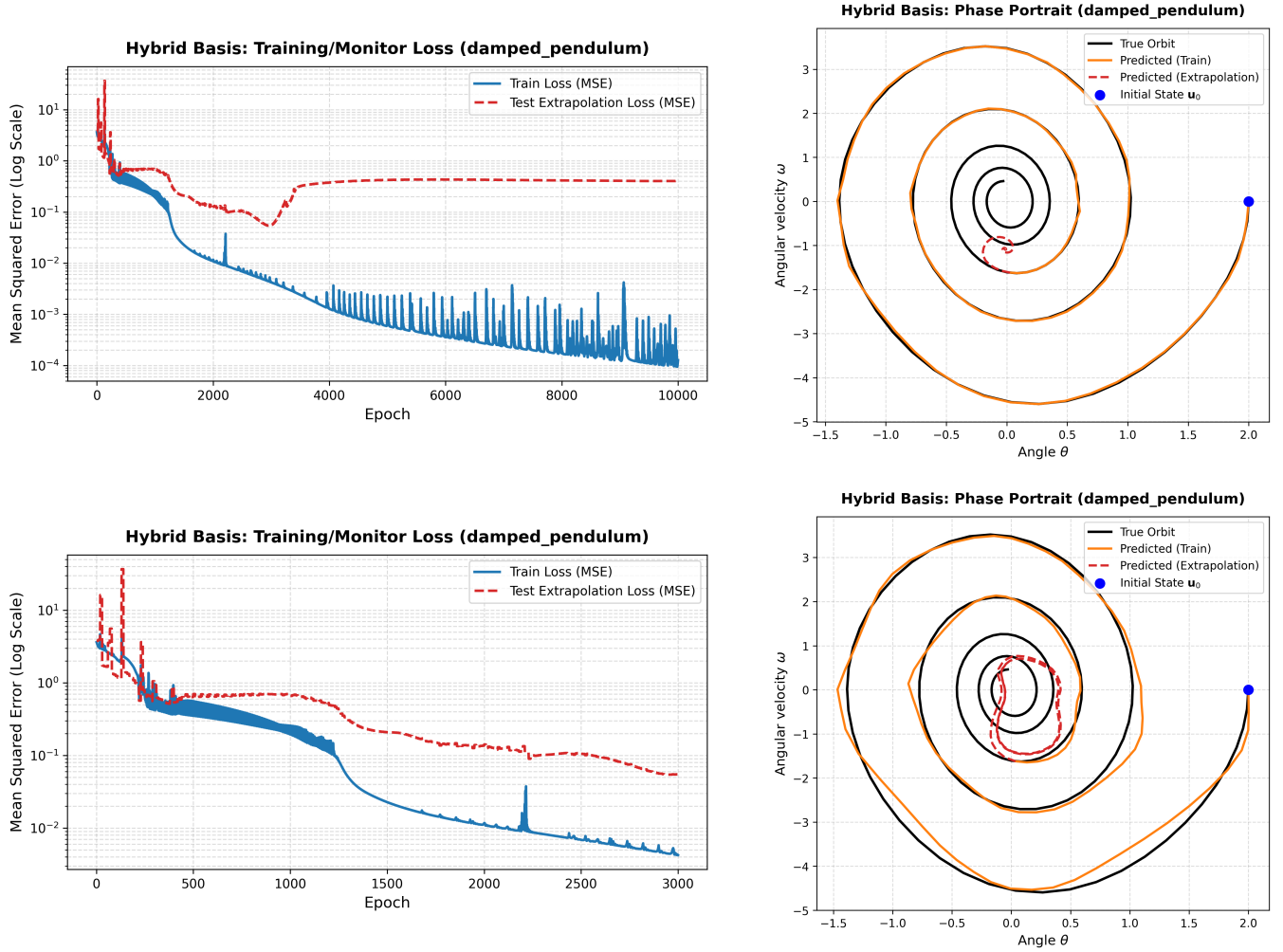


Figure 32: Hybrid basis on the pendulum. Top: the full 10,000-epoch run; training loss keeps falling while the extrapolation (monitor) loss bottoms out near epoch 2950 and climbs back to ≈ 0.4 (left), and the extrapolated orbit collapses into a small wrong loop (right). Bottom: the same recipe stopped at 3000 epochs; train and monitor loss fall together (left) and the extrapolation follows the decaying spiral (right).

step $\Delta t = 0.05$. The goal is to map which (solver, μ) combinations converge and which fail, and to test whether any failure is actually a stiffness effect.

Question

As damping ratio increases the pendulum's dynamics stiffen – do lower-order explicit solvers genuinely fail as stiffness grows?

Expected

Higher damping ratios should push some (solver, μ) combinations outside that solver's classical stability region, producing divergence.

Observed

Linear stability theory predicts zero divergences at every tested μ , and the sweep confirms it. Every observed failure traces to random seed, not stiffness.

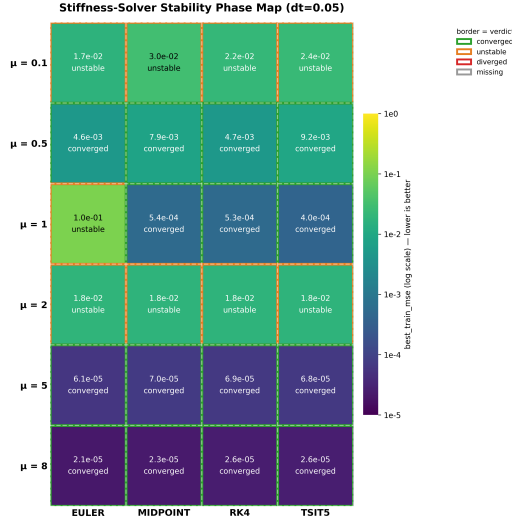
4.10.1 The stability question resolves quickly. Linearizing at equilibrium gives eigenvalues $\lambda = (-\mu \pm \sqrt{\mu^2 - 4g/L})/2$. For every

underdamped $\mu < 2\sqrt{g/L} \approx 6.26$, $|\lambda| = \sqrt{g/L} \approx 3.13$ is constant, so $h|\lambda| \approx 0.157$ at $\Delta t = 0.05$, far inside even Euler's stability region $h|\lambda| \leq 2$ [4]. Even the overdamped case ($\mu = 8.0$, fast eigenvalue $\lambda_{\text{fast}} \approx -7.7$) gives $h|\lambda_{\text{fast}}| \approx 0.385$, still comfortably stable. This predicts, and the sweep confirms, zero divergences across all 24 cells: not one run produced an invalid gradient at any point in training. All four solvers tested are explicit Runge-Kutta methods with a bounded stability region. We implement no implicit solver, so this analysis cannot say whether an implicit method would behave differently at a genuinely stiffer configuration.

4.10.2 What actually drives the failures: initialization. Two anomalies stand out: $\mu = 2.0$ fails identically across all four solvers to two significant figures, and $\mu = 1.0$ fails only for Euler. Retraining Euler at $\mu = 2.0$ with two more seeds isolates the cause (seed 42:

Table 21: Original 24-cell sweep (single seed 42), best training MSE and verdict.

Damping ratio μ	Euler	Midpoint	RK4	Tsit5
0.1	unstable, 1.7×10^{-2}	unstable, 3.0×10^{-2}	unstable, 2.2×10^{-2}	unstable, 2.4×10^{-2}
0.5	converged , 4.6×10^{-3}	converged , 7.9×10^{-3}	converged , 4.7×10^{-3}	converged , 9.2×10^{-3}
1.0	unstable, 1.0×10^{-1}	converged, 5.4×10^{-4}	converged, 5.3×10^{-4}	converged, 4.0×10^{-4}
2.0	unstable, 1.8×10^{-2}	unstable, 1.8×10^{-2}	unstable, 1.8×10^{-2}	unstable, 1.8×10^{-2}
5.0	converged, 6.1×10^{-5}	converged, 7.0×10^{-5}	converged, 6.9×10^{-5}	converged, 6.8×10^{-5}
8.0	converged, 2.1×10^{-5}	converged, 2.3×10^{-5}	converged, 2.6×10^{-5}	converged, 2.6×10^{-5}

Figure 33: Stability verdict heatmap, 24-cell single-seed probe sweep, $\Delta t = 0.05$.

unstable, 1.82×10^{-2} ; seed 1: unstable, 1.68×10^{-2} ; seed 7: **converged**, 7.09×10^{-5}): the trap depends on the random initialization. Extending this to all four solvers $\times$ six $\mu \times$ three seeds (72 cells, checkpoints saved throughout) confirms the pattern generally. Only the two extremes ($\mu = 0.1$, uniformly hard; $\mu \in \{5.0, 8.0\}$, uniformly easy) hold their verdict across every seed. Every transitional $\mu \in \{0.5, 1.0, 2.0\}$ has at least one of three seeds land in a bad local minimum. At $\mu = 1.0$, two *different* seeds fail for two different reasons: seed 1 produces a universally bad initialization shared by all four solvers ($\text{MSE} \approx 9.7\text{--}9.8 \times 10^{-2}$, the same signature as the $\mu = 2.0$ shared trap), while seed 42 carries a second, Euler-specific failure on top of it. Euler’s apparent $\mu = 1.0$ weakness in Table 21 is therefore the sum of two separate effects, one shared and one solver-specific, compressed into a single misleading number.

Consequence for the original heatmap. The 24-cell sweep used one shared seed for all four solvers. All four solvers failing identically at $\mu = 2.0$ reflects four solvers inheriting the same coin flip from a shared initialization, at this step size the seed $\times\mu$ interaction determines outcome far more than solver choice does.

4.10.3 The mechanism: a genuine spurious equilibrium. We evaluated a checkpoint saved for Euler’s trapped $\mu = 2.0$ run at the true

Table 22: Loss-landscape rise, single direction-pair (seed 0).

μ	euler	midpoint	rk4	tsit5
1.0	5.53	9.03	8.99	9.12
2.0	4.23	4.24	4.22	4.20
5.0	8.75	8.75	8.76	8.76
8.0	9.14	9.15	9.12	9.12

equilibrium and at a nearby grid-search minimum. The learned field f_θ has a large nonzero derivative at $(0, 0)$ ($|f_\theta| = 2.74$) and a genuine, stable, spurious equilibrium $0.05\text{--}0.07$ units away ($|f_\theta| = 0.0086$). This holds independently for all four solvers’ own trained models ($|f_\theta(0, 0)| \in [2.65, 2.88]$ across euler, midpoint, rk4, tsit5): all four independently-trained networks converge to essentially the same wrong fixed point.

The full predicted trajectories tell the same story over time rather than in a single snapshot. Figure 35 shows every solver’s predicted angle, angular velocity, and error at $\mu = 2.0$ settling onto the same wrong, generic trajectory; Figure 36 shows the $\mu = 1.0$ case, where only Euler’s predicted angle visibly departs from the true one.

4.10.4 Weight-space loss landscape: an independent measurement. The phase-space plots above describe the learned dynamics. A separate question is how forgiving the *training loss* is around each solution: does a small nudge to the weights leave the fit essentially unchanged, or make it collapse? We measured this by stepping each trained model’s weights along two random directions and re-evaluating training error on a grid, following the visualization technique of Li et al. [9] adapted to KAN-ODE’s 360-parameter weight space. Every cell traces the same qualitative shape (a dip at the trained point that rises moving outward), and we summarize each by its “rise”: the number of orders of magnitude the log-loss climbs by the edge of the grid.

The converged models (rise ≈ 9) sit at the bottom of a narrow, steep basin: stepping even a small distance from the trained weights in either random direction drives the log-loss up by nine orders of magnitude, so the surrounding terrain looks like a sharp valley with the solution pinned at its floor. The trapped cells (bold, rise $\approx 4\text{--}5.5$) sit in a basin roughly half as steep. The terrain around them is closer to a broad, gently sloped plain than a valley, and the same size step barely raises the loss by comparison. This is the opposite of the usual intuition that a flat minimum generalizes better. Here, the flat region is where training settled for a generic,

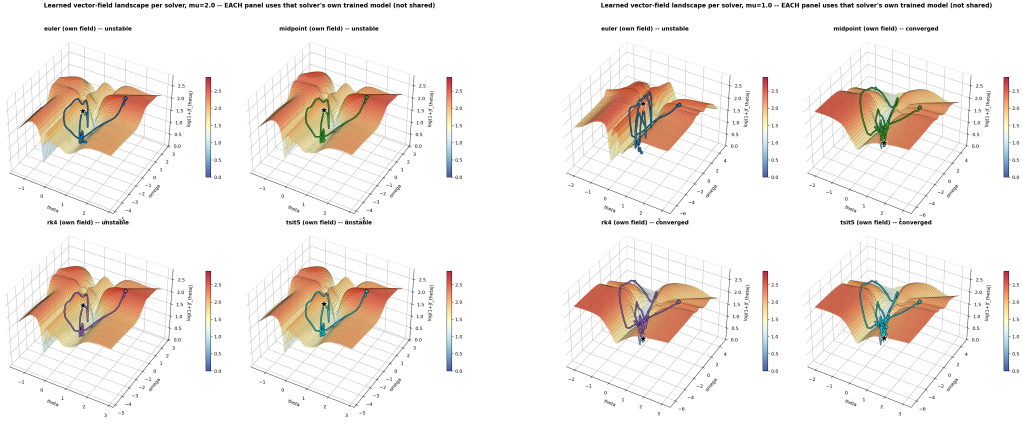


Figure 34: Learned vector-field landscape $|f_\theta(\theta, \omega)|$, per solver’s own trained model. Left ($\mu = 2.0$): every solver’s true equilibrium (star) sits partway up a slope, while a separate, lower point nearby is where each trajectory actually settles. Right ($\mu = 1.0$): Euler’s star sits high on a ridge, well apart from where its trajectory spirals down and rests; for the other three solvers, star and resting point coincide at the valley floor.

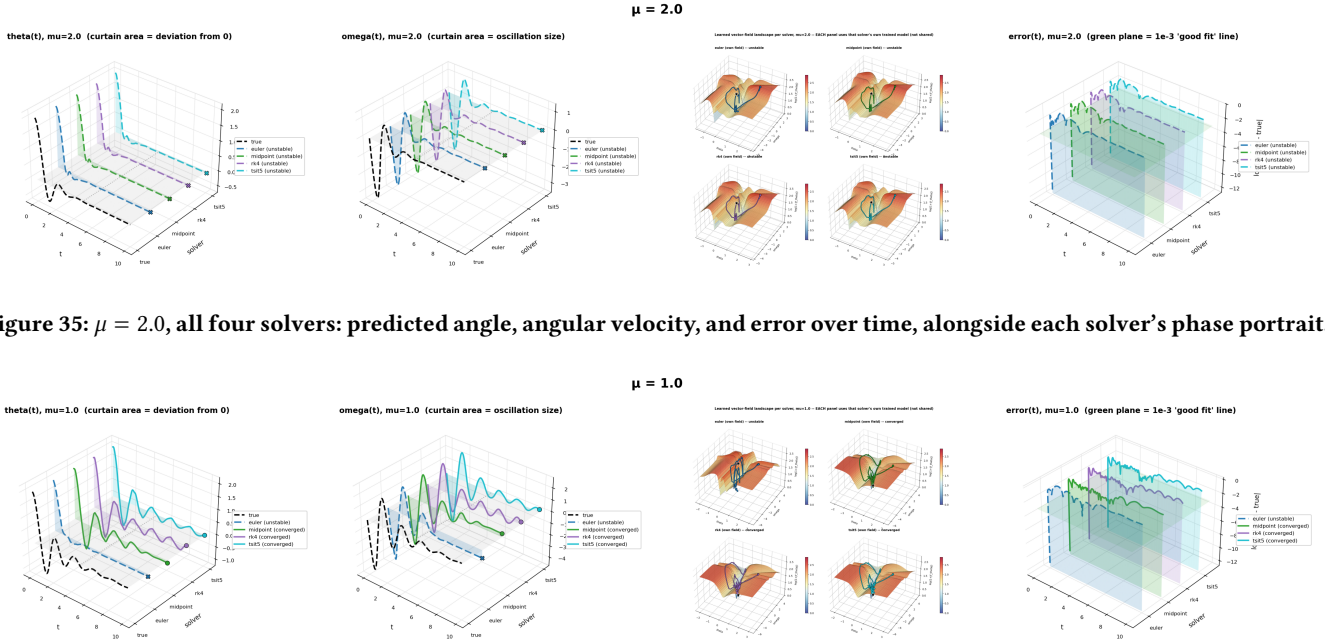


Figure 35: $\mu = 2.0$, all four solvers: predicted angle, angular velocity, and error over time, alongside each solver’s phase portrait.

Figure 36: $\mu = 1.0$, all four solvers: the same panel layout, showing Euler’s trajectory diverging from the true angle while the other three track it closely.

low-effort answer, and the steep, narrow basin is where it actually learned the pendulum’s dynamics.

Figure 37 renders this directly as a surface. The terrain around Euler’s trapped $\mu = 2.0$ solution is a broad, gently rolling plain with no sharply defined bottom. This contrasts clearly with Figure 39’s converged $\mu = 1.0$ solution, which sits at the bottom of a narrow, sharply walled valley; it is the same contrast the rise numbers above report as a factor of roughly two, made visible. Figure 38 shows the trapped $\mu = 1.0$ case under the same solver whose

$\mu = 2.0$ terrain is shown alongside it, confirming the flat terrain is a property of the trapped solution rather than of any one damping value.

Robustness check across three direction-pairs. Repeating the measurement with two more independent random direction pairs per cell (72 measurements total) found one direction-pair acting as a shared outlier across nearly every cell. Because every model shares

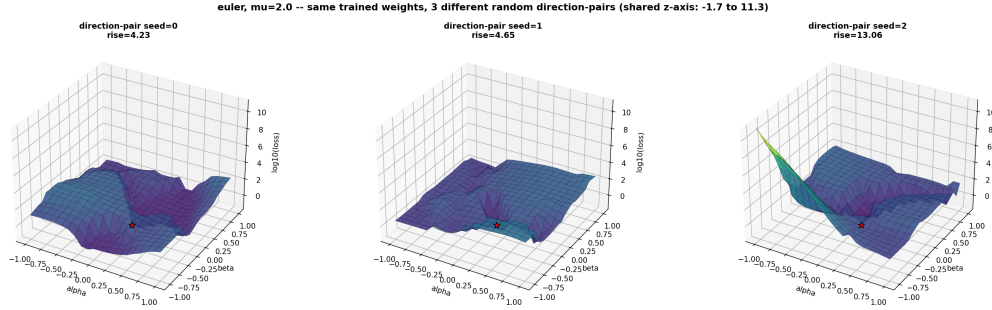


Figure 37: Euler, $\mu = 2.0$ (trapped): the training-loss surface around the trained weights, viewed under three independent random direction-pairs. The basin near the trained point (the marked star) is broad and shallow in all three views; the third panel's steep spike on the far side is the outlier direction discussed below, not the basin itself.

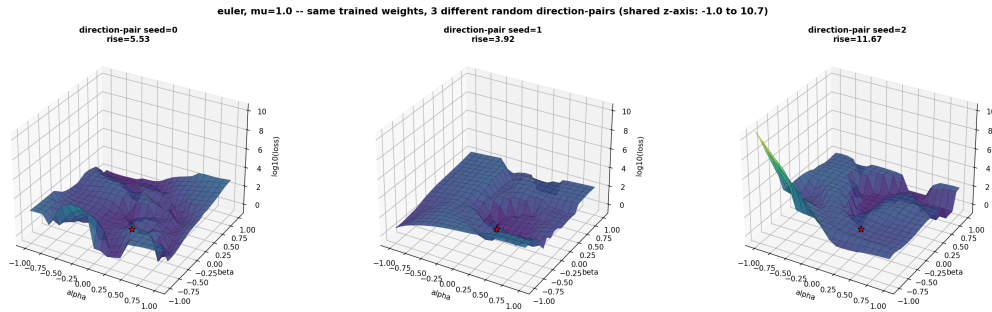


Figure 38: Euler, $\mu = 1.0$ (also trapped): the same broad, shallow basin shape recurs at a different damping value under the same solver.

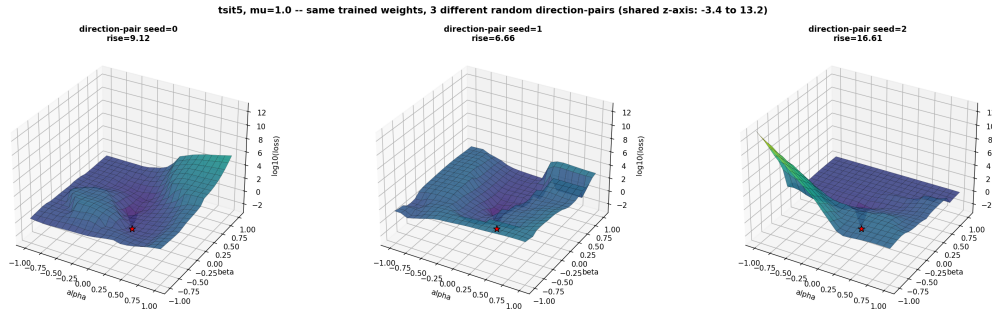


Figure 39: Tsit5, $\mu = 1.0$ (converged): the basin around the trained point is visibly narrower and drops noticeably lower, in every one of the three direction-pairs, than either trapped case above.

the same architecture, one particular raw random draw runs toward a steeper wall in all 24 cells at once, inflating that direction's rise by roughly 2–3 $\times$ everywhere. The size of the measured effect depends on how many direction choices are averaged. The single-seed $\approx 2\times$ gap between trapped and converged cells narrows to roughly 20–30% once all three direction-pairs are averaged, since the outlier direction inflates the trapped cells' mean more than the converged cells' mean. The ranking between cells holds regardless; only the apparent size of the gap changes. The multi-seed average is the figure to trust; the single-seed table above illustrates the shape of the effect at a scale that is easier to see.

The cells trapped by the spurious-equilibrium mechanism ($\mu = 2.0$ across all solvers, and Euler's own $\mu = 1.0$) sit around 20–30% lower on this chart than converged cells: a modest but measured gap rather than an order-of-magnitude separation. $\mu = 0.1$ carries the “unstable” label for an unrelated reason: its trajectory never settles within the training window at all, and its bars sit in the same range as the converged $\mu \in \{5.0, 8.0\}$ cells, occasionally above them. The flatness effect belongs specifically to the spurious-equilibrium trap cases, and does not generalize to every cell the pipeline labels “unstable.”

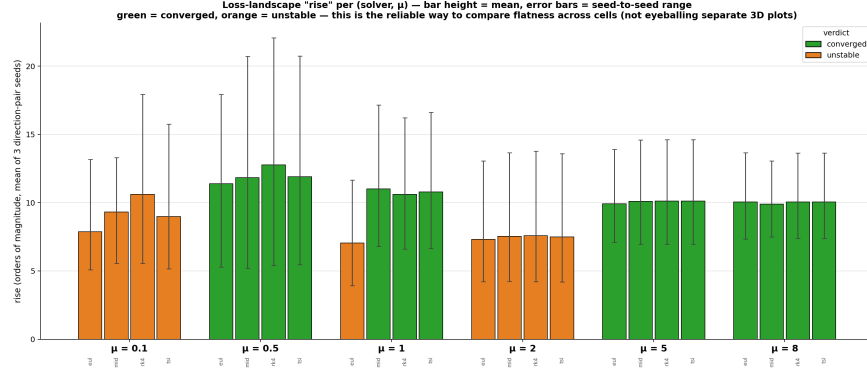


Figure 40: Mean rise per (solver, μ), 3-seed average with seed-to-seed range as error bars, colored by verdict.

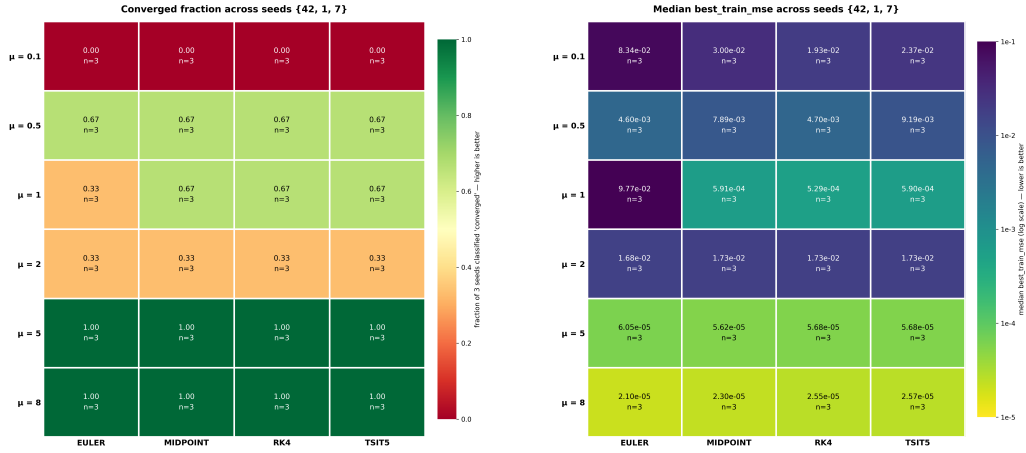


Figure 41: Converged fraction (left) and median training MSE (right) over 3 seeds, all 4 solvers $\times$ 6 μ (72 cells). The two agree everywhere: low converged-fraction cells are exactly the cells with elevated median error.

Verdict. Every divergence-free prediction from linear stability theory held exactly, so nothing in this sweep is a classical numerical-stiffness failure. The mechanism at work is a seed-dependent local minimum in a 360-parameter optimization landscape, producing a genuine, solver-independent spurious equilibrium in the learned dynamics. Three independent lines of evidence converge on this: per-solver trained-weight evaluation, phase-space vector-field visualization, and an independently-measured, direction-robust weight-space loss landscape.

4.11 Sparse regression and transfer to an outbreak curve

Here we compare two very different ways of discovering governing equations from data: sparse symbolic regression and gradient-based neural fitting. We also test separately whether the cross-domain stability fixes of Section 4.6 generalize beyond the mechanistic system that produced them.

4.11.1 Noise robustness: KAN-ODE vs. sparse symbolic regression. We fit SINDy (Section 2.6) to the same noise-corrupted

Table 23: Extrapolation MSE vs. observation noise σ .

Noise level σ	Sparse regression	KAN-ODE	KAN-ODE advantage
0.00	9.90×10^{-2}	8.92×10^{-5}	1110×
0.01	1.58×10^{-1}	1.22×10^{-3}	130×
0.05	1.48×10^{-1}	2.54×10^{-2}	5.8×
0.10	8.89×10^{-1}	1.37×10^{-1}	6.5×

Lotka-Volterra observations that the noise-sweep checkpoints (Section 4.4) were trained on, and score both methods against clean ground truth.

Question

How does a sparse, symbolic method compare to a dense, 240-parameter neural method as observation noise increases?

Expected

Sparse regression’s tiny library should make it more robust to noise; KAN-ODE’s larger parameter count should make it more accurate.

Observed

Both expectations hold, but the KAN-ODE advantage collapses 170× across the sweep, from 1110× at zero noise to 6.5× at $\sigma = 0.1$.

SINDy vs. KAN-ODE robustness to observational noise (Lotka-Volterra, identical data)

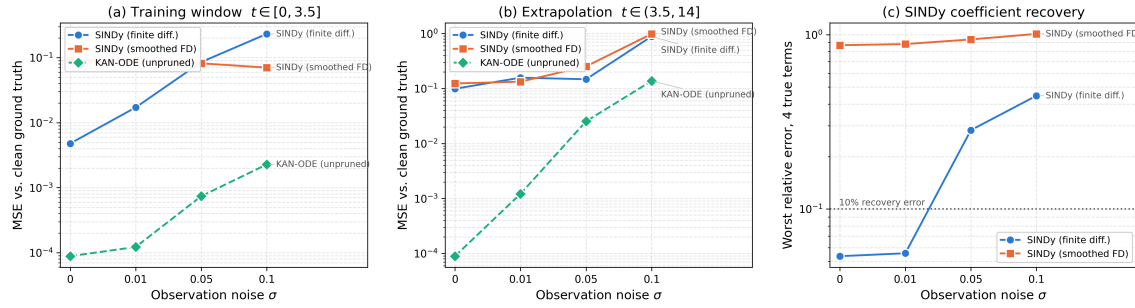


Figure 42: Training and extrapolation MSE vs. noise level, both methods.

KAN-ODE wins on absolute accuracy at every level, but the margin collapses 170 $\times$ across the sweep (sparse regression degrades 9.0 $\times$ from $\sigma = 0$ to 0.1; KAN-ODE degrades 1540 $\times$). The six-term library of sparse regression caps how much noise can move it. The KAN's 240 free parameters fit the noise directly, and the error compounds through the solver. Per-term recovery shows that the bilinear cross-term (xy) is recovered to within 5.7% at every noise level. This is exactly the term a single-layer KAN edge cannot represent directly: each edge is a function of only one input variable, and a layer sums its edges additively. Sparse regression writes xy as a literal library column, while a KAN can only approximate it through composition across a hidden layer. A term-by-term symbolic comparison would therefore compare a coefficient against a distributed representation, so we compare trajectory reconstruction instead.

4.11.2 Edge pruning: testing whether a term-by-term comparison is even possible. Part of the original motivation for a smooth, globally-supported basis such as RBF is that a sparse, pruned KAN edge structure could in principle be read back as a closed-form symbolic law. Sparse regression reaches the same interpretability goal by construction. The base paper gets there by adding the L_1 penalty of Equation 14 to the training loss for symbolic pruning. Every checkpoint we trained used $\lambda = 0$: the plain mean-squared-error term alone, with no sparsity pressure. A KAN-ODE analogue of a sparse coefficient list would instead come from pruning weak edges *post hoc* and reading off what survives. We rank each edge by mean spline-coefficient magnitude and mask the weakest, applied to the existing (never retrained) noise-sweep checkpoints. Removing the two weakest of forty edges raises the noise-free full-horizon MSE from 8.90×10^{-5} to 3.43, a factor of 3.9×10^4 and worse than predicting the constant dataset mean (2.91). No checkpoint was trained with sparsity-inducing regularization, so nothing during training ever asked the network to concentrate its fit onto a few edges. The flat plateau at ≈ 3.5 – 3.8 across all pruning levels of 25% or more is the signature of a model where every edge carries part of the fit. It confirms that the pruning procedure itself works correctly on a network that has no prunable structure to find.

4.11.3 Testing whether SIR's fixes transfer to a non-mechanistic epidemic curve. The outbreak curve of Section 3.1, which no compartmental ODE generated, tests whether the SIR fixes of Section 4.6

generalize by mechanism rather than by coincidence of the SIR system's own structure. The same init-time diagnostic we used to root-cause SIR reproduces an almost identical speed-mismatch signature (18.77 $\times$ vs. SIR's 19 $\times$). This justifies a time rescaling by a factor of 24 *a priori*, before any training.

Question

SIR's three stability fixes were each derived from a property of the SIR generator itself – do they still work on data no compartmental model produced?

Expected

Only the unit-rescaling fix, which assumes nothing about the data's structure, should transfer; the two structural fixes should not.

Observed

One in three predictions was wrong: the vanishing-field gate transfers and is the best single option, despite being predicted inapplicable.

The vanishing-field gate was *predicted* inapplicable: the infected fraction at $t = 0$ is 0.0027, which seemed too small to avoid freezing the field entirely. A control arm overturns that prediction. Gating the learned field by the initial state correctly encodes that the rate of change is proportional to current prevalence. It is the only configuration that reproduces the existence of a peak (day 48 predicted vs. true day 45) from a training window that never observed one.

The extrapolation runaway traces to the train/test split. The default split lands exactly on the infected peak (0% of the decay phase is in the training window), so every model is asked to predict a monotone decay it has never observed. Moving the split just 15 days later collapses the predicted maximum from 6.18 $\times$ the true peak to 0.964 (true maximum = 1.0), turning over at day 47 against a true day 45. The runaway is a property of where the window ends, not of the method fitting the data.

Verdict.

Time rescaling transfers unconditionally because it is a unit change with no structural precondition. The other two fixes are structural priors that transfer exactly insofar as their structure is actually present in the new data: the conservation projection's precondition is confirmable in advance by directly measuring the compartment sum, while the vanishing-field gate's precondition turned out to require running the control arm to settle – argument alone predicted the wrong outcome. That arm cost 27 minutes and overturned the prediction.

Prey-component reconstruction: cleanest vs. noisiest level

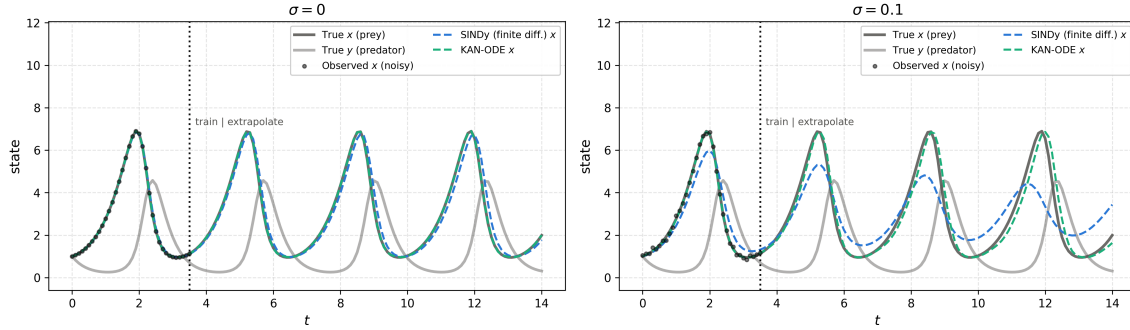


Figure 43: Reconstructed trajectories at $\sigma = 0$ and $\sigma = 0.1$: sparse regression loses amplitude under noise; KAN-ODE keeps amplitude but drifts in phase.

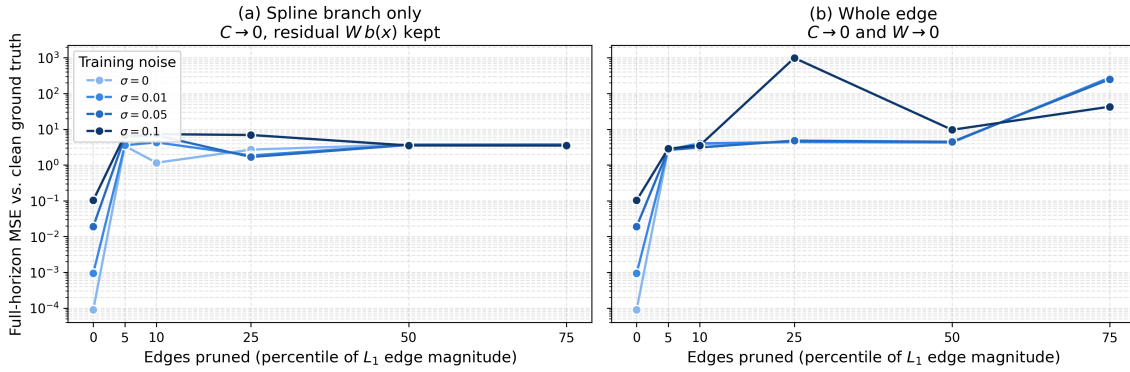
Effect of L_1 edge pruning on the Phase-2 noise-sweep checkpoints (no retraining)

Figure 44: Full-horizon MSE vs. edge-pruning fraction, all four noise levels.

Table 24: Which of SIR’s three fixes transfer to a non-mechanistic curve.

Fix	Precondition	Holds?	Outcome
Time rescaling	None – a unit change	Yes	Removes a 263,049× gradient spike
Conservation via subspace projection	An exact invariant, $\sum y = c$	No ($\sum y$ spans 0.002–1.54)	Inapplicable; 25× worse fit
Vanishing-field gate at the equilibrium point	$y_d = 0$ is a manifold of equilibria	Yes (unexpectedly)	Best single option, 47× better fit

4.12 Epoch-budget validation

Every result in Sections 4.1–4.11 was measured at a fixed 10^4 -epoch budget, a tenth of the base paper’s, for the cost reasons of Section 3.4. Here we ask whether that budget is long enough for the *rankings* to be trusted, and whether any gap to the paper comes from the budget or from an implementation difference.

Question

Do the conclusions drawn at 10^4 epochs survive five times the training budget?

Expected

Absolute errors fall; orderings between configurations hold, since every 10^4 -epoch model had already reached a training loss near 10^{-4} .

Observed

Three of four conclusions hold. The fourth, the one the paper headlines, reverses: at 5×10^4 epochs the parameter-matched MLP extrapolates 6.8× *better* than KAN-ODE.

4.12.1 Design. We chose five configurations to test the conclusions most exposed to the budget: Euler (the one solver short of

Time-scale sweep on the empirical outbreak (2,000-epoch probes)

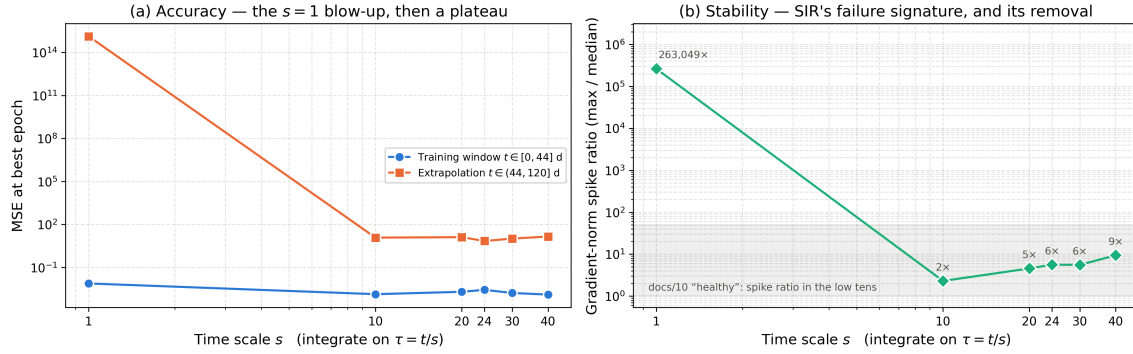


Figure 45: Time-rescaling sweep: accuracy and gradient-spike ratio vs. the rescaling factor.

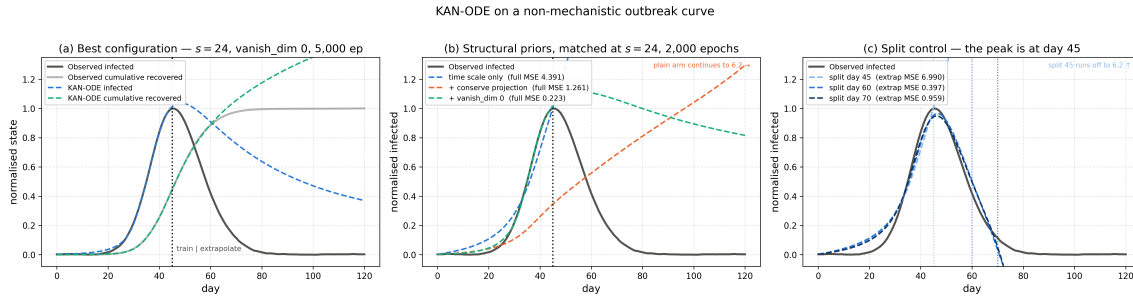


Figure 46: Epidemic-curve fit: default split (runaway), the vanishing-field-gate arm, and the fifteen-day-later split control.

10^{-4} at 10^4 epochs), Tsit5/RBF (the reference), B-spline (the near-tie with RBF), and both MLP baselines. Each was retrained from scratch at 5×10^4 epochs with the recipe of Table 4 unchanged, on cloud CPUs. The B-spline run was cut to 2.5×10^4 epochs because its short calibration run predicted about 35 hours for the full budget. The real run turned out 1.63 $\times$ faster than that calibration (1.544 against 2.52 s/epoch), a reminder that a five-epoch sample is dominated by start-up cost. The batch used 23.2 serial CPU-hours against a planned 17.0, and 10.7 hours of wall-clock with the five jobs in parallel. Before analysis we checked every run's stored metrics against the epoch count of its training log, and reloaded and re-scored every checkpoint; the rollouts reproduce the stored extrapolation MSE to within 0.6%.

We added a sixth configuration later, as a separate follow-up. The implementation audit (Section 4.13) turned up the base paper's own learning rate and initialization scale for its MLP baseline, so we reran both budgets with the identical $[2, 50, 2]$ tanh architecture under those settings instead of our default. We report it alongside the original five throughout.

4.12.2 Results. Every working configuration improves its training MSE by 7.5–52 $\times$ (Table 25); what matters is which orderings survive. Table 26 gives the verdict for each conclusion the earlier sections draw.

The paper-spec MLP is stuck, not slow. With five times the budget its training loss moves by 7% ($1.083 \rightarrow 1.011$), its extrapolation error rises 2.6 $\times$, and R^2 turns negative. The optimizer spends the run in a pathological regime: clipping engages on 99.3% of steps against 2–11% for every other configuration, the median pre-clip gradient norm is 19.8 against 0.03–0.17, and the Lipschitz estimate nearly triples to 104.8, an order of magnitude above every other model. This settles half of the question left open in Section 4.1: whatever is wrong here is a property of this training run, not of the epoch count, since five times the budget only entrenches it. It is not, however, a property of the architecture itself. The next paragraph shows the identical network training cleanly once its optimizer settings are corrected.

Resolving the audit's open optimizer row: the paper's own learning rate and initialization. The base paper's own published code (not distributed with our project, but readable in its public repository) sets two things differently for its MLP baseline than our default: a learning rate of 10^{-2} (five times our 2×10^{-3}) and Glorot initial weights divided by 10^5 , near zero rather than full-scale. Holding architecture, activation, dataset, solver and seed fixed and changing only those two settings, the identical $[2, 50, 2]$ tanh network trains cleanly at both budgets (Table 25, row six; panel f of Figure 47). At 10^4 epochs, training MSE falls from 1.08 to 1.11×10^{-3} and extrapolation MSE from 1.69 to 9.65×10^{-3} ($R^2 : 0.42 \rightarrow 0.997$). At 5×10^4 epochs training MSE reaches 5.43×10^{-5} and extrapolation

Table 25: The five originally planned configurations at 10^4 epochs (Sections 4.1–4.3) and at the extended budget, plus a sixth added after auditing the paper’s own optimizer settings for its MLP baseline (Section 4.13). “Clip” is the fraction of optimizer steps on which gradient clipping at norm 1.0 engaged; $\|\nabla\|_{\text{med}}$ is the median pre-clip gradient norm over the extended run. The sixth configuration ran with no clipping by design (see the paragraph following this table), so those two columns are blank throughout.

Configuration	Epochs	Training MSE	Extrap. MSE	Extrap. R^2	$\hat{L}$	Clip	$\ \nabla\ _{\text{med}}$
Euler / RBF	10k	1.38×10^{-4}	2.51×10^{-3}	0.99914	8.55		
	50k	8.85×10^{-6}	1.93×10^{-4}	0.99993	7.72	4.7%	0.147
Tsit5 / RBF	10k	8.83×10^{-5}	8.92×10^{-5}	0.99997	6.91		
	50k	6.59×10^{-6}	4.02×10^{-5}	0.999986	7.75	4.3%	0.169
Tsit5 / B-spline	10k	7.20×10^{-5}	8.38×10^{-5}	0.99997	8.44		
	25k	9.65×10^{-6}	4.26×10^{-5}	0.999985	7.05	2.2%	0.033
MLP-ODE, SiLU	10k	9.50×10^{-5}	4.77×10^{-4}	0.99984	5.07		
	50k	1.82×10^{-6}	5.90×10^{-6}	0.999998	5.65	11.1%	0.162
MLP-ODE, tanh (paper, default lr/init)	10k	1.08×10^0	1.69×10^0	0.4215	38.2		
	50k	1.01×10^0	4.32×10^0	-0.4841	104.8	99.3%	19.8
MLP-ODE, tanh (paper’s own lr/init)	10k	1.11×10^{-3}	9.65×10^{-3}	0.99669	9.23		
	50k	5.43×10^{-5}	4.31×10^{-4}	0.99985	9.67	—	—

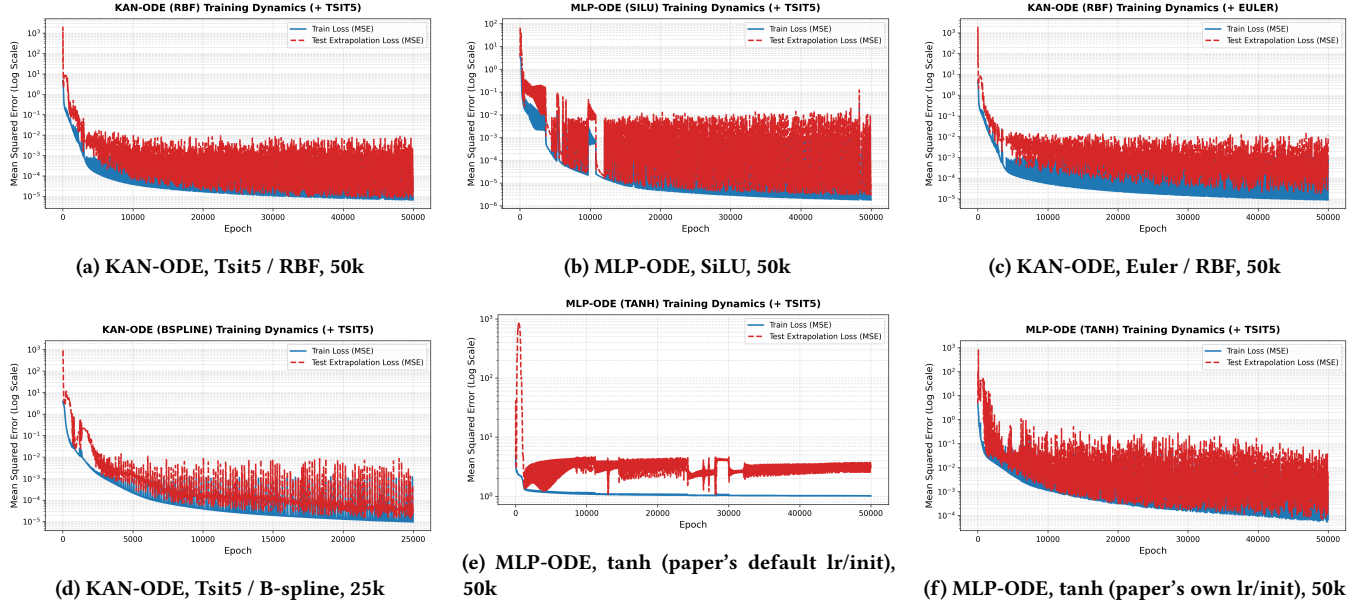


Figure 47: Training and test loss of the six extended runs, as saved by each run. Training loss is logged every epoch; the test loss (full horizon $[0, 14]$) every ten epochs, and it spikes by up to two orders of magnitude throughout, which is why the comparison in the text uses a 500-epoch rolling median rather than raw values. Panel (f) shows the identical $[2, 50, 2]$ tanh architecture of panel (e), retrained with only the learning rate and initialization scale changed to the values in the paper’s own published code.

MSE 4.31×10^{-4} ($R^2 = 0.99985$), against 1.01 and 4.32 under the default settings at the same budget. Neither run needed gradient clipping or hit a single non-finite gradient step. The conclusion Table 7 drew does not survive this: changing only the learning rate and initialization scale, with the exact literal architecture and activation that table showed failing, is enough on its own. What that

factorial actually measured was sensitivity to activation *at our optimizer settings*, not an architecture-level incompatibility with the system.

Against the other two architectures at 5×10^4 epochs, the corrected tanh MLP is still not competitive. Its extrapolation MSE (4.31×10^{-4}) is $10.7\times$ KAN-ODE’s and $73\times$ the SiLU MLP’s, and its learned equilibrium sits 1.72 from the truth, the furthest of the three (Table 12). Its one advantage is conservation: a first-integral

Table 26: Which 10^4 -epoch conclusions survive the extended budget.

Conclusion at 10^4 epochs	At 10^4	At 5×10^4	Verdict
RBF and B-spline are statistically tied Euler is the worst solver	6.1% apart 28.1× Tsit5	5.6% apart (25k) 4.8× Tsit5	Holds. The basis choice can be made on cost. Holds , with a much smaller margin: part of the gap was under-training.
The paper-spec MLP fails because of its architecture, not its budget	train 1.08, $R^2 = 0.42$	train 1.01, $R^2 = -0.48$	Budget part holds; architecture part reversed. More epochs alone do not help, but the identical architecture trains under the paper’s own learning rate and initialization (below): $R^2 = 0.9998$ at this budget.
KAN-ODE extrapolates better than a parameter-matched MLP	KAN 5.35× better	MLP 6.82× better	Reverses. The advantage is a budget effect.

drift of 0.06% is the smallest of the three (KAN 0.08%, SiLU-MLP 0.21%), though its on-orbit field error (2.7%) is the largest. It also falls short of the paper’s own reported target. The paper states that its MLP reaches a training loss of 3.0×10^{-5} within 10^5 epochs; this run’s training loss crosses 10^{-4} at epoch 39,695 but had not reached 3.0×10^{-5} by epoch 50,000. That trend is consistent with, but does not confirm, the paper’s own budget for this baseline.

The KAN advantage over the MLP reverses. At 10^4 epochs, from training losses 7% apart, the KAN-ODE extrapolates 5.35× better than the SiLU MLP, the result that reproduces the paper’s accuracy claim. At 5×10^4 epochs the MLP’s extrapolation MSE is 5.90×10^{-6} and the KAN’s 4.02×10^{-5} : the MLP is 6.82× better, far outside the single-seed noise that separates RBF from B-spline. The checkpoint analyses of Section 4.5 agree on every measure. At 50k the MLP has the smaller on-orbit field error (0.6% against 1.1%), the smaller first-integral drift (0.4% against 0.9%), an equilibrium closer to the truth (0.68 against 1.65 away), and a lower far-horizon error (maximum 2.4×10^{-2} against 6.5×10^{-2} on (14, 28]). The crossover is late. Smoothing the logged full-horizon MSE of Figure 47 with a 500-epoch rolling geometric median, the two models are level at 2.5×10^4 epochs (1.7×10^{-4} against 2.0×10^{-4}), and the MLP stays ahead for good only from about epoch 48,500. The MLP is also the faster learner at every threshold: in the extended runs it reaches 10^{-4} training MSE at epoch 5225 against the KAN’s 5695, and 10^{-5} at 13,815 against 31,412.

The honest reading is that KAN-ODE has a strong *early* inductive bias for this smooth periodic problem. Within a fixed, moderate budget it produces the better extrapolator, but it is not the better function class once both models are trained to convergence. Neither model learns the conservative structure of the system (Figure 18); the MLP simply fits the training orbit’s phase more precisely in the end.

4.13 Comparison with the base paper

This subsection gathers, in one place, how our results line up with the base paper [8]. We first compare our recipe with the paper’s stated methodology field by field (Table 27), so that no difference is silently absorbed into “the paper trained longer”. The table uses the settings recorded in our own planning notes and code, and marks as open every field those do not determine. One such field, the MLP baseline’s optimizer settings, we later resolved by reading the paper’s own published code, which is how the follow-up runs of Section 4.12 came about.

The efficiency claim. The paper reports its KAN-ODE reaching a training loss of 2.6×10^{-5} in 10^4 epochs and its MLP needing 10^5 epochs for 3.0×10^{-5} . Our extended runs locate both targets directly. Our KAN-ODE reaches 2.6×10^{-5} at epoch 13,632, within 1.4× of the paper’s KAN. Our SiLU MLP reaches 3.0×10^{-5} at epoch 8,585, nearly 12× sooner than the paper’s MLP. The literal [2, 50, 2] tanh MLP never gets below 1.0 under our default optimizer settings, but under the paper’s own learning rate and initialization it crosses 10^{-4} at epoch 39,695 and had not reached 3.0×10^{-5} by epoch 50,000: a trend consistent with, but short of confirming, the paper’s own 10^5 -epoch target. Allowing for the open loss-normalisation row, the KAN side of the efficiency claim is reproduced. The tenfold advantage over the literal MLP baseline, however, depends on that baseline’s optimizer settings as much as on its architecture, since the identical network trains once those settings are corrected.

The accuracy claim. At the paper’s own 10^4 -epoch budget we reproduce the extrapolation advantage: KAN-ODE is 5.4× better than a parameter-matched MLP (Section 4.1), and the gap widens to 40.6× at twice the horizon (Section 4.6). At five times that budget the ranking reverses and the MLP is 6.8× better (Section 4.12), so we read the paper’s accuracy claim as a statement about moderate training budgets rather than about the two function classes at convergence.

The three fixed design choices. None of the paper’s three fixed choices turns out to be load-bearing on this benchmark. Tsit5 is not needed for accuracy: above order two, every solver lands within 3.8% of the others, and Midpoint matches Tsit5 at 29% of the cost (Section 4.2). The switch from B-spline to RBF is justified on cost rather than accuracy: the two are statistically tied at both budgets, with RBF 3.9× cheaper per epoch (Section 4.3). And the adjoint method’s memory saving, while real at every trajectory length we tested, never matters at our scale, while it costs 1.9–2.1× in wall-clock (Section 4.7).

Wall-clock. Our absolute training times are not comparable with the paper’s. Our integration loop runs uncompiled in PyTorch on CPU, whereas the paper compiles its loop in Julia’s SciML ecosystem (Table 5). All our timing comparisons are therefore made only between our own runs, on the same hardware.

Outcome of the comparison. The 10^4 -epoch budget gives trustworthy *rankings* for the solver and basis ablations: the RBF/B-spline tie and Euler’s last place both survive five times the budget. It does

Table 27: Field-by-field comparison with the base paper [8].

Field	Base paper	Ours	Verdict
KAN architecture	[2, 10, 2], $G = 5$, 240 parameters	identical	Matches.
Edge basis	Gaussian RBF, $\exp(-r^2/2h^2)$	identical, after fixing a missing factor $\frac{1}{2}$	Matches. The fix predates every reported run.
MLP baseline	[2, 50, 2], tanh, 252 parameters	identical, plus a working [2, 14, 8, 8, 2] SiLU variant	Matches literally, and trains under the paper’s own optimizer settings (below).
Integrator	adaptive Tsit5 (Julia)	fixed-step Tsit5, $h = 0.05$	Explained: the fixed step’s own error is 4.7×10^{-14} in MSE (Table 9).
Gradients	continuous adjoint	direct autograd	Explained: they differ in memory and time, not in the loss minimised (Section 4.7).
Framework	compiled Julia / SciML	interpreted PyTorch, CPU	Explained: wall-clock is not comparable across the two (Section 3.4).
Epoch budget	10^4 (KAN), 10^5 (MLP)	10^4 , and 5×10^4 in Section 4.12	Explained: Section 4.12.
Optimizer, learning rate (MLP baseline)	Adam, 10^{-2} , Glorot init $\div 10^5$, found in the paper’s own published code	Adam, 2×10^{-3} by default, Glorot gain 1.0, clip 1.0	Resolved: this mismatch, not the architecture, caused the earlier training failure. The paper’s own KAN learning rate remains unconfirmed.
Loss normalisation	not recorded	mean over points and components	Open; affects absolute loss values only.

not for the KAN-versus-MLP comparison, which reverses: the paper’s extrapolation advantage is an early-training effect under our protocol, and its speed advantage over the literal MLP baseline depended on that baseline’s default optimizer settings, not on its architecture. Corrected, the identical network trains, though it still does not close the gap with either KAN-ODE or the SiLU MLP within our budget. Every result is single-seed.

4.14 Discussion

Eight patterns recur across studies that were designed independently and address unrelated questions. That they recur is itself informative about how this class of model should be studied going forward.

A trained Neural ODE models its training orbit and its solver, not the vector field. The numerical analysis of Section 4.5 found this in three independent ways. First, the Euler-trained network is an accurate model of the inverse modified equation, and no model transfers across solver families (Figure 11). Second, every trained model, KAN or MLP, at either budget, has an attracting limit cycle through the training data around a spurious unstable equilibrium 1.2–1.7 away from the true one, where Lotka-Volterra has a neutral centre. Third, the field is accurate only in a thin tube around the observed orbit (Figure 19). Low trajectory error on the training orbit’s continuation is therefore compatible with a qualitatively wrong phase portrait. Checks that probe the field directly, such as locating its equilibria, integrating from new initial conditions or evaluating a known invariant, are cheap once a checkpoint exists. They should be part of the protocol whenever a learned field is to be read as physics.

Architecture comparisons are budget-dependent. The KAN-versus-MLP ranking reversed between 10^4 and 5×10^4 epochs (Section 4.12), while the solver and basis rankings did not. The difference is instructive. The solver and basis comparisons are between models of the same function class, and they were decided by truncation error or locality, properties that do not change with training time. The architecture comparison is between two function classes that converge at different rates, and a fixed budget measures the rates as much as the limits. A claim that one

architecture “generalises better” should state the budget, or better, report the error against training time for both.

A reasoned hypothesis still requires a control arm. Three separate studies advanced a specific, mechanistically plausible hypothesis and then tested it directly: gradient-norm roughness as the cause of Euler’s convergence floor (Section 4.8), SiLU’s representational limit as the cause of the pendulum’s early failure (Section 4.6), and the predicted inapplicability of the vanishing-field gate to non-mechanistic data (Section 4.11). Each hypothesis had a clear argument behind it before any run was launched, and each was overturned by the run itself. In all three cases the result we report is the outcome of the control arm; the original argument survives only as the motivation for running it. This is not a coincidence of three unlucky guesses. It reflects the difficulty of reasoning about a system where the integrator, the optimizer, and the learned function interact inside the same backpropagation graph. The standard practice of ablating one variable while holding others fixed rests on an assumption of near-independence that does not hold here.

A single seed and a single train/test split account for results that first looked like properties of the solver or the method. The stiffness-map failures at $\mu \in \{0.5, 1.0, 2.0\}$ (Section 4.10) and the epidemic-fit extrapolation runaway (Section 4.11) both appeared, on first inspection, to indict a solver or a modeling approach. Retraining with additional seeds in the stiffness-map case, and shifting the train/test split by fifteen days in the epidemic case, resolved both into an artifact of one run’s random initialization or one window boundary. Neither observation survives as a claim about solver order or about KAN-ODE’s capacity once the seed or the split is varied. Every quantitative result we report remains $N = 1$. The stiffness study’s small extension to $N = 3$ was enough to dissolve a headline finding, which underscores how much of the rest of our work rests on the same single-seed foundation.

Measuring a quantity before training is more informative than tuning after a failed run. The SIR divergence (Section 4.6), the epidemic-curve fit (Section 4.11), and the stiffness map’s spurious equilibrium (Section 4.10) were each root-caused by evaluating the model’s own initial field, or its trained field at a candidate equilibrium, and comparing that number against the system’s true

scale. In every case this produced a specific, falsifiable prediction (a rescaling factor, a fix that should or should not transfer, an equilibrium location) that a short follow-up run confirmed or refuted directly. A hyperparameter sweep would instead have required an open-ended search.

Training accuracy and generalization quality are decoupled. The hybrid basis matches pure RBF’s training loss to 1.7% at 10,000 epochs yet collapses in extrapolation ($R^2 = -2.17$ vs. $+0.647$, Section 4.9). The stiffness map’s trapped models (Section 4.10) reach training losses of 1.8×10^{-2} , qualitatively acceptable on a casual inspection, while their learned field has a large nonzero derivative at the true equilibrium. And the epidemic-curve model (Section 4.11) produces a plausible training fit while predicting an infected peak six times the physical maximum. In all three cases, a reader who judged the model solely by training-window MSE would reach the wrong conclusion. We adopted the evaluation protocol of Section 3.3 (scoring strictly on extrapolation, never on training loss) precisely because we anticipated this decoupling. These studies go further: they show that configurations differing by two orders of magnitude in extrapolation quality can produce numerically indistinguishable training fits. The number to watch is always the one computed outside the training window, and no training-window metric we tested reliably predicts it.

Cost and accuracy form a clear Pareto frontier. Across the solver ablation (Section 4.2, Figure 12), the basis ablation (Section 4.3), and the hybrid-basis study (Section 4.9), a consistent pattern emerges: the cheapest methods that work (Midpoint at $1/3$ the function evaluations of Tsit5; RBF at $1/3.9$ the wall-clock of B-spline) match the accuracy of their more expensive alternatives to within single-seed noise. Adding cost, whether more function evaluations per step, more basis evaluations per edge, or a second basis evaluated in parallel, does not buy a measurable accuracy improvement on any system tested. The hybrid-basis result makes this particularly sharp: the blend pays roughly the sum of both bases’ per-epoch compute for two extra parameters and never recovers the cost in accuracy.

These KAN edges do not produce interpretable symbolic structure without explicit encouragement. The pruning experiment (Section 4.11) found that removing the two weakest of forty edges degrades a noise-free model to worse than a constant predictor, a factor of 3.9×10^4 . Every edge carries part of the fit, the magnitude ranking is close to arbitrary, and no edge can be identified with a specific term in the governing equation. This is a direct consequence of training without sparsity-inducing regularization. It means the interpretability advantage that motivated the original choice of a smooth, globally-supported basis function remains a hypothesis rather than a demonstrated property of our trained models. A future retraining with explicit edge-level regularization would be needed to test whether sparse, readable edge functions emerge when the training loss actively asks for them.

5 Conclusion and Future Work

We reproduced the KAN-ODE recipe of the base paper on the Lotka-Volterra system with every integrator and basis implemented from its defining formula and verified numerically. We

then extended the recipe to a damped pendulum and an SIR epidemic model, diagnosed and fixed two genuine cross-domain training failures, ran five follow-on studies that each isolate one question, and re-tested the budget-sensitive conclusions at five times the training budget. Every quantitative claim traces to a saved training record, a re-scored checkpoint or a regenerable figure (Appendix A).

5.1 Key takeaways

- **Solver order matters only up to two.** All six integrators satisfy their order conditions to round-off (37 rooted trees checked). Above order two the solver does not change accuracy at the training step, and Midpoint matches Tsit5 within 0.3% at 29% of the wall-clock.
- **The network learns the field its solver needs.** The Euler model matches the inverse modified equation $\mathbf{f} + \frac{h}{2} J\mathbf{f}$ (cosine 0.92, magnitude ratio 1.04), and models lose one to four orders of magnitude when integrated with a solver from another family.
- **Trained models learn an orbit, not the vector field.** Every trained model, KAN or MLP, is an attracting limit cycle around a spurious unstable equilibrium, where Lotka-Volterra has a neutral centre; the field is accurate only in a thin tube around the training orbit.
- **Locality, not conditioning, separates the bases.** B-spline and RBF are tied at both budgets, and the global polynomials extrapolate 4–5× worse. Extrapolation MSE scales as $12.9 \sigma^{2.02}$ with observation noise.
- **KAN-ODE’s advantage over the MLP is budget-dependent.** At 10^4 epochs KAN-ODE extrapolates 5.4× better than a parameter-matched MLP (widening to 40.6× at twice the horizon); at 5×10^4 epochs the MLP is 6.8× better. The paper’s literal tanh MLP trains once given the paper’s own learning rate and initialization.
- **Measure before you tune.** SIR’s failure traced to a 19× unit-scale mismatch at initialization. A change of time units and a conservation projection remove the divergence, and a vanishing-field gate brings extrapolation R^2 from -20.97 to $+0.9981$.
- **Follow-on studies.** Direct autograd is the right default at our scale (the adjoint saves memory but costs 1.9–2.1× in time); gradient roughness does not explain Euler’s floor ($\rho = -0.088$); a learnable spline/RBF blend never beats the better pure basis; stiffness-map failures trace to the random seed; and sparse regression is far more noise-robust than KAN-ODE (9× vs. 1540× degradation) but less accurate at every noise level.

5.2 Limitations

- **Single seed.** Every configuration was trained with one random seed. The stiffness study, where going from one seed to three dissolved a headline finding, shows how much this can matter.
- **Single train/test split.** Most results use one split per system; on the epidemic curve, moving the split by fifteen days changed the conclusion.
- **Timing comparability.** Our loop runs uncompiled in PyTorch, so our wall-clock numbers compare our own configurations

with each other but not with the base paper’s Julia implementation.

- **Explicit solvers, fixed steps.** We implemented only explicit Runge-Kutta methods at a fixed step; we cannot say how an implicit or adaptive solver would behave on a genuinely stiff system.
- **No sparsity pressure.** All models were trained with $\lambda = 0$ in Equation 14, so our pruning results say nothing about how interpretable a sparsity-regularised KAN-ODE would be.
- **Synthetic epidemic curve.** The outbreak curve is synthetic rather than observed case data.

5.3 Future work

The most valuable next experiment is multi-seed replication of the KAN-versus-MLP crossover, since it is the one conclusion that reversed with budget. Two cheaper follow-ups come straight out of the numerical analysis: an affine input normalisation that uses the whole first-layer grid on positive-valued systems, and training from several initial conditions, the most direct way to ask a Neural ODE for the vector field rather than a single orbit. Retraining with the L_1 penalty of Equation 14 would test whether readable edge functions emerge when the loss asks for them, and fitting observed outbreak data would complete the epidemic transfer test.

Acknowledgments

We thank our supervisor, Niaz Rahman (Lecturer, Department of CSE, BUET), for his guidance throughout this project.

References

- [1] Blealtan. 2024. efficient-kan. <https://github.com/Blealtan/efficient-kan>.
- [2] Steven L. Brunton, Joshua L. Proctor, and J. Nathan Kutz. 2016. Discovering governing equations from data by sparse identification of nonlinear dynamical systems. *Proceedings of the National Academy of Sciences* 113, 15 (2016), 3932–3937.
- [3] Ricky T. Q. Chen, Yulia Rubanova, Jesse Bettencourt, and David Duvenaud. 2018. Neural Ordinary Differential Equations. *Advances in Neural Information Processing Systems* 31 (2018).
- [4] Germund Dahlquist. 1963. A special stability problem for linear multistep methods. *BIT Numerical Mathematics* 3, 1 (1963), 27–43.
- [5] Ernst Hairer, Syvert P. Nørsett, and Gerhard Wanner. 1993. *Solving Ordinary Differential Equations I: Nonstiff Problems* (2 ed.). Springer, Berlin.
- [6] Alan A. Kaptanoglu, Brian M. de Silva, Urban Fasel, Kadierdan Kaheman, Andy J. Goldschmidt, Jared L. Callahan, Charles B. Delahunt, Zachary G. Nicolaou, Kathleen Champion, Jean-Christophe Loiseau, J. Nathan Kutz, and Steven L. Brunton. 2022. PySINDy: A comprehensive Python package for robust sparse system identification. *Journal of Open Source Software* 7, 69 (2022), 3994.
- [7] Diederik P. Kingma and Jimmy Ba. 2015. Adam: A Method for Stochastic Optimization. *International Conference on Learning Representations* (2015).
- [8] Benjamin C. Koenig, Suyong Kim, and Sili Deng. 2024. KAN-ODEs: Kolmogorov-Arnold Network Ordinary Differential Equations for Learning Dynamical Systems and Hidden Physics. *Computer Methods in Applied Mechanics and Engineering* 432 (2024), 117397. arXiv:2407.04192.
- [9] Hao Li, Zheng Xu, Gavin Taylor, Christoph Studer, and Tom Goldstein. 2018. Visualizing the Loss Landscape of Neural Nets. In *Advances in Neural Information Processing Systems*, Vol. 31.
- [10] Ziming Liu, Yixuan Wang, Sachin Vaidya, Fabian Ruehle, James Halverson, Marin Soljačić, Thomas Y. Hou, and Max Tegmark. 2024. KAN: Kolmogorov-Arnold Networks. *arXiv preprint arXiv:2404.19756* (2024).
- [11] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Advances in Neural Information Processing Systems*, Vol. 32.
- [12] Ch. Tsitouras. 2011. Runge-Kutta pairs of order 5(4) satisfying only the first column simplifying assumption. *Computers & Mathematics with Applications*

Table 28: What each figure and table produced after training is computed from.

Figure or table	Computed from
Tables 9, 10, 11, 6, 5, 25	saved final metrics of each run
Table 8	the implemented solver tableaux
Figure 9, truncation column of Table 9	the true Lotka-Volterra field
Figure 10	solver tableaux and the Tsit5 checkpoint
Figure 14, κ_2 in Table 10	the implemented basis functions
Table 12, Figure 20	trained checkpoints
Figure 18	trained checkpoints
Figure 11, modified-equation analysis	checkpoints of the solver sweep
Figure 19	trained checkpoints
Figure 22	RBF and B-spline checkpoints
Table 13, budget-validation crossover	per-epoch training histories
Figure 12	saved final metrics of each run
Figure 17 (right)	runs of the noise sweep
Figure 21	trained checkpoints
Diagrams (Figures 1, 2, 6, 3, 15)	drawn from the implemented coefficients and shapes

- 62, 2 (2011), 770–775.
- [13] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, et al. 2020. SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods* 17 (2020), 261–272.

A Reproducibility

A.1 How the numbers and figures are regenerated

Every training run saves its final metrics, a per-epoch training history, and two checkpoints: the one with minimum training loss and the final one. A single command-line entry point trains any configuration (model type, basis, solver, sub-steps, learning rate, epoch budget and seed), and a separate script re-scores any saved checkpoint against the ground truth it was trained on. The extended runs of the budget validation were launched as independent cloud jobs with the same entry point.

Everything reported after training is produced by an analysis pipeline that retrains nothing. Its first step re-integrates every noise-free $\Delta t = 0.1$ checkpoint and checks that the rollout reproduces the run’s stored extrapolation MSE (the worst relative deviation is 6.2×10^{-3} , from cloud-trained checkpoints re-scored on a different CPU). The remaining steps compute the order conditions, work-precision data, stability regions, Newton equilibria, first-integral drift, cross-solver matrix, field-error maps and training milestones reported in Sections 4.2–4.5 and 4.12. Table 28 lists what each derived figure and table is computed from.

A.2 The integrator implementation

The fixed-step Tsit5 step and the trajectory integrator, with docstrings shortened. Every other method follows the same pattern with its own tableau.

Fixed-step Tsit5 step
<pre>def step_tsit5(func, t, y, dt): # 6 of 7 stages: the 7th (FSAL) stage has TSIT5_B[6] == 0.0 k = [] k.append(func(t, y)) for stage_idx in range(1, 6): t_stage = t + TSIT5_C[stage_idx] * dt</pre>

```

y_stage = y
for j in range(stage_idx):
    y_stage = y_stage + dt * TSIT5_A[stage_idx][j] * k[j]
    k.append(func(t_stage, y_stage))
y_next = y
for j in range(len(k)):
    if TSIT5_B[j] != 0.0:
        y_next = y_next + dt * TSIT5_B[j] * k[j]
return y_next

```

Trajectory integrator

```

def odeint(func, y0, t, method="tsit5", substeps=1):
    step_fn = STEP_SOLVERS[method.lower()]
    trajectory = [y0]
    curr_y = y0
    for i in range(len(t) - 1):
        dt_sub = (t[i + 1] - t[i]) / substeps    # h = dt / substeps
        for s in range(substeps):
            curr_t = t[i] + s * dt_sub
            curr_y = step_fn(func, curr_t, curr_y, dt_sub)
            trajectory.append(curr_y)
    return torch.stack(trajectory, dim=0)

```

Because the loop is ordinary PyTorch, reverse-mode differentiation of the loss records every stage evaluation; this is the “direct

autograd” gradient method of Section 4.7, and the reason training cost is proportional to the number of function evaluations.

A.3 Order-condition check

The verification of Table 8 needs only the tableau. Rooted trees are represented as sorted tuples of their subtrees; for a tree $\tau = [\tau_1, \dots, \tau_m]$ the elementary weight vector is $\mathbf{g}(\tau) = \prod_k A \mathbf{g}(\tau_k)$ (elementwise, with $\mathbf{g}(\bullet) = \mathbf{1}$), the density is $\gamma(\tau) = |\tau| \prod_k \gamma(\tau_k)$, and the condition is $\mathbf{b}^\top \mathbf{g}(\tau) = 1/\gamma(\tau)$.

Order-condition check

```

def elem_weight(t, A):
    g = np.ones(A.shape[0])
    for child in t:
        g = g * (A @ elem_weight(child, A))
    return g

def gamma_of(t):
    return order_of(t) * math.prod(gamma_of(c) for c in t)

for q in range(1, 7):
    # 1, 1, 2, 4, 9, 20 trees
    devs = [b @ elem_weight(t, A) - 1.0 / gamma_of(t) for t in trees[q]]

```